

Data-Driven Dynamical Systems

Dynamical systems with machine learning

Lecture notes

based on the video lectures by Steven L. Brunton
(University of Washington), with one lecture by J. Nathan Kutz

Personal study notes of Alessio Martini

These notes are a personal, non-commercial study aid derived from the YouTube playlist *Dynamical Systems (with Machine Learning)* by Steve Brunton, which accompanies the book by S. L. Brunton and J. N. Kutz, *Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control* (Cambridge University Press; chapters at <http://databookuw.com>). Lecture content is paraphrased and re-derived from the (automatic) transcripts of the videos, not transcribed; the Matlab demonstrations are rewritten in Python. Any mistakes are mine. All rights on the original lectures and book belong to their authors.

How to use these notes

The playlist has 24 videos, recorded over several years, that are not numbered as lectures. These notes group them by topic: each chapter collects the videos on one subject, and its sections follow the videos. Every chapter opens with a **Sources** box listing the videos (each one a link) and the matching sections of the book by Brunton and Kutz. The order is the order of the playlist, with one exception: the long review *Topics in Dynamical Systems*, the last video of the playlist, is Chapter 2, because it is the background for everything else. Part I reviews dynamical systems and how to simulate them; Part II covers the two families of regression methods that learn models from data (DMD and SINDy); Part III is the operator-theoretic view (Koopman, HAVOK) that ties them together.

Boxes.

- **Definitions** (blue), **theorems and propositions** (red), **exercises** and **remarks** are marked with a coloured bar on the left. A definition box titled *New concepts* marks the first time a concept appears.
- **Key formulas** boxes collect what to remember.
- **Intuition** boxes give the reading of a method in a physicist's language.
- **Application** boxes say where a method is used in science and engineering.
- **Case study** boxes follow a data set through a lecture (Lorenz, the cylinder wake, measles, ...).
- **Python** boxes rewrite the Matlab demonstrations of the lectures in Python (numpy, scipy); every listing has been run.
- **Reading** boxes point to the book.
- **Beyond the core** blocks (grey bar on the left, smaller type) hold material that can be skipped on a first reading.

The right column. The page is laid out for a screen: the text is always on the left and a column on the right holds **examples**, **course notes**, **history** boxes and **asides** (curiosities), your **notes** with their **answers**, and **feedback** on your solutions.

Video timestamps. A tag such as [▶ V1.2 @ 3:10](#) means: chapter 1, video 2 of that chapter, at minute 3:10 of the video. Every tag is a link to the video at that second.

Your annotations. While reading, write directly in the chapter files. The macro `\note{...}` prints a comment in the right column next to the text where it is written: a doubt, an idea, a reminder. When a doubt is answered, the note and its answer

become one box in the right column. Requests about the notes themselves are collected in the chapter *Requests about these notes*, which also lists how read and unread notes are marked.

Requests about these notes

Your margin notes are of two kinds: notes on the *content* (a doubt, an idea about a mechanism), which stay next to the text they refer to, and requests about the *notes themselves* (layout, boxes, conventions), which do not depend on where they were written. The second kind is collected here, with what was done about it, so that the chapters read more smoothly. Write new ones anywhere with `\note{...}` and they will be moved here, or add them directly at the end of this list.

How the margin notes are marked.

- `\note{...}`: a *new* note, not yet read. “Read my notes” means these, and only these.
- A note that has been answered becomes one box with the question on top and a dark yellow **ANSWER** bar below it (`\begin{answer}{question}[topic] ... \end{answer}`). A note with several questions gets one ANSWER bar per question. A question asked a second time gets a short answer that links to the first one.
- `\note*{...}`: a request that has been carried out (green DONE tag).
- `\note+{...}`: a comment that has been read and needs no answer (blue SEEN tag).
- `\feedback{...}`: a comment on one of your own solutions (indigo).

The list

DONE

All chapters, exercises. esercizi brevi/a sé stanti lungo il testo, quelli collegati o complessi alla fine

WHAT WAS DONE

Standalone exercises now follow the section they practise (right before the next heading); connected or integrative exercises stay together at the end of the chapter, in the original order. In chapters where none were left, the final Exercises section was removed.

Contents

How to use these notes	i
Requests about these notes	iii
I Dynamical systems	1
1 Overview and Anatomy of a Dynamical System	2
1.1 The anatomy of a dynamical system	3
1.2 From first principles to data	5
1.3 Challenges of modern dynamical systems	6
1.4 What models are for	9
1.5 Techniques and the plan of the course	10
2 Fundamentals: Fixed Points, Linearization, Invariant Manifolds, Bifurcations, Chaos	12
2.1 From linear to chaotic	12
2.2 Fixed points and linearization	13
2.3 The Duffing oscillator and invariant manifolds	14
2.4 Bifurcations	16
2.5 Discrete-time systems and the flow map	17
2.6 Chaos	18
3 Simulating Continuous and Discrete Dynamics	20
3.1 Simulating the Lorenz system	20
3.2 From continuous to discrete time	22
3.3 The logistic map	24
II Learning models from data: regression	27
4 Dynamic Mode Decomposition	28
4.1 The DMD algorithm	28
4.2 Extensions, applications, and a toy example	31
4.3 DMD in practice: the cylinder wake	33
4.4 Compressed sensing and DMD	34

5	Sparse Identification of Nonlinear Dynamics	37
5.1	The SINDy algorithm	37
5.2	The cylinder wake: rediscovering the mean-field model	40
5.3	Extensions: parameters, forcing, delays, control	40
5.4	PDE-FIND: discovering partial differential equations	42
III	The Koopman operator	45
6	Koopman Spectral Analysis	46
6.1	The Koopman operator	47
6.2	Two examples: closure and non-closure	48
6.3	Data-driven approximations	49
6.4	Koopman and control	51
6.5	Continuous spectrum and multiscale systems	52
7	Koopman Observable Subspaces and Control	54
7.1	Koopman invariant subspaces that contain the state	54
7.2	Koopman operator optimal control	55
7.3	When finite closure is impossible	57
8	Hankel Alternative View of Koopman (HAVOK)	59
8.1	Chaos and the Koopman view	60
8.2	Delay coordinates and the Koopman reading of the Hankel matrix	60
8.3	Chaos as an intermittently forced linear system	61
8.4	Real systems: magnetic field reversal and measles	64
8.5	The structure of the HAVOK model	64

Part I

Dynamical systems

Chapter 1

Overview and Anatomy of a Dynamical System

SOURCES FOR THIS CHAPTER

Video	Length	Content
V1.1 Data-Driven Dynamical Systems Overview	21:43	The general system $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, t, \mathbf{u}; \boldsymbol{\beta})$; from first principles to data; challenges (nonlinearity, unknown dynamics, dimension, chaos, noise, multiscale, uncertainty); uses (prediction, design, control, interpretability); techniques (regression, neural networks, genetic programming); Koopman coordinates; physics in the learning.
V1.2 The Anatomy of a Dynamical System	17:53	The same anatomy in more detail: state, dynamics, time, control, parameters, measurements; the challenges as optimization problems; interpretable and generalizable models.

Transcripts [materials/transcripts/C01-V01.txt](#), [C01-V02.txt](#). **Book** Brunton & Kutz, *Data-Driven Science and Engineering*, Chapter 7 introduction and Sec. 7.1 “Overview, motivations, and challenges”. The second video (recorded about three years after the first) repeats much of the first: the two are merged here, and each point is tagged with the video where it is said best.

Motivation

These two short videos open the series. They contain no algorithm: they fix the language used in all the following chapters (state, dynamics, control, parameters, measurements) and they set the agenda, that is, which problems of dynamical systems are still open and why data and machine learning might help with them. The thesis is simple. For three centuries dynamical systems were *derived*: one wrote down Newton’s law, a Lagrangian or a Hamiltonian, and obtained the equations of motion. For many systems of interest today (the brain, the climate, an epidemic, a financial market) there is no such first-principles equation, but there is more and more *data*. The program of the course is to *learn* the dynamical system from the data, in a form that is still interpretable [[► V1.1 @ 0:22](#)] [[► V1.1 @ 4:50](#)].

The reading for a physicist: nothing in this chapter is new mathematics, but the point of view is not the textbook one. In a course on classical mechanics the equations are given and one studies their solutions; here the solutions (the data) are given and one looks for the equations.

1.1 The anatomy of a dynamical system

Why data, why now. Two things happened at the same time [► V1.1 @ 0:43]:

1. an explosion of data, from experiments, simulations and historical records, in climate science, neuroscience, disease modelling, fluid dynamics and almost every other field;
2. much better algorithms in machine learning, regression and optimization.

Their confluence makes it possible, for the first time, to discover and characterize dynamical systems purely from data [► V1.1 @ 1:03].

The general form. Dynamical systems are our models of the evolving world: how fluids mix, how a rocket flies and lands, how a disease spreads on a network [► V1.2 @ 0:06]. They are fundamentally tied to *time*: they describe how things change and co-evolve in time [► V1.2 @ 1:28]. The general object of the course is a system of coupled ordinary differential equations [► V1.1 @ 2:26][► V1.2 @ 1:49]

$$\frac{d}{dt}\mathbf{x}(t) = \mathbf{f}(\mathbf{x}(t), t, \mathbf{u}(t); \boldsymbol{\beta}), \quad \mathbf{y}(t) = \mathbf{g}(\mathbf{x}(t), t), \quad (1.1)$$

together with the measurements $\mathbf{y}$ that we can actually take of it. Each symbol is one “organ” of the anatomy.

Definition 1.1 (New concepts: the anatomy of a dynamical system). In (1.1):

- (a) the **state** $\mathbf{x} \in \mathbb{R}^n$ is the minimal set of numbers that describes the system uniquely at one instant; n is the *dimension* of the system [► V1.2 @ 1:49];
- (b) the **dynamics** $\mathbf{f} = (f_1, \dots, f_n)$ is a vector-valued function: f_j gives the rate of change $\dot{x}_j$ of the j -th state variable. Since it assigns a vector $\mathbf{f}(\mathbf{x})$ to every point $\mathbf{x}$ of state space, it is also called a **vector field** [► V1.1 @ 3:49][► V1.2 @ 2:52];
- (c) t is **time**. If $\mathbf{f}$ depends explicitly on t the system is **non-autonomous**, otherwise it is **autonomous** [► V1.1 @ 2:26];
- (d) the **control** (or *actuation, input*) $\mathbf{u}$ collects the variables we can manipulate actively, the “knobs” [► V1.2 @ 4:14];
- (e) the **parameters** $\boldsymbol{\beta}$ are all the other constants on which $\mathbf{f}$ depends, which we do not control but want to understand [► V1.2 @ 4:55];
- (f) the **measurements** (or *outputs*) $\mathbf{y} = \mathbf{g}(\mathbf{x}, t)$ are what we can actually observe; in general $\mathbf{y}$ has far fewer components than $\mathbf{x}$ [► V1.2 @ 5:15].

The state. The state is the minimal description: for a pendulum the angle θ alone is not enough, because the same angle can be crossed with different speeds; the state is $\mathbf{x} = (\theta, \dot{\theta})$ [► V1.2 @ 2:09]. The state can be enormous: for the stock market it might be the value of all the stocks (and possibly more information), for the weather the conditions on a one-kilometre grid covering the Earth [► V1.2 @ 2:31].

The dynamics as a vector field. Every n -th order ODE can be written as a first-order system by enlarging the state, as in the pendulum; so (1.1) is fully general. The picture to keep in mind is the vector field: at every point of state space $\mathbf{f}$ is an arrow saying how

fast and in which direction the state moves [► V1.1 @ 4:09]. A solution is a curve everywhere tangent to the arrows (fig. 1.1). Often $\mathbf{f}$ comes from Newton's second law or something like it [► V1.2 @ 3:12].

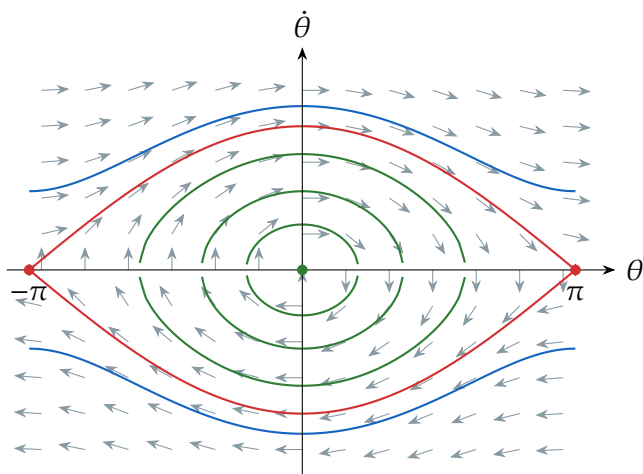


Figure 1.1: State space of the pendulum $\dot{\theta} = \omega$, $\dot{\omega} = -\sin \theta$ ($g/L = 1$). Grey: the vector field $\mathbf{f}$ (arrows of equal length, only the direction is shown). Green: oscillations around the stable equilibrium; red: the separatrix through the unstable (upright) equilibria; blue: rotations, the pendulum goes over the top. Each curve is a level set of the energy $E = \frac{1}{2}\omega^2 - \cos \theta$.

Time. The explicit dependence on t matters, because in many systems the rules themselves change in time [► V1.2 @ 3:33]. The equations that describe your brain while you watch a lecture are different from those that describe it while you sleep; the weather and the climate change; we may be actively changing the dynamics of a system, as with the ocean [► V1.2 @ 3:53].

Control versus parameters. Both $\mathbf{u}$ and β are “inputs” of $\mathbf{f}$, but they play opposite roles. The control $\mathbf{u}$ is what we act on, possibly in real time; the parameters β are given by the situation. For an aircraft, β would contain wind speed, humidity, altitude and weight, and $\mathbf{u}$ the deflection of the flaps and of the elevators [► V1.1 @ 3:28]. We want to understand the explicit dependence of $\mathbf{f}$ on β , because a small change of a parameter can produce a large qualitative change in the dynamics, a *bifurcation* (here it is only named; bifurcations are treated in Chapter 2) [► V1.2 @ 4:55].

Measurements. We rarely have access to the full state [► V1.2 @ 5:15]. One cannot record every neuron of a brain; at best a small subset, or a crude proxy such as an EEG or an ECG [► V1.2 @ 5:36]. Hence the second equation in (1.1). The data from which models are learned are samples of $\mathbf{y}$, t , $\mathbf{u}$ and possibly β [► V1.1 @ 4:29].

ASIDE: Where the names come from

Dynamical comes from the Greek *dýnamis*, “power, force”: dynamics was the part of mechanics that studies motion together with the forces that cause it (Leibniz coined *dynamica* in the 1690s). The word survived the forces: a “dynamical system” today is any rule that makes a state evolve in time. *Autonomous* is from the Greek *autós* “self” and *nómos* “law”: the system obeys its own law, with no external clock. *Actuation* is from the medieval Latin *acturare*, “to put into action”.

Example 1.2 (The pendulum). A pendulum of length L in gravity g obeys $\ddot{\theta} = -(g/L)\sin \theta$. With $\mathbf{x} = (x_1, x_2) = (\theta, \dot{\theta})$ it becomes a first-order system

$$\dot{x}_1 = x_2, \quad \dot{x}_2 = -\frac{g}{L} \sin x_1,$$

so $n = 2$ and $\beta = (g, L)$ (and the mass, which drops out here). If the pivot is mounted on a cart that we can drive, the cart's acceleration is a control u [► V1.2 @ 4:34]. If we film the pendulum and extract only the horizontal position of the bob, the measurement is $y = L \sin x_1$: one number, from which the state must be reconstructed.

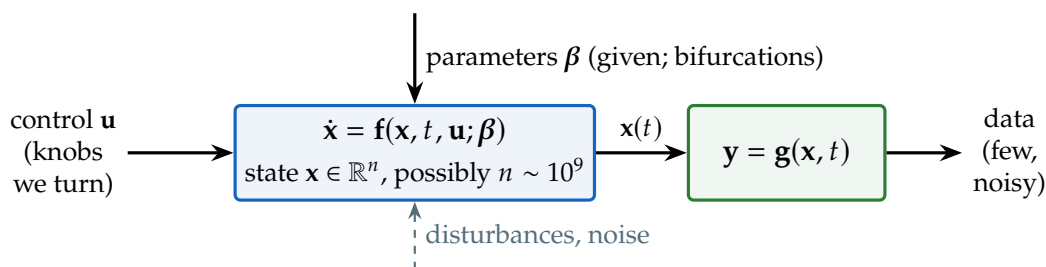


Figure 1.2: The anatomy of a dynamical system as a block diagram, the way control engineers draw it (Chapter 8 of the book). The machine-learning tasks of the course attack one block at a time: learn f , reconstruct x from y , choose u , study the dependence on β .

◊ **BEYOND THE CORE: Precise setting: manifolds, existence and uniqueness**

In the book the autonomous system $\dot{x} = f(x)$ is formulated more generally: the state lives on a smooth n -dimensional manifold M and f is a vector field, a section of the tangent bundle TM , so that $f(x) \in T_x M$. Example: the pendulum angle is defined modulo 2π , so the true state space is the cylinder $S^1 \times \mathbb{R}$, and [fig. 1.1](#) should be rolled up so that $\theta = -\pi$ and $\theta = \pi$ coincide. In practice one takes $M = \mathbb{R}^n$ and asks f to be *Lipschitz*, which guarantees existence and uniqueness of solutions.

Theorem 1.3 (Picard–Lindelöf). *Let $f : \mathbb{R} \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ be continuous in t and Lipschitz in x on a neighbourhood of (t_0, x_0) : $\|f(t, x) - f(t, x')\| \leq L\|x - x'\|$. Then the problem $\dot{x} = f(t, x)$, $x(t_0) = x_0$ has a unique solution on some interval $[t_0 - \epsilon, t_0 + \epsilon]$.*

Uniqueness is what makes “state” a meaningful word: two trajectories cannot cross, so the present state determines the future. It fails for $\dot{x} = \sqrt{|x|}$ at $x = 0$ (not Lipschitz): both $x(t) = 0$ and $x(t) = t^2/4$ start from $x(0) = 0$.

Exercise 1.1 (Non-autonomous to autonomous). Show that $\dot{x} = f(x, t)$, $x \in \mathbb{R}^n$, can be written as an autonomous system in $\mathbb{R}^{n+1}$. Apply this to the forced oscillator $\ddot{x} + \gamma\dot{x} + \omega_0^2 x = F \cos \Omega t$, and then show that, because the forcing is periodic, the extra coordinate can be taken on a circle S^1 instead of $\mathbb{R}$. What is the dimension of the state space of the forced pendulum?

Exercise 1.2 (Eigenvalues of the damped oscillator). Write $\ddot{x} + 2\zeta\omega_0\dot{x} + \omega_0^2 x = 0$ as $\dot{x} = Ax$.

- Compute the eigenvalues of A and classify the motion for $0 < \zeta < 1$, $\zeta = 1$, $\zeta > 1$.
- For $\zeta = 1$ the matrix is not diagonalizable: compute e^{At} anyway (Jordan form) and find the $te^{-\omega_0 t}$ term.
- Sample the underdamped solution with step Δt : give the eigenvalues $\mu_{\pm}$ of the discrete-time map and show that $|\mu_{\pm}| < 1$. Where do they lie in the complex plane as Δt grows?

1.2 From first principles to data

The classical way. Dynamical systems are not new: Newton’s second law $F = ma$ is already a dynamical system, and it gives the trajectories of projectiles, the motion of a pendulum and of the planets [[V1.1 @ 4:50](#)]. For centuries the method has been *physics based*: write down Newton’s law, the Lagrangian or the Hamiltonian, and derive the equations of

motion from first principles [► V1.1 @ 5:11][► V1.2 @ 6:17]. With these models we built satellites and went to the Moon [► V1.1 @ 6:53].

Why it is no longer enough. For the systems we most want to understand today, such as the brain, the climate, a financial market or the spread of a disease, there are no first-principles equations that are easy to write, simulate and control [► V1.1 @ 5:52]. “There is no master equation for your brain”, and no agreed governing law for how a disease spreads or how the climate works [► V1.1 @ 6:13][► V1.2 @ 6:57]. But there are more and more data: recordings of brain activity, for example, come in ever larger volumes [► V1.1 @ 6:13]. The next generation of problems, like controlling turbulence or understanding the brain, will not always be solvable with first-principles models, so the models will have to be derived from data [► V1.1 @ 7:14]. The field is in its infancy: knowing the physics still lets you do much more than data alone [► V1.1 @ 7:55].

Remark 1.4 (All models are models). Even $F = ma$, the Navier–Stokes equations of fluids and the Schrödinger equation are not “the truth” but *models* of reality, excellent ones [► V1.1 @ 6:33]. Seen this way, learning a model from data is not a different activity from physics but the same one, done with other tools. The book adds that, as systems become more complex, choosing the “correct” model becomes more subjective, which is a reason to want automatic, reproducible model discovery.

Reduced-order models. Data-driven models are useful even where the equations are known. The Navier–Stokes equations describe a fluid exactly but may be far too expensive to use; what one wants is a *reduced-order model*, a dynamical system for a much smaller state that captures only what we care about in the flow [► V1.2 @ 7:18].

Definition 1.5 (New concepts: data-driven and reduced-order models). Two ways of obtaining a model, and one kind of model:

- (a) A **first-principles** model is derived from accepted physical laws (Newton, Lagrange, Hamilton, conservation laws); a **data-driven** model is obtained by fitting (regression, optimization) to measurements of the system.
- (b) A **reduced-order model** (ROM) is a dynamical system $\dot{\mathbf{a}} = \tilde{\mathbf{f}}(\mathbf{a})$ for a state $\mathbf{a} \in \mathbb{R}^r$ with $r \ll n$, built so that a known map $\mathbf{a} \mapsto \mathbf{x}$ reproduces the quantities of interest of the full system $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x})$.

HISTORY: Newton, Poincaré and the birth of dynamical systems

The book dates *modern* dynamical systems to Henri Poincaré’s work on the motion of planets. In 1889 Poincaré won the prize offered by King Oscar II of Sweden for a memoir on the stability of the solar system (the three-body problem). While the memoir was being printed, queries from the editor Lars Phragmén led Poincaré to a serious error; correcting it, he discovered that the stable and unstable manifolds of a periodic orbit can intersect transversally (homoclinic tangle), the first glimpse of chaos. The printed copies were recalled and destroyed, and Poincaré paid for the reprinting, more than the 2500 crowns of the prize. The corrected memoir appeared in *Acta Mathematica* in 1890 and grew into *Les méthodes nouvelles de la mécanique céleste* (1892–1899), where the qualitative, geometric study of ODEs begins.

References: J. Barrow-Green, *Poincaré and the Three Body Problem* (AMS, 1997); Brunton & Kutz, Ch. 7 introduction.

1.3 Challenges of modern dynamical systems

Brunton’s list is explicitly personal: another dynamicist would give a different one, and this is his group’s “to-do list” [► V1.1 @ 12:03]. The two primary challenges, those that data can help with most, are [► V1.1 @ 9:38]: nonlinearity and unknown dynamics.

1. Nonlinearity. If the system is linear, $\dot{\mathbf{x}} = A\mathbf{x}$, everything is known from the eigenvalues and eigenvectors of A [► V1.1 @ 8:16]. Even a small nonlinearity, an x^2 , an x^3 or a $\cos x$ on the right-hand side, makes it very hard to understand, predict, estimate and control the future [► V1.1 @ 8:57]. Superposition fails: the sum of two solutions is no longer a solution [► V1.2 @ 7:38]. For linear systems “you could just write down the solution and go home, and we would all be out of business”: nonlinearity is “job security” [► V1.2 @ 7:59]. There are

special classes of nonlinear systems and special solutions, but no general, one-size-fits-all theory [► V1.2 @ 8:19]. Why linear systems count as “solved” is worth seeing in detail.

Proposition 1.6 (Linear systems are solved by their eigenvectors). Let $\dot{\mathbf{x}} = A\mathbf{x}$ with $A \in \mathbb{R}^{n \times n}$ diagonalizable, $AT = T\Lambda$, where $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$ and the columns ξ_j of T are eigenvectors. Then

- (i) the solution is $\mathbf{x}(t) = e^{At}\mathbf{x}(0) = Te^{\Lambda t}T^{-1}\mathbf{x}(0) = \sum_{j=1}^n b_j e^{\lambda_j t} \xi_j$, with $\mathbf{b} = T^{-1}\mathbf{x}(0)$;
- (ii) in the eigenvector coordinates $\mathbf{z} = T^{-1}\mathbf{x}$ the dynamics decouple: $\dot{z}_j = \lambda_j z_j$;
- (iii) sampled at times $t_k = k\Delta t$, the system is the linear map $\mathbf{x}_{k+1} = e^{A\Delta t}\mathbf{x}_k$, whose eigenvalues are $\mu_j = e^{\lambda_j \Delta t}$.

Proof. Substitute $\mathbf{x} = T\mathbf{z}$: $T\dot{\mathbf{z}} = AT\mathbf{z} = T\Lambda\mathbf{z}$, and multiplying by T^{-1} gives $\dot{\mathbf{z}} = \Lambda\mathbf{z}$, i.e. (ii). Each scalar equation has solution $z_j(t) = e^{\lambda_j t} z_j(0)$, so $\mathbf{z}(t) = e^{\Lambda t}\mathbf{z}(0)$ and $\mathbf{x}(t) = Te^{\Lambda t}T^{-1}\mathbf{x}(0)$; writing $Te^{\Lambda t}\mathbf{b}$ column by column gives the sum in (i). The power series of the exponential gives $e^{\Lambda t} = \sum_k (T\Lambda T^{-1})^k t^k / k! = T(\sum_k \Lambda^k t^k / k!)T^{-1} = Te^{\Lambda t}T^{-1}$. For (iii), $\mathbf{x}_{k+1} = e^{A\Delta t}\mathbf{x}_k$ follows from (i) with initial time t_k , and $e^{A\Delta t}T = Te^{\Lambda\Delta t}$ shows that the eigenvectors are the same, with eigenvalues $e^{\lambda_j \Delta t}$. □

INTUITION: Normal modes

Proposition 1.6 is the normal-mode analysis of small oscillations in classical mechanics. Each eigenvector ξ_j is a spatial pattern that evolves on its own, growing or decaying at rate $\text{Re } \lambda_j$ and oscillating at frequency $\text{Im } \lambda_j$; any motion is a superposition of them. The whole course can be read as the search for such modes when $\mathbf{f}$ is nonlinear or unknown: DMD (Chapter 4) fits them to data, and Koopman theory (Chapter 6) looks for coordinates in which a nonlinear system has them.

2. Unknown dynamics. Perhaps more fundamental: often $\mathbf{f}$ is simply not known [► V1.1 @ 9:18]. This challenge “goes unmentioned in a lot of cases” [► V1.2 @ 6:17]. For a pendulum you open a physics book and derive the equations; for the brain or a disease network we have models and approximations, but no agreed governing law, and $\mathbf{f}$ must be discovered from data [► V1.2 @ 6:37][► V1.2 @ 6:57]. Even when $\mathbf{f}$ is known it may be unusable in practice, as with the Navier–Stokes equations, and one needs a reduced model [► V1.2 @ 7:18].

Other challenges. Important, but secondary to the first two [► V1.1 @ 9:58]:

- **High dimension.** The state may have millions or billions of components. To simulate the climate one chops the Earth into maybe a million to a billion grid cells, at a resolution of a kilometre or less [► V1.1 @ 10:20]; a brain has about a hundred billion neurons, so a hundred-billion-dimensional state just to say which neurons are active [► V1.2 @ 8:40]; a realistic epidemic model needs the network of contacts of billions of people [► V1.2 @ 9:00].
- **Multiscale dynamics.** Coherent structures at many scales of space and time [► V1.1 @ 11:21]. In a turbulent fluid you can zoom in, and in, and still find structure [► V1.1 @ 11:42]. For the weather, Earth-scale seasonal patterns and hurricanes matter, but so do tiny features, for a forecast three weeks out [► V1.2 @ 9:41]. The brain is multiscale in space (regions made of neurons) and in time: you hear words over seconds, connect them to what was said minutes ago, against a lifetime of experience [► V1.2 @ 10:02].
- **Chaos and transients.** Not all nonlinear systems are chaotic, but many are: the future depends sensitively on small changes of the initial condition, of the parameters and of the dynamics [► V1.2 @ 10:22]. This is why the weather cannot be predicted three weeks ahead: the system is chaotic and we can measure it, and know its equations and parameters, only so accurately [► V1.2 @ 11:02].
- **Latent (hidden) variables.** With a big state, we are surely not measuring everything;

a few neurons, or a proxy. The unmeasured variables may matter a lot [► V1.2 @ 11:23].

- **Noise and stochasticity.** Real systems are forced by disturbances and measured with noise [► V1.1 @ 11:01][► V1.2 @ 11:45]. And the noise is not the Gaussian white noise of textbooks (“the only thing we can actually solve”) but correlated, structured and biased [► V1.2 @ 11:45].
- **Uncertainty** in everything: the initial condition, the parameters, the model f itself, what is measured and how, even *when* a measurement was taken; it must be handled and propagated [► V1.1 @ 12:03][► V1.2 @ 12:05].

The systems of interest, such as neuroscience, turbulence, financial markets and disease, have all of these at once: high-dimensional, multiscale, nonlinear, with unknown f [► V1.1 @ 12:24].

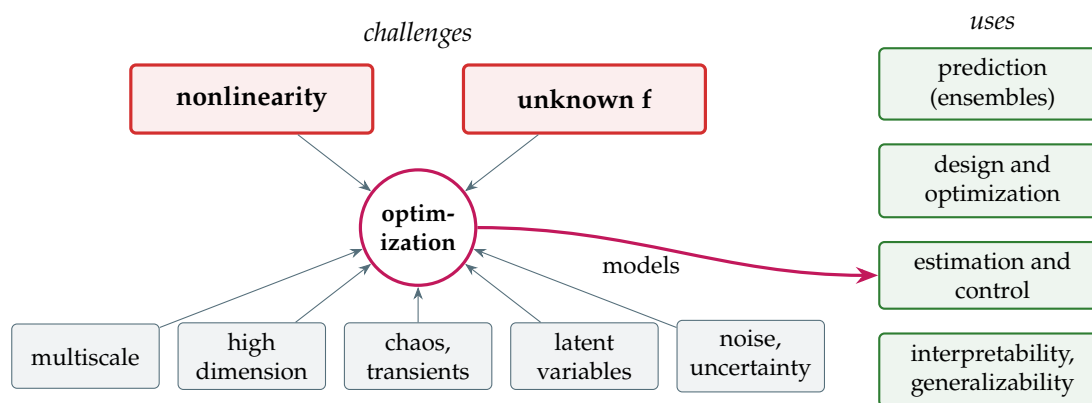


Figure 1.3: Map of the chapter. The two primary challenges (red) and the secondary ones (grey) can all be recast as optimization problems, which is where machine learning enters [► V1.2 @ 12:47]; the resulting models serve the four uses (green).

Challenges as optimization problems. Most of these challenges can be recast as *optimization* problems [► V1.2 @ 12:47]: handle noise and disturbances optimally in a prediction or estimate, propagate uncertainty optimally, reconstruct the full state optimally from the limited measurements y . And machine learning is, at bottom, “optimization based on data”: building models by optimization over a wealth of data [► V1.2 @ 13:07]. So the anatomy of [definition 1.1](#) lets one pick a single task and attack it with these tools [► V1.2 @ 13:28]. Every challenge on the list is not one problem but whole careers of work [► V1.2 @ 14:29].

Exercise 1.3 (How big is the weather?). Estimate the dimension n of the state of a global weather model with a horizontal resolution of 1 km, 100 vertical levels and 6 variables per cell (three velocity components, temperature, pressure, humidity); the Earth’s surface is about $5.1 \times 10^8 \text{ km}^2$. How many bytes does one snapshot take in double precision? How many snapshots of this size fit in a petabyte?

Exercise 1.4 (Superposition). Show that the set of solutions of $\dot{x} = Ax$ is a vector space. Then show, with an explicit counterexample, that for $\dot{x} = -x + x^2$ the sum of two solutions is not a solution. Solve this equation exactly (separation of variables or the substitution $\varphi = 1/x$) and discuss which initial conditions blow up in finite time.

Example 1.7 (State reconstruction as optimization). Given measurements $y = Cx + \eta$ with $C \in \mathbb{R}^{p \times n}$, $p \ll n$, the state cannot be obtained by inverting C . One instead solves an optimization problem: least squares with a prior (the Kalman filter, Chapter 8 of the book), or least squares in a basis where x is sparse (compressed

1.4 What models are for

Again a personal, not definitive, list [► V1.1 @ 13:06].

1. Future state prediction. The most obvious use [► V1.1 @ 13:06][► V1.2 @ 13:48]: forecast the weather a week ahead, or, if someone is lost at sea, integrate from a rough initial position where they will be two hours later. Prediction need not be of a single trajectory: it can be statistical, an *ensemble* of initial conditions propagated together to give a probability density of the future state [► V1.1 @ 13:26][► V1.2 @ 14:09]. For the climate we care not about one trajectory but about the probability of evolving towards higher or lower temperatures [► V1.1 @ 13:47]; for the Gulf oil spill, the source is known only up to a Gaussian, which must be propagated into a forecast of where the oil will be [► V1.2 @ 14:09].

2. Design and optimization. With a model, tune the parameters β to get a desired performance [► V1.1 @ 14:09][► V1.2 @ 14:51]. Formula One cars, America's Cup yachts and rocket engines are complex multiphysics systems dominated by turbulent fluid dynamics; the dynamics change a lot with rain, humidity or the shape of an aerodynamic scoop, and so does the drag. Much effort goes into reduced-order models of these systems, cheap enough to optimize the design rapidly [► V1.1 @ 14:29][► V1.1 @ 14:51]. This requires understanding how the system depends on its parameters [► V1.2 @ 15:11].

3. Control. Different from design: here we measure the system in real time and, with a model, compute an actuation signal that modifies its behaviour through sensor-based *feedback* [► V1.1 @ 15:11]. If design optimization is done in real time from measurements, the parameters become actuators and design becomes feedback control [► V1.2 @ 15:11]. Examples: suppressing epileptic seizures, suppressing a disease as it spreads through a continent, increasing the power extracted by a turbine from a flow [► V1.1 @ 15:32].

4. Interpretability and physical intuition. Often missing from such lists, but for Brunton maybe the most important [► V1.1 @ 15:52]. Many machine-learning methods are hard to interpret and give no intuition; Brunton argues for tailoring them so that the models they produce are interpretable and teach us more physics than we knew [► V1.1 @ 16:15]. Besides manipulating and predicting the world, we want to understand how and why it works [► V1.2 @ 15:53]. Two key words [► V1.2 @ 16:13]:

- **interpretable:** the model is simple enough that we can communicate and reason about it;
- **generalizable:** the model works in situations it was not built for, for instance at parameter values never tested [► V1.2 @ 16:33].

Newton's second law is the ultimate interpretable and generalizable model: so simple that anyone can understand it, valid in a vast range of situations, and it changed how we understand and interact with the world [► V1.2 @ 16:33][► V1.2 @ 16:54].

INTUITION: Interpretable = few terms

For a physicist, "interpretable" has a concrete meaning that will come back in SINDy (Chapter 5): a model with few terms, each one recognizable (a damping, a quadratic coupling, a restoring force). A neural network with 10^6 weights may predict well, but it does not tell you that $\mathbf{f}$ contains $-\sin \theta$. Generalization is the reward for parsimony: a

law with few terms extrapolates, a curve fitted with many terms does not (overfitting).

1.5 Techniques and the plan of the course

Methods. An incomplete list of the techniques that work [► V1.1 @ 16:57]:

- **regression**, linear and *sparse*, probably the foremost one [► V1.1 @ 17:17];
- **neural networks** and deep learning, very powerful at learning dynamics, but with interpretability problems; the course shows how to constrain them so that they are interpretable by construction [► V1.1 @ 17:39];
- **genetic algorithms** and **genetic programming**, used effectively to discover dynamical systems from data [► V1.1 @ 17:39].

Definition 1.8 (New concepts: *sparse regression*, *genetic programming*). Two ways of finding a model with few terms:

- (a) **Sparse regression** fits $y \approx \Theta \xi$ while asking that most entries of the coefficient vector ξ vanish (by an ℓ_1 penalty or by thresholding), so that only a few columns of Θ , the few “terms” of the model, survive. *Sparse*, from the Latin *sparsus*, “scattered”: few non-zero entries scattered in a sea of zeros.
- (b) **Genetic programming** searches over formulas (expression trees) by mimicking evolution: a population of candidate equations is mutated and recombined, and the ones that fit the data best with the fewest terms survive.

Coping with nonlinearity: Koopman. A preview of a large part of the course [► V1.1 @ 18:21]. Koopman analysis looks for a nonlinear change of coordinates $\varphi(\mathbf{x})$ such that, in the new coordinates, the dynamics is linear,

$$\frac{d}{dt} \varphi(\mathbf{x}) = K \varphi(\mathbf{x}),$$

so that the whole linear toolbox of [proposition 1.6](#) applies. Finding coordinates that make a system simpler is an old goal of physics; what is new is using deep learning or sparse regression to find them from data [► V1.1 @ 18:41][► V1.1 @ 19:02]. The other half of the course is to discover $\mathbf{f}$ itself by regression [► V1.1 @ 19:02]. The methods to come: dynamic mode decomposition, Koopman analysis, delay coordinates, neural networks [► V1.1 @ 19:23].

Put the physics in. The final message [► V1.1 @ 19:43]. Most of machine learning was developed for *static* tasks, like image classification or scene labelling. Dynamical systems instead have physics: constraints, conservation of energy and mass, $F = ma$ [► V1.1 @ 20:03]. Even where the exact equations are unknown, as for a reduced model of a turbulent fluid, the brain or an epidemic, there are surely constraints, symmetries and invariants [► V1.1 @ 20:03][► V1.1 @ 20:25]. Building them into the learning procedure pays twice: the models are more interpretable, and they can be learned with less data [► V1.1 @ 20:45] [► V1.1 @ 21:06].

ASIDE: Symbolic regression by evolution

The best-known example is the work of M. Schmidt and H. Lipson, *Distilling free-form natural laws from experimental data*, Science 324 (2009), who recovered Hamiltonians and Lagrangians of double pendulums and oscillators from motion-tracking data with genetic programming (ref. [477] of the book’s Chapter 7, together with Bongard and Lipson 2007, ref. [68]).

Example 1.9 (A nonlinear system made linear). $\dot{x} = x^2$ is nonlinear. With $\varphi(x) = 1/x$ (for $x \neq 0$), $\dot{\varphi} = -\dot{x}/x^2 = -1$: in the coordinate φ the motion is uniform, $\varphi(t) = \varphi(0) - t$, so $x(t) = x_0/(1 - x_0 t)$, with the blow-up at $t = 1/x_0$ in

INTUITION: Symmetries as priors

Enforcing a conservation law shrinks the space of candidate models. If energy is conserved, $\mathbf{f}$ must satisfy $\nabla E \cdot \mathbf{f} = 0$: a linear constraint on the coefficients of a regression, which removes whole directions of the search space. Fewer free parameters, less data needed: this is the statistical face of Noether's theorem.

plain view. Chapter 7 shows that such finite linear representations exist only in special cases.

Homework from the lecture. Brunton closes with an invitation [► V1.2 @ 17:15]: pick a system you care about (the brain, an epidemic, the climate) and ask what its state vector is, what its controls and parameters are, what rules might govern its evolution, what can be measured, and where it sits among the challenges and the uses [► V1.2 @ 17:35]. The exercise at the end of the chapter does exactly this.

Exercise 1.5 (Energy constraint as a linear constraint). Suppose you look for a model of a one-degree-of-freedom conservative system, $\dot{q} = p$, $\dot{p} = \sum_{k=0}^3 \xi_k q^k$, as a linear combination of monomials. Show that energy conservation is automatic for any ξ , and write the energy. Now allow also $\dot{p}$ to contain a term $\xi_4 p$: which value of ξ_4 does energy conservation impose? Relate this to the last intuition box of [section 1.5](#).

KEY FORMULAS

- Anatomy: $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, t, \mathbf{u}; \boldsymbol{\beta})$, $\mathbf{y} = \mathbf{g}(\mathbf{x}, t)$; state, dynamics (vector field), time, control, parameters, measurements.
- Linear systems: $\mathbf{x}(t) = T e^{\Lambda t} T^{-1} \mathbf{x}(0) = \sum_j b_j e^{\lambda_j t} \xi_j$; decoupled in $\mathbf{z} = T^{-1} \mathbf{x}$; discrete-time eigenvalues $\mu_j = e^{\lambda_j \Delta t}$.
- Primary challenges: nonlinearity, unknown $\mathbf{f}$. Secondary: dimension, multiscale, chaos, latent variables, noise, uncertainty. All recast as optimization.
- Uses: prediction (also of ensembles), design, control, interpretability and generalizability.
- Koopman program: find $\boldsymbol{\varphi}(\mathbf{x})$ with $\frac{d}{dt} \boldsymbol{\varphi} = K \boldsymbol{\varphi}$.

Exercises

Standalone exercises follow the section they practise; connected or integrative ones, like this one, are at the end.

Exercise 1.6 (Anatomy of a system of your choice). Choose a system: (a) an SIR epidemic in a city; (b) a wind turbine in a turbulent wind; (c) the temperature of a room heated by a thermostat. For each one:

- define a state $\mathbf{x}$ and estimate its dimension;
- list the controls $\mathbf{u}$ and the parameters $\boldsymbol{\beta}$, and say which parameters could become controls;
- say what can realistically be measured, $\mathbf{y} = \mathbf{g}(\mathbf{x}, t)$;
- say which challenges of [section 1.3](#) apply, and which use of [section 1.4](#) you would care most about.

Chapter 2

Fundamentals: Fixed Points, Linearization, Invariant Manifolds, Bifurcations, Chaos

SOURCES FOR THIS CHAPTER

Video	Length	Content
V2.1 Topics in Dynamical Systems: Fixed Points, Linearization, Invariant Manifolds, Bifurcations & Chaos	32:11	Linear, mildly nonlinear and chaotic systems; fixed points and linearization; eigenvector coordinates; the Duffing oscillator; stable and unstable manifolds; pitchfork and Hopf bifurcations; discrete-time maps and the logistic map; the flow map; particles in flows and FTLE; chaos as stretching and folding.

Transcript materials/transcripts/C02-V01.txt. **Book** Sec. 7.1 (“Dynamical systems”, “Discrete-time systems”, “Linear dynamics and spectral decomposition”), Sec. 8.2 (linear systems). This is the *last* video of the playlist (recorded about three years after most of the others): it is placed here because it is the background for everything that follows.

Motivation

A “mile-high” tour of what one learns in a course on dynamical systems [[► V2.1 @ 0:20](#)]. Every vignette (fixed points, linearization, manifolds, bifurcations, maps, chaos) would be a couple of lectures in a real course [[► V2.1 @ 31:11](#)]. The reason it belongs in a course on machine learning is stated at the very end: when a model of a dynamical system is learned from data, these are the properties one checks it against. Are the fixed points in the right places? Do the local linearizations have the right stability? Does the model mix statistically like the data? [[► V2.1 @ 31:32](#)].

For a physics MSc most of this is a refresher of classical mechanics and of the theory of ODEs; the chapter is kept compact, with the derivations that are reused later (linearization, eigenvector coordinates, flow map) written out in full.

2.1 From linear to chaotic

A dynamical system is any system that evolves in time according to some rules [[► V2.1 @ 0:41](#)]: an apple falling under $F = ma$, a fluid obeying the Navier–Stokes equations, a pendulum, a car, but also the brain, whose neurons fire according to rules that certainly exist even if we cannot write them all down [[► V2.1 @ 1:02](#)][[► V2.1 @ 1:22](#)]. Brunton orders systems by

increasing nonlinearity [► V2.1 @ 1:42]:

1. **linear** systems: the pendulum at small angles, a network of springs, masses and dampers; they oscillate, and superposition holds [► V2.1 @ 2:03][► V2.1 @ 2:23];
2. **mildly nonlinear** systems, such as Hamiltonian or Lagrangian dynamics: the Duffing oscillator, a bead sliding on a wire shaped like a double well, with several fixed points [► V2.1 @ 3:05][► V2.1 @ 3:26];
3. **chaotic** systems: still deterministic, but with limited forecast ability, because an imperfectly known initial state or dynamics makes the prediction diverge from the truth after some time (double pendulum, Lorenz attractor, weather, turbulence) [► V2.1 @ 3:46][► V2.1 @ 4:06].

Definition 2.1 (New concepts: linearity, superposition). The system $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x})$ is **linear** if $\mathbf{f}(\mathbf{x}) = A\mathbf{x}$ for a matrix A . Then **superposition** holds: if $\mathbf{x}_1(t)$ and $\mathbf{x}_2(t)$ are solutions, so is $a\mathbf{x}_1(t) + b\mathbf{x}_2(t)$; adding two initial conditions adds the two solutions [► V2.1 @ 2:44]. Superposition holds *only* for linear systems.

The state is again the minimal description, $(\theta, \dot{\theta})$ for the pendulum, and $\mathbf{f}$ comes from the Euler–Lagrange equations, $F = ma$ or Hamilton’s equations [► V2.1 @ 4:48][► V2.1 @ 5:09]([definition 1.1](#)).

2.2 Fixed points and linearization

The pendulum behaves linearly when kicked a little near the bottom, nonlinearly when released from near $\theta = \pi$ [► V2.1 @ 5:29][► V2.1 @ 5:50]. The standard strategy for a nonlinear system is: find the fixed points, linearize around them, and do linear analysis there [► V2.1 @ 9:35].

Definition 2.2 (New concepts: fixed point, Jacobian, linearization). (a) A **fixed point** (equilibrium) is a state $\bar{\mathbf{x}}$ with $\mathbf{f}(\bar{\mathbf{x}}) = 0$: started there, the system stays there [► V2.1 @ 6:10].

(b) The **Jacobian** of $\mathbf{f}$ at $\bar{\mathbf{x}}$ is the matrix of partial derivatives $A = D\mathbf{f}(\bar{\mathbf{x}})$, $A_{ij} = \partial f_i / \partial x_j (\bar{\mathbf{x}})$ [► V2.1 @ 8:54].

(c) The **linearization** at $\bar{\mathbf{x}}$ is the linear system $\frac{d}{dt}\Delta\mathbf{x} = A\Delta\mathbf{x}$ for the displacement $\Delta\mathbf{x} = \mathbf{x} - \bar{\mathbf{x}}$.

The name *Jacobian* honours Carl Gustav Jacob Jacobi (1804–1851), who studied these determinants of partial derivatives in 1841.

The pendulum has two fixed points: down, $(\theta, \dot{\theta}) = (0, 0)$, and up, $(\pi, 0)$; the second is unstable, but perfectly balanced it stays put [► V2.1 @ 6:10][► V2.1 @ 6:31]([fig. 1.1](#)).

Derivation. Zoom into a neighbourhood of $\bar{\mathbf{x}}$ and write $\mathbf{x} = \bar{\mathbf{x}} + \Delta\mathbf{x}$ with $\Delta\mathbf{x}$ small [► V2.1 @ 7:12]. Since $\bar{\mathbf{x}}$ is constant,

$$\frac{d}{dt}\Delta\mathbf{x} = \dot{\mathbf{x}} = \mathbf{f}(\bar{\mathbf{x}} + \Delta\mathbf{x}) = \underbrace{\mathbf{f}(\bar{\mathbf{x}})}_{=0} + D\mathbf{f}(\bar{\mathbf{x}})\Delta\mathbf{x} + O(\|\Delta\mathbf{x}\|^2) \quad (2.1)$$

by Taylor expansion [► V2.1 @ 7:32][► V2.1 @ 7:52]. Close to the fixed point the higher-order terms are negligible, and what remains is an honest linear system $\Delta\dot{\mathbf{x}} = A\Delta\mathbf{x}$ [► V2.1 @ 8:13]

[► V2.1 @ 9:15]. Its eigenvalues say whether the fixed point is stable; its eigenvectors say along which directions it is unstable [► V2.1 @ 9:15].

Theorem 2.3 (Hartman–Grobman). *If no eigenvalue of $A = D\mathbf{f}(\bar{\mathbf{x}})$ has zero real part (hyperbolic fixed point), then in a neighbourhood of $\bar{\mathbf{x}}$ the flow of $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x})$ is topologically conjugate (by a homeomorphism) to the flow of $\Delta\dot{\mathbf{x}} = A\Delta\mathbf{x}$.*

Theorem 2.3 is what justifies the strategy, and its hypothesis is what limits it: for eigenvalues on the imaginary axis (centres, as in the Duffing wells below) the linearization alone decides nothing.

Eigenvector coordinates. As in proposition 1.6, the solution is $\mathbf{x}(t) = e^{At}\mathbf{x}_0$, which advances any initial condition any time forward or backward, and is easy to compute [► V2.1 @ 9:55][► V2.1 @ 10:16]. With the eigenvectors T of A and $\mathbf{x} = T\mathbf{z}$: $\dot{\mathbf{z}} = T^{-1}\dot{\mathbf{x}} = T^{-1}A\mathbf{x} = T^{-1}AT\mathbf{z} = \Lambda\mathbf{z}$; it looks worse, but $T^{-1}AT = \Lambda$ is diagonal, so the dynamics decouple [► V2.1 @ 10:37][► V2.1 @ 10:57][► V2.1 @ 11:18].

INTUITION: Three steps: map in, advance, map out

Brunton’s favourite reading of $\mathbf{x}(t) = Te^{\Lambda t}T^{-1}\mathbf{x}_0$ [► V2.1 @ 11:59], read from right to left [► V2.1 @ 12:40]:

1. T^{-1} maps the initial condition into eigenvector coordinates, $\mathbf{z}_0 = T^{-1}\mathbf{x}_0$;
2. there the system is decoupled: each z_j is advanced independently, $z_j(t) = e^{\lambda_j t}z_j(0)$;
3. T maps back to physical space [► V2.1 @ 13:00][► V2.1 @ 13:41].

The same “lift, evolve linearly, project back” structure returns in DMD and Koopman theory, where the lift is nonlinear.

Exercise 2.1 (Pendulum fixed points). Linearize $\dot{\theta} = \omega$, $\dot{\omega} = -\sin \theta$ at $(0, 0)$ and $(\pi, 0)$, compute eigenvalues and eigenvectors, and compare with fig. 1.1. Why does Hartman–Grobman say nothing about the stability of $(0, 0)$ by itself? What does energy conservation add?

2.3 The Duffing oscillator and invariant manifolds

CASE STUDY: Particle in a double well

The Duffing oscillator without friction [► V2.1 @ 14:43]:

$$\dot{x} = v, \quad \dot{v} = x - x^3. \quad (2.2)$$

It is Newton’s law $\ddot{x} = -V'(x)$ in the double-well potential $V(x) = x^4/4 - x^2/2$ (a frictionless roller coaster with this shape) [► V2.1 @ 17:08][► V2.1 @ 17:28]. In the video the potential is written loosely as “ $x^4 - x^2$ ”, which has the same shape.

- *Fixed points:* $v = 0$ and $x - x^3 = 0$, i.e. $x \in \{0, \pm 1\}$ [► V2.1 @ 15:04].
- *Phase portrait:* a particle released at rest on a wall accelerates to maximal speed at the bottom, decelerates to rest on the opposite wall, comes back: a closed orbit in the (x, v) plane, forever, without friction [► V2.1 @ 15:44][► V2.1 @ 16:27][► V2.1 @ 16:47].

- *Jacobian* [► V2.1 @ 17:49]: $A(\bar{x}) = \begin{pmatrix} 0 & 1 \\ 1 - 3\bar{x}^2 & 0 \end{pmatrix}$; friction would add a term in the lower right corner [► V2.1 @ 18:10].
- At $(\pm 1, 0)$: $A = \begin{pmatrix} 0 & 1 \\ -2 & 0 \end{pmatrix}$, eigenvalues $\pm i\sqrt{2}$: neutrally stable, oscillation forever (a *centre*) [► V2.1 @ 18:30].
- At $(0, 0)$: $A = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$, eigenvalues ± 1 : a *saddle*; a small push sends the car away [► V2.1 @ 18:51]. The eigenvectors $(1, \pm 1)/\sqrt{2}$ are the 45-degree lines of [fig. 2.1](#) [► V2.1 @ 19:11][► V2.1 @ 19:31].

PYTHON: fixed points and stability of the Duffing oscillator

```
import numpy as np

def f(x):
    # Duffing without friction: x' = v, v' = x - x^3
    return np.array([x[1], x[0] - x[0]**3])

def jacobian(x):
    return np.array([[0.0, 1.0],
                    [1.0 - 3.0 * x[0]**2, 0.0]])

for xbar in ([0.0, 0.0], [1.0, 0.0], [-1.0, 0.0]):
    xbar = np.array(xbar)
    assert np.allclose(f(xbar), 0)          # it is a fixed point
    lam, T = np.linalg.eig(jacobian(xbar))
    print(xbar, 'eigenvalues', np.round(lam, 3))
    if np.isreal(lam).all():
        print('    eigenvectors (columns)\n', np.round(T, 3))
# [0. 0.] eigenvalues [ 1.+0.j -1.+0.j] eigenvectors (0.707, 0.707), (-0.707, 0.707)
# [1. 0.] eigenvalues [0.+1.414j 0.-1.414j]
# [-1. 0.] eigenvalues [0.+1.414j 0.-1.414j]
```

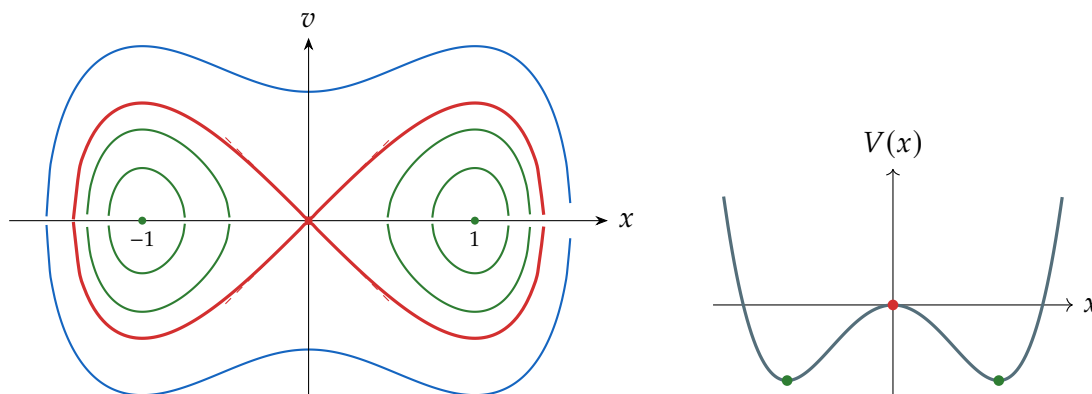


Figure 2.1: Left: phase portrait of the Duffing oscillator (2.2), level sets of the energy $E = \frac{1}{2}v^2 + V(x)$. Green: oscillations inside one well (around the centres $(\pm 1, 0)$); red: the two homoclinic orbits through the saddle, which are at the same time its stable and unstable manifolds; blue: an orbit above the barrier, visiting both wells. Dashed: the eigenvectors of the saddle, tangent to the manifolds. Right: the potential $V(x) = x^4/4 - x^2/2$.

Invariant manifolds. Continuing the eigenvector lines of the saddle beyond the linear neighbourhood sweeps out the *stable* and *unstable manifolds* [► V2.1 @ 19:52]. A second example has a single fixed point at the origin which is a saddle, stable along y and unstable along x : an initial condition first approaches along the stable direction, then leaves along the unstable one [► V2.1 @ 19:52][► V2.1 @ 20:12].

Definition 2.4 (New concepts: *stable, unstable, invariant manifolds*). For a fixed point $\bar{x}$:

- (a) the **stable manifold** is $W^s = \{x_0 : x(t) \rightarrow \bar{x} \text{ as } t \rightarrow +\infty\}$;
- (b) the **unstable manifold** is $W^u = \{x_0 : x(t) \rightarrow \bar{x} \text{ as } t \rightarrow -\infty\}$;
- (c) a set is **invariant** if trajectories starting in it stay in it for all times; W^s and W^u are invariant.

By the stable manifold theorem, at a hyperbolic fixed point W^s and W^u are smooth manifolds tangent at $\bar{x}$ to the stable and unstable eigenspaces of the linearization. *Manifold*, the English rendering of Riemann's *Mannigfaltigkeit*, “multiplicity”: a space that looks locally like $\mathbb{R}^k$.

INTUITION: Saddles amplify uncertainty

A ball of uncertainty advected near a saddle is squashed along the stable direction and stretched along the unstable one [► V2.1 @ 20:33][► V2.1 @ 20:54]. Repeat this with saddles whose manifolds fold back (as in a bounded attractor) and you get *stretching and folding*: one way to think about chaos.

Exercise 2.2 (Damped Duffing). Add friction: $\dot{v} = x - x^3 - \delta v$, $\delta > 0$.

- (a) Compute the Jacobian at the three fixed points and classify them as a function of δ (stable focus or node; saddle).
- (b) Show that $E = \frac{1}{2}v^2 + V(x)$ satisfies $\dot{E} = -\delta v^2 \leq 0$, and sketch the phase portrait. What are the basins of attraction of the two wells?

Example 2.5 (A saddle with a curved manifold). A standard system with one saddle (the book uses it for Koopman theory in Sec. 7.4) is $\dot{x} = \mu x$, $\dot{y} = \lambda(y - x^2)$ with $\lambda < 0 < \mu$. The y -axis $x = 0$ is invariant and is the stable manifold. The unstable manifold is the parabola $y = \frac{\lambda}{\lambda - 2\mu}x^2$: substituting $y = cx^2$ gives $\dot{y} = 2cx\dot{x} = 2c\mu x^2$, which equals $\lambda(c - 1)x^2$ for $c = \lambda/(\lambda - 2\mu)$. Its tangent at the origin is the x -axis, the unstable eigenvector.

2.4 Bifurcations

Systems often depend on a parameter μ : the inflow velocity or the size of the body in a wind tunnel, the infection rate of a disease like COVID-19 [► V2.1 @ 21:15][► V2.1 @ 21:35]. One wants to know how the dynamics changes with μ .

Definition 2.6 (New concepts: *bifurcation, bifurcation diagram*). A **bifurcation** is a qualitative change of the dynamics (number or stability of fixed points, appearance of periodic orbits, ...) at a parameter value μ_c : a tiny change of μ produces a big change of behaviour [► V2.1 @ 25:00]. A **bifurcation diagram** plots the fixed points (or attractors) against μ . From the Latin *bifurcus*, “two-pronged fork”.

Pitchfork. $\dot{x} = \mu x - x^3 = x(\mu - x^2)$ [► V2.1 @ 21:56]. For $\mu < 0$ everything contracts to $x = 0$. For $\mu > 0$ the origin is linearly unstable: x grows exponentially until the cubic term saturates it at $x^2 = \mu$ [► V2.1 @ 22:16][► V2.1 @ 22:57][► V2.1 @ 23:18]. The linearization $\partial_x(\mu x - x^3) = \mu - 3x^2$ gives μ at the origin and -2μ at $x = \pm\sqrt{\mu}$: two new stable branches [► V2.1 @ 23:38].

Hopf. The same mechanism in two dimensions [► V2.1 @ 23:58]: in polar coordinates

$$\dot{r} = \mu r - r^3, \quad \dot{\theta} = \omega, \quad (2.3)$$

i.e. $\dot{x} = \mu x - \omega y - x(x^2 + y^2)$, $\dot{y} = \omega x + \mu y - y(x^2 + y^2)$. For $\mu < 0$ the origin is a stable spiral; at $\mu = 0$ a pair of complex eigenvalues $\mu \pm i\omega$ crosses the imaginary axis, and for $\mu > 0$ trajectories settle on a stable *limit cycle* of radius $\sqrt{\mu}$, where the linear growth balances the cubic term [► V2.1 @ 24:19]. In polar form it is “directly analogous to the pitchfork” [► V2.1 @ 24:40]. Hopf bifurcations are everywhere in physical instabilities, for example the onset of vortex shedding behind a cylinder.

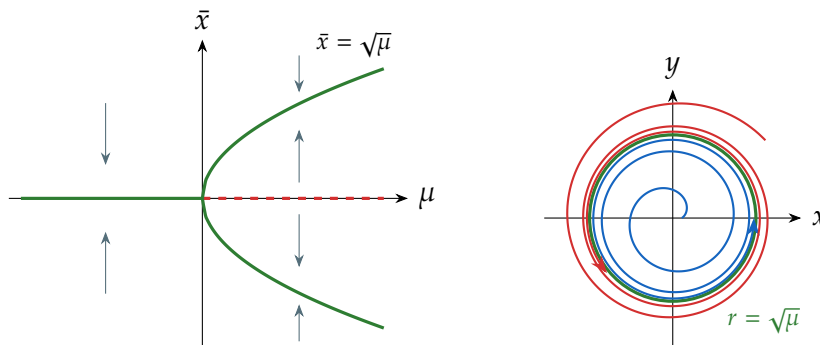


Figure 2.2: Left: pitchfork bifurcation of $\dot{x} = \mu x - x^3$ (solid: stable, dashed: unstable fixed points; arrows: the flow in x at fixed μ). Right: Hopf normal form (2.3) for $\mu > 0$: trajectories from inside and outside spiral onto the limit cycle.

Remark 2.7. This is a cartoon of what is a whole graduate course: the classification of bifurcations (saddle-node, transcritical, pitchfork, Hopf, ...) and normal forms [► V2.1 @ 24:40].

Exercise 2.3 (Hopf in Cartesian coordinates). Starting from (2.3), derive the equations for $x = r \cos \theta$, $y = r \sin \theta$, compute the Jacobian at the origin, and check that its eigenvalues are $\mu \pm i\omega$. Solve the radial equation exactly (hint: $u = 1/r^2$) and show that every $r(0) > 0$ tends to $\sqrt{\mu}$ for $\mu > 0$.

Exercise 2.4 (Subcritical pitchfork). Study $\dot{x} = \mu x + x^3 - x^5$. Draw the bifurcation diagram, find the saddle-node bifurcations, and show the hysteresis loop obtained by slowly increasing and then decreasing μ .

2.5 Discrete-time systems and the flow map

Maps. Dynamics can also be given in discrete time, $\mathbf{x}_{k+1} = \mathbf{F}(\mathbf{x}_k)$ [► V2.1 @ 25:21]. The *logistic map* $x_{k+1} = \mu x_k(1 - x_k)$ models a population, say of bunnies in year k : compute the right-hand side to get next year’s population, iterate year after year [► V2.1 @ 25:41][► V2.1 @ 26:02]. Here μ is a reproduction rate, and sweeping it gives very different behaviours: around $\mu = 3.45$ the population cycles reproducibly among a few values; for larger μ it wanders over a fractal set, the discrete-time analogue of chaos [► V2.1 @ 26:22][► V2.1 @ 26:43]. Chapter 3 simulates it.

The flow map. Continuous-time systems induce discrete-time ones, which is what a computer simulation does [► V2.1 @ 27:04][► V2.1 @ 27:24].

Definition 2.8 (New concepts: flow map). The **flow map** of $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x})$ over a time t is

$$\mathbf{F}_t(\mathbf{x}(t_0)) = \mathbf{x}(t_0 + t) = \mathbf{x}(t_0) + \int_{t_0}^{t_0+t} \mathbf{f}(\mathbf{x}(\tau)) d\tau. \quad (2.4)$$

The integration variable τ is there because the particle sees a different $\mathbf{f}$ as it moves [► V2.1 @ 27:45]. Sampling with step Δt gives the map $\mathbf{x}_{k+1} = \mathbf{F}_{\Delta t}(\mathbf{x}_k)$, a program that steps the system forward [► V2.1 @ 28:06]. The flow maps form a group: $\mathbf{F}_t \circ \mathbf{F}_s = \mathbf{F}_{t+s}$, $\mathbf{F}_0 = \text{id}$.

For a linear system, $\mathbf{F}_t = e^{At}$ (proposition 1.6). The book adds that maps are more general than flows: they include discontinuous and hybrid systems, and they are the natural setting for sampled experimental data and digital control.

Particles in flows. In fluid mechanics $\mathbf{f}$ is the velocity field and $\mathbf{x}$ the position of a tracer particle; integrating many particles shows how the fluid mixes and how a buoy or a pollutant spreads in the ocean [► V2.1 @ 28:26][► V2.1 @ 29:08]. The field then depends on time, and the *finite-time Lyapunov exponent* (FTLE) fields are the time-varying analogues of stable and unstable manifolds [► V2.1 @ 28:46].

◊ **BEYOND THE CORE: Finite-time Lyapunov exponents**

For the flow map $\mathbf{F}_{t_0}^{t_0+T}$ of a time-dependent field, the Cauchy–Green tensor is $C = (D\mathbf{F})^T D\mathbf{F}$ and the FTLE is $\sigma_T(\mathbf{x}_0) = \frac{1}{|T|} \ln \sqrt{\lambda_{\max}(C)}$, the exponential rate at which nearby particles separate. Ridges of σ_T computed forward in time ($T > 0$) mark repelling structures (analogues of stable manifolds), backward ($T < 0$) attracting ones (unstable manifolds): the Lagrangian coherent structures of G. Haller.

Exercise 2.5 (Flow map of a linear system). For $\dot{\mathbf{x}} = A\mathbf{x}$ with $A = \begin{pmatrix} -1 & 2 \\ -2 & -1 \end{pmatrix}$, compute $\mathbf{F}_t = e^{At}$ in closed form. Sample at Δt : what are the eigenvalues of $\mathbf{F}_{\Delta t}$? Check numerically (`scipy.linalg.expm`) and verify $\mathbf{F}_t \mathbf{F}_s = \mathbf{F}_{t+s}$.

2.6 Chaos

Integrate a tight cube of initial conditions through the Lorenz system: they stay together for a while, then stretch, fold and mix along the whole attractor [► V2.1 @ 29:28][► V2.1 @ 29:49][► V2.1 @ 30:09].

Definition 2.9 (New concepts: chaos, attractor, prediction horizon). A deterministic system is **chaotic** if it has sensitive dependence on initial conditions: nearby trajectories separate exponentially, at a rate given by the largest Lyapunov exponent $\lambda_1 > 0$, and mix over a bounded **attractor** (the set the trajectories approach). An initial uncertainty δ_0 grows like $\delta_0 e^{\lambda_1 t}$, so a tolerance Δ is reached after the **prediction horizon** $t_* \approx \lambda_1^{-1} \ln(\Delta/\delta_0)$; after that only statistical statements are possible [► V2.1 @ 30:30][► V2.1 @ 30:51]. From the Greek *cháōs*, the gaping void before creation.

The logarithm is the bad news: improving the initial measurement by a factor 10 only adds $\lambda_1^{-1} \ln 10$ to the horizon. Local weather (where a hurricane will go) and turbulence

are chaotic [► V2.1 @ 30:51][► V2.1 @ 31:11].

Exercise 2.6 (Prediction horizon). The largest Lyapunov exponent of the Lorenz system at standard parameters is $\lambda_1 \approx 0.906$. With an initial error of 10^{-6} and a tolerance of 1, estimate the horizon. How much does it grow if the error is reduced to 10^{-12} ?

KEY FORMULAS

- Fixed point $\mathbf{f}(\bar{\mathbf{x}}) = 0$; linearization $\Delta\dot{\mathbf{x}} = D\mathbf{f}(\bar{\mathbf{x}})\Delta\mathbf{x}$; valid at hyperbolic points (Hartman–Grobman).
- $\mathbf{x}(t) = Te^{\Lambda t}T^{-1}\mathbf{x}_0$: map in, advance decoupled modes, map out.
- Duffing $\dot{x} = v, \dot{v} = x - x^3$: centres $(\pm 1, 0)$ with $\pm i\sqrt{2}$, saddle $(0, 0)$ with ± 1 .
- Pitchfork $\dot{x} = \mu x - x^3$, Hopf $\dot{r} = \mu r - r^3, \dot{\theta} = \omega$: branches/cycle at $\sqrt{\mu}$.
- Flow map $\mathbf{F}_t(\mathbf{x}_0) = \mathbf{x}_0 + \int_0^t \mathbf{f}(\mathbf{x}(\tau)) d\tau$; prediction horizon $t_* \approx \lambda_1^{-1} \ln(\Delta/\delta_0)$.

HISTORY: Lorenz and the butterfly

Edward Lorenz (MIT) found sensitive dependence in 1961 when he restarted a weather simulation from a printout rounded to three decimals and the forecast diverged completely. His 1963 paper *Deterministic nonperiodic flow* (J. Atmos. Sci. 20, 130–141) reduced convection to the three equations now named after him. The “butterfly effect” comes from the title of a 1972 talk at the AAAS: *Predictability: Does the flap of a butterfly’s wings in Brazil set off a tornado in Texas?*

References: E. N. Lorenz, *The Essence of Chaos* (Univ. of Washington Press, 1993); J. Gleick, *Chaos* (1987).

Chapter 3

Simulating Continuous and Discrete Dynamics

SOURCES FOR THIS CHAPTER

Video	Length	Content
V3.1 Simulating the Lorenz System in Matlab	15:09	The Lorenz equations; coding the right-hand side; ode45 with tight tolerances; plotting the butterfly; a blob of initial conditions stretched and folded.
V3.2 Discrete-Time Dynamical Systems	9:46	From $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x})$ to $\mathbf{x}_{k+1} = \mathbf{F}(\mathbf{x}_k)$: sampling, the flow map; maps are more general; forward Euler as a crude flow map; preview of the logistic map.
V3.3 Simulating the Logistic Map in Matlab	16:29	Iterating $x_{k+1} = \beta x_k(1 - x_k)$ for $\beta \in [0, 4]$; removing transients; the bifurcation diagram (“mathematical mountains”); the logistic model of a population.

Transcripts `materials/transcripts/C03-V0*.txt`. **Book** Sec. 7.1 (Lorenz code, logistic map, Fig. 7.1–7.2). **Code** `code/ch03_lorenz.py`, `ch03_euler.py`, `ch03_logistic.py` (the Matlab of the videos rewritten in Python; run from the repository root, they regenerate the figures).

Motivation

Three practical videos. The point is not the code itself but what it will be used for: simulated trajectories of systems like Lorenz are the *data* on which every method of the course (DMD, SINDy, Koopman, HAVOK) is tested [► V3.1 @ 14:36]. On the way, the second video makes precise the relation between continuous and discrete time, which matters because data always come sampled.

3.1 Simulating the Lorenz system

The Lorenz system (1963) is one of the earliest models of chaos: a three-dimensional caricature of turbulent atmospheric convection [► V3.1 @ 0:43]:

$$\dot{x} = \sigma(y - x), \quad \dot{y} = x(\rho - z) - y, \quad \dot{z} = xy - \beta z, \quad (3.1)$$

with state $\mathbf{x} = (x, y, z)^T$ and parameter vector $\boldsymbol{\beta} = (\sigma, \rho, \beta)^T$ [► V3.1 @ 1:03][► V3.1 @ 1:45]. The classic chaotic values used by Lorenz are $\sigma = 10$, $\rho = 28$, $\beta = 8/3$ [► V3.1 @ 6:58]. A trajectory oscillates around one lobe, jumps to the other, oscillates there, and so on [► V3.1 @ 2:27].

♦ **BEYOND THE CORE: What the Lorenz variables are**

Lorenz truncated the Boussinesq equations of a fluid layer heated from below (Rayleigh–Bénard convection, following B. Saltzman 1962) to three Fourier modes: x is the intensity of the convective roll, y the temperature difference between rising and falling fluid, z the distortion of the vertical temperature profile. σ is the Prandtl number, ρ the Rayleigh number divided by its critical value, β a geometric factor. Fixed points: the origin (conduction) and, for $\rho > 1$, the two rolls $C_{\pm} = (\pm\sqrt{\beta(\rho-1)}, \pm\sqrt{\beta(\rho-1)}, \rho-1)$, the centres of the two lobes; for $\sigma = 10$, $\beta = 8/3$ they lose stability in a subcritical Hopf bifurcation at $\rho \approx 24.74$, so at $\rho = 28$ all three fixed points are unstable.

The recipe. Everything is in three steps [► V3.1 @ 13:34]:

1. code the right-hand side $f(t, \mathbf{x}; \beta)$ as a function. The integrator wants a function of $(t, \mathbf{x})$ even when f does not depend on t ; the parameters are “locked in” by wrapping it (in Matlab `@(t,x) lorenz(t,x,Beta)`, in Python `args=(beta,)`) [► V3.1 @ 3:08] [► V3.1 @ 8:42][► V3.1 @ 9:23];
2. choose parameters, initial condition $\mathbf{x}_0 = (0, 1, 20)$, time step $\Delta t = 0.001$ and span up to $t = 50$, i.e. 50 000 samples [► V3.1 @ 7:20][► V3.1 @ 7:40];
3. call an adaptive Runge–Kutta integrator (Matlab `ode45`) with tight relative and absolute tolerances, 10^{-12} : the integrator shrinks its internal step until the error estimate meets them [► V3.1 @ 10:05][► V3.1 @ 10:49]. The output is a $50\,000 \times 3$ array: one column per variable, one row per time [► V3.1 @ 11:50].

PYTHON: Lorenz attractor and a blob of initial conditions (`../code/ch03_lorenz.py`)

```
import numpy as np
from scipy.integrate import solve_ivp
import matplotlib.pyplot as plt

def lorenz(t, x, beta):
    # t first, like ode45 / solve_ivp expect
    sigma, rho, b = beta
    return [sigma * (x[1] - x[0]),
            x[0] * (rho - x[2]) - x[1],
            x[0] * x[1] - b * x[2]]

beta = (10, 28, 8 / 3)
x0 = [0, 1, 20]
dt = 0.001
t = np.arange(dt, 50 + dt / 2, dt)
sol = solve_ivp(lorenz, (0, 50), x0, args=(beta,), t_eval=t,
                method='DOP853', rtol=1e-12, atol=1e-12)
X = sol.y.T
print(X.shape)

# a small blob of initial conditions: sensitive dependence
x0b = np.array(x0) + 1e-3 * np.random.default_rng(0).standard_normal((200, 3))
ends = [solve_ivp(lorenz, (0, 15), p, args=(beta,), rtol=1e-10, atol=1e-10).y[:, -1]
        for p in x0b]
print('blob size: 1e-3 ->', np.ptp(np.array(ends), axis=0).round(1))

ax = plt.figure(figsize=(5, 4)).add_subplot(projection='3d')
ax.plot(*X.T, lw=0.3, color='k')
ax.scatter(*np.array(ends).T, s=4, color='#D32F2F')
ax.set_xlabel('x'); ax.set_ylabel('y'); ax.set_zlabel('z')
plt.savefig('lecture-notes/figures/ch03-lorenz.png', dpi=200, bbox_inches='tight')
```

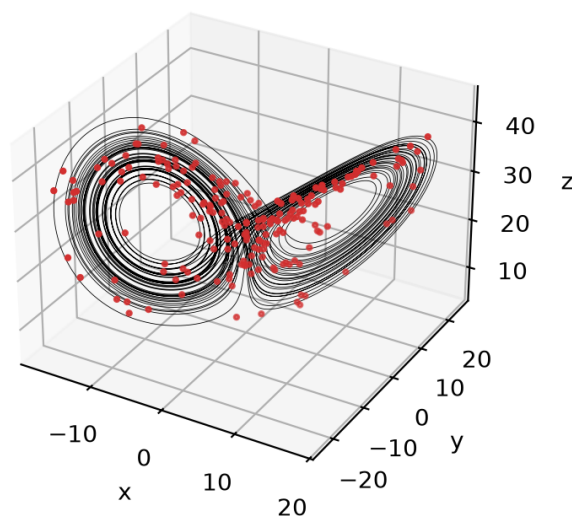


Figure 3.1: Output of `ch03_lorenz.py`: the Lorenz butterfly (black), and in red where 200 initial conditions, all within $\sim 10^{-3}$ of $\mathbf{x}_0$, are at $t = 15$. They have spread over the whole attractor (their range is ~ 35 – 45 in each coordinate).

Sensitive dependence. Brunton closes with a movie of a small red blob of initial conditions integrated through the system: it stretches, folds and mixes over the butterfly wings [► V3.1 @ 13:55][► V3.1 @ 14:15]. Any uncertainty in the initial condition grows exponentially; two points starting very close end up far apart, and after long enough only a statistical description of where the points are is possible [► V3.1 @ 14:36]. The red points of [fig. 3.1](#) reproduce this.

Exercise 3.1 (Lorenz fixed points). Find the fixed points of (3.1) for $\rho > 1$ and compute the Jacobian at the origin. Show that the origin is stable for $\rho < 1$ and a saddle with a one-dimensional unstable manifold for $\rho > 1$. Show that the volume of phase space contracts at the constant rate $\nabla \cdot \mathbf{f} = -(\sigma + 1 + \beta)$, and conclude that the attractor has zero volume.

Exercise 3.2 (Two nearby trajectories). Modify `ch03_lorenz.py` to integrate $\mathbf{x}_0$ and $\mathbf{x}_0 + 10^{-8}(1, 0, 0)$, and plot $\log\|\delta\mathbf{x}(t)\|$. Fit the slope of the linear part: compare with the largest Lyapunov exponent $\lambda_1 \approx 0.906$. Why does the curve saturate?

Exercise 3.3 (Tolerance matters). Integrate Lorenz to $t = 50$ with `rtol` = $10^{-3}, 10^{-6}, 10^{-9}, 10^{-12}$ and compare the final states. Up to which time do the trajectories agree? Relate this to the prediction horizon of Chapter 2.

COURSE NOTE: Matlab and Python

Matlab's `ode45` is the Dormand–Prince 5(4) pair; scipy's equivalent is `method='RK45'`. Here `DOP853` (order 8) is used because at tolerance 10^{-12} it is much faster. `t_eval` plays the role of `tspan`: the integrator chooses its own steps and interpolates at the requested times.

3.2 From continuous to discrete time

A continuous-time system $\dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}(t))$ (the populations of a predator and a prey, or anything else) [► V3.2 @ 0:01][► V3.2 @ 0:22] induces a discrete-time system $\mathbf{x}_{k+1} = \mathbf{F}(\mathbf{x}_k)$ [► V3.2 @ 1:03]. Usually k is the step of an integrator, but it need not be time at all [► V3.2 @ 1:23]. Sampling every Δt (say, counting a population once a day), $\mathbf{x}_k = \mathbf{x}(k\Delta t)$, and $\mathbf{F}$ is the flow

map of [definition 2.8](#) [► V3.2 @ 1:43][► V3.2 @ 2:04]:

$$\mathbf{x}_{k+1} = \mathbf{F}_{\Delta t}(\mathbf{x}_k) = \mathbf{x}_k + \int_{k\Delta t}^{(k+1)\Delta t} \mathbf{f}(\mathbf{x}(\tau)) d\tau, \quad (3.2)$$

where the state inside the integral moves along the trajectory [► V3.2 @ 2:25][► V3.2 @ 3:08]
[► V3.2 @ 3:50].

Proposition 3.1 (Maps are more general than flows). *Every continuous-time system defines a discrete-time one through (3.2), but not every map is the flow map of some ODE* [► V3.2 @ 4:10]
[► V3.2 @ 4:32].

Real populations, with seasons and other discrete events, are naturally maps [► V3.2 @ 4:52]. A mathematical obstruction: a flow map $\mathbf{F}_{\Delta t} = \mathbf{F}_{\Delta t/2} \circ \mathbf{F}_{\Delta t/2}$ is a diffeomorphism isotopic to the identity, so in one dimension it is increasing; the logistic map, which is not invertible, cannot be the flow map of any one-dimensional ODE (a one-dimensional ODE cannot even oscillate, let alone be chaotic).

Forward Euler as a crude flow map. Replace the derivative by a finite difference [► V3.2 @ 5:13][► V3.2 @ 5:34]:

$$\frac{\mathbf{x}_{k+1} - \mathbf{x}_k}{\Delta t} \approx \mathbf{f}(\mathbf{x}_k) \implies \mathbf{x}_{k+1} = \mathbf{x}_k + \Delta t \mathbf{f}(\mathbf{x}_k).$$

This is (3.2) with the integral approximated by a left rectangle: $\mathbf{f}$ held constant over the step [► V3.2 @ 6:16][► V3.2 @ 6:57]. Runge–Kutta schemes are better approximations of the same true flow map [► V3.2 @ 7:38]. Only a handful of systems have a closed-form flow map, notably the linear ones, $\mathbf{F}_t = e^{At}$; in general it must be approximated numerically [► V3.2 @ 7:59].

Proposition 3.2 (Order of Euler and RK4). *For smooth $\mathbf{f}$ the local error of one Euler step is $O(\Delta t^2)$ and the global error at a fixed final time is $O(\Delta t)$; for the classical fourth-order Runge–Kutta scheme they are $O(\Delta t^5)$ and $O(\Delta t^4)$.*

Proof. Taylor: $\mathbf{x}(t + \Delta t) = \mathbf{x} + \Delta t \mathbf{f} + \frac{1}{2} \Delta t^2 D\mathbf{f} \mathbf{f} + O(\Delta t^3)$; Euler keeps the first two terms, so its local error is $\frac{1}{2} \Delta t^2 D\mathbf{f} \mathbf{f}$. Over $N = T/\Delta t$ steps the local errors add up (each amplified at most by e^{LT} , L the Lipschitz constant), giving $N \cdot O(\Delta t^2) = O(\Delta t)$. RK4 matches the Taylor series up to Δt^4 (one checks it term by term), hence local $O(\Delta t^5)$, global $O(\Delta t^4)$. $\square$

PYTHON: Euler and RK4 against the exact flow map (../code/ch03_euler.py)

```
import numpy as np
from scipy.linalg import expm

A = np.array([[0.0, 1.0], [-1.0, 0.0]]) # harmonic oscillator, flow map exp(A t)
f = lambda x: A @ x
x0 = np.array([1.0, 0.0]); T = 2 * np.pi

def euler(x, dt):
    return x + dt * f(x)

def rk4(x, dt):
    k1 = f(x); k2 = f(x + dt / 2 * k1); k3 = f(x + dt / 2 * k2); k4 = f(x + dt * k3)
    return x + dt / 6 * (k1 + 2 * k2 + 2 * k3 + k4)

exact = expm(A * T) @ x0 # true flow map F_T
for dt in [0.1, 0.01, 0.001]:
    n = round(T / dt); dt = T / n # land exactly on T
    for step in (euler, rk4):
        x = x0.copy()
```

```

for _ in range(n):
    x = step(x, dt)
print(f'dt={dt:.4f} {step.__name__:5} error={np.linalg.norm(x - exact):.1e}')

```

For the harmonic oscillator over one period the script prints, for $\Delta t \approx 0.1, 0.01, 0.001$: Euler errors 3.7×10^{-1} , 3.2×10^{-2} , 3.1×10^{-3} (a factor 10 per factor 10 in Δt : first order); RK4 errors 5.2×10^{-6} , 5.2×10^{-10} , 5.3×10^{-14} (a factor 10^4 : fourth order).

Exercise 3.4 (Backward Euler). For $\dot{\mathbf{x}} = A\mathbf{x}$, backward Euler is $\mathbf{x}_{k+1} = (I - \Delta t A)^{-1} \mathbf{x}_k$. Compute its eigenvalues for the harmonic oscillator and show that it spirals *inward*. Which combination of forward and backward Euler (the trapezoidal rule) preserves $|\mu| = 1$ exactly?

Example 3.3 (Euler spirals outward). For $\dot{\mathbf{x}} = A\mathbf{x}$ with $A = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}$ Euler gives $\mathbf{x}_{k+1} = (I + \Delta t A)\mathbf{x}_k$, whose eigenvalues $1 \pm i\Delta t$ have modulus $\sqrt{1 + \Delta t^2} > 1$: the discrete orbit spirals out, energy grows by a factor $(1 + \Delta t^2)$ per step, while the true eigenvalues $e^{\pm i\Delta t}$ lie on the unit circle.

3.3 The logistic map

The model. The logistic map

$$x_{k+1} = \beta x_k (1 - x_k) \quad (3.3)$$

is a crude model of a population [► V3.2 @ 8:40][► V3.3 @ 14:39]. Start from exponential growth, $x_{k+1} = \beta x_k$; resources are finite, so with a carrying capacity normalized to 1 the growth factor must go to zero as $x \rightarrow 1$; multiplying by $(1 - x_k)$ does that [► V3.3 @ 14:59][► V3.3 @ 15:20]. It has fixed points for zero and for “full” population, and as β is turned up it becomes chaotic [► V3.2 @ 9:01][► V3.2 @ 9:22].

The algorithm [► V3.3 @ 1:03][► V3.3 @ 1:45]:

1. for each β on a grid from 0 to 4, start from $x_0 = 0.5$;
2. iterate a few thousand times without saving, to remove the transient and land on the attractor [► V3.3 @ 2:26][► V3.3 @ 3:28];
3. iterate up to a thousand more times, saving the pairs (β, x_k) ; stop early if x_k returns within 10^{-3} of the first saved value (a fixed point or a short periodic orbit needs no more points) [► V3.3 @ 3:49][► V3.3 @ 4:52][► V3.3 @ 5:12];
4. scatter-plot all the pairs: the *bifurcation diagram* [► V3.3 @ 5:54].

PYTHON: bifurcation diagram of the logistic map (../code/ch03_logistic.py)

```

import numpy as np
import matplotlib.pyplot as plt

betas, xs = [], []
for beta in np.arange(0, 4, 0.001):          # beta < 4: x stays in [0, 1]
    x = 0.5
    for _ in range(2000):                    # transient: let x settle on the attractor
        x = (x - x**2) * beta                # x_{k+1} = beta x_k (1 - x_k)
    xss = x
    for _ in range(1000):                    # store the steady state
        x = (x - x**2) * beta
        betas.append(beta); xs.append(x)
        if abs(x - xss) < 1e-3:              # back to the start: periodic, stop early
            break
print(len(xs), 'points')

```

```
fig, ax = plt.subplots(1, 2, figsize=(9, 3.2))
for a, lim in zip(ax, [(0, 4), (3, 4)]):
    a.plot(betas, xs, 'b', color='#1565C0', alpha=0.5)
    a.set_xlim(lim); a.set_xlabel(r'$\beta$'); a.set_ylabel(r'$x$')
plt.tight_layout()
plt.savefig('lecture-notes/figures/ch03-logistic.png', dpi=200)
```

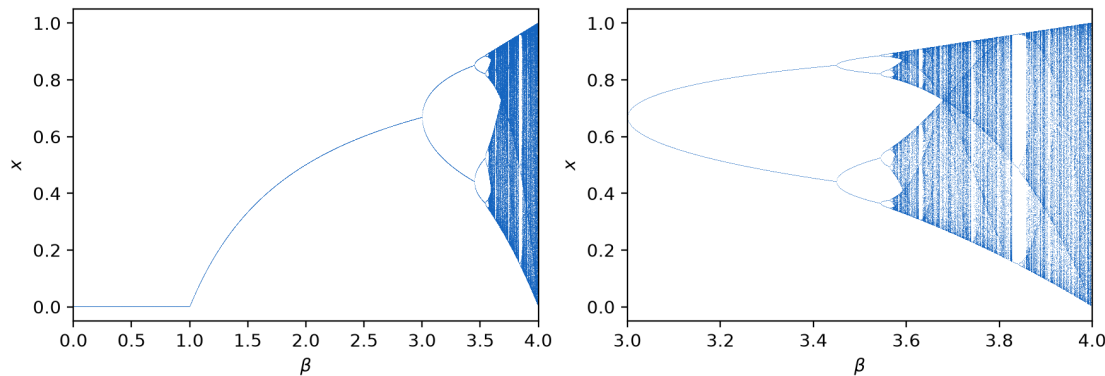


Figure 3.2: Output of `ch03_logistic.py`: bifurcation diagram of the logistic map (left, $\beta \in [0, 4)$) and zoom on $\beta \in [3, 4]$ (right). In the video the axes are flipped so that the picture looks like mountains, “mathematical mountains” [► V3.3 @ 8:43][► V3.3 @ 13:57].

Reading the diagram [► V3.3 @ 7:20]:

- $0 \leq \beta < 1$: every orbit goes to $x = 0$, the population dies out;
- $1 < \beta < 3$: a single nonzero fixed point, reached after a transient and kept forever [► V3.3 @ 7:40];
- at $\beta = 3$ the fixed point splits into two values visited alternately, a period-2 orbit: the population is bigger, smaller, bigger, smaller from one generation to the next [► V3.3 @ 8:02];
- increasing β the period doubles again, to 4 (Brunton calls it “quasi periodic”, but it is a period-4 orbit), then 8, 16, ... [► V3.3 @ 8:22];
- between about 3.57 and 4 the system is chaotic, with windows of periodic behaviour (the clear window near $\beta \approx 3.83$ is a period-3 orbit) [► V3.3 @ 8:22][► V3.3 @ 13:16].

Keeping the transients in the plot shows the lines along which x jumps from 0.5 to the attractor [► V3.3 @ 11:08][► V3.3 @ 11:52]; that is why they are usually removed [► V3.3 @ 12:14].

COURSE NOTE: Operator precedence

In the video the update is written as “`xold - xold^2 times beta`”, i.e. $(x - x^2)\beta = \beta x(1 - x)$. Typed literally as `x - x**2*beta` it would be the map $x - \beta x^2$, which is conjugate to the logistic map at fixed parameter 1 and never becomes chaotic (and for $\beta > 2$ leaves $[0, 1]$ and diverges): a bug made, and caught, while writing these notes.

Proposition 3.4 (First two bifurcations). For (3.3), $F(x) = \beta x(1 - x)$:

- (i) the fixed points are $x = 0$ and $x^* = 1 - 1/\beta$; $x = 0$ is stable for $\beta < 1$, and x^* is stable for $1 < \beta < 3$;
- (ii) at $\beta = 3$, $F'(x^*) = -1$ and a stable period-2 orbit is born (period-doubling, or flip, bifurcation); it is stable for $3 < \beta < 1 + \sqrt{6} \approx 3.449$.

INTUITION: Faster growth, more chaos

The moral Brunton draws from this crude model: the faster the growth rate, the more erratic the population; “breakneck population growth” is a bad idea [► V3.3 @ 15:41] [► V3.3 @ 16:02]. The physical reason is overshoot: with a large β a big population produces a crash the next year, and the discrete delay (one generation) turns a smooth relaxation into oscillations.

Exercise 3.5 (Period three). Plot $F^3(x) - x$ for the logistic map near $\beta = 1 + \sqrt{8} \approx 3.828$ and show that a pair of period-3 orbits is born there in a saddle-node (tangent) bifurcation. Run `ch03_logistic.py` zoomed on $[3.82, 3.86]$ and describe what happens at the edges of the window. (Li and Yorke, 1975: “period three implies chaos”.)

Exercise 3.6 (Feigenbaum’s constant). Using bisection on the multiplier of the period-2ⁿ orbit, or by reading the zoomed bifurcation diagram, estimate the first four period-doubling values $\beta_1 = 3$, $\beta_2 \approx 3.449$, β_3 , β_4 , and the ratios $(\beta_n - \beta_{n-1})/(\beta_{n+1} - \beta_n)$.

KEY FORMULAS

- Lorenz: $\dot{x} = \sigma(y - x)$, $\dot{y} = x(\rho - z) - y$, $\dot{z} = xy - \beta z$; $(10, 28, 8/3)$; $\mathbf{x}_0 = (0, 1, 20)$.
- Flow map $\mathbf{x}_{k+1} = \mathbf{x}_k + \int_{k\Delta t}^{(k+1)\Delta t} \mathbf{f}(\mathbf{x}(\tau)) d\tau$; Euler $\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta t \mathbf{f}(\mathbf{x}_k)$, global error $O(\Delta t)$; RK4 $O(\Delta t^4)$.
- Logistic $x_{k+1} = \beta x_k(1 - x_k)$: $x^* = 1 - 1/\beta$ stable for $1 < \beta < 3$; period doubling at 3 , $1 + \sqrt{6}$, ...; chaos beyond ≈ 3.57 .

Proof. A fixed point of a map is stable if $|F'(x^*)| < 1$ (the linearization of $x_{k+1} = F(x_k)$ is $\Delta x_{k+1} = F'(x^*)\Delta x_k$). $F'(x) = \beta(1 - 2x)$, so $F'(0) = \beta$ and $F'(x^*) = \beta(1 - 2 + 2/\beta) = 2 - \beta$, with $|2 - \beta| < 1 \iff 1 < \beta < 3$. At $\beta = 3$ the multiplier crosses -1 : deviations alternate in sign without decaying. The period-2 points solve $F(F(x)) = x$; dividing out the fixed points leaves $\beta^2 x^2 - \beta(\beta + 1)x + (\beta + 1) = 0$, real for $\beta > 3$; the multiplier of the orbit is $F'(x_1)F'(x_2) = -\beta^2 + 2\beta + 4$, which equals -1 at $\beta = 1 + \sqrt{6}$. $\square$

HISTORY: May, Feigenbaum and universality

The ecologist Robert May made the logistic map famous with *Simple mathematical models with very complicated dynamics*, Nature 261 (1976) 459–467, a plea to teach students that simple nonlinear maps can be chaotic. Mitchell Feigenbaum (Los Alamos, 1975–78) found, at first with an HP-65 pocket calculator, that the parameter values of successive period doublings converge geometrically with ratio $\delta = 4.6692\dots$, the same for every map with a quadratic maximum: a universal constant, later explained by renormalization. Libchaber and Maurer saw the cascade in convecting liquid helium in 1980. References: R. M. May, Nature 261 (1976); M. J. Feigenbaum, J. Stat. Phys. 19 (1978) 25–52; S. Strogatz, *Nonlinear Dynamics and Chaos*, ch. 10.

Part II

Learning models from data: regression

Chapter 4

Dynamic Mode Decomposition

SOURCES FOR THIS CHAPTER

Video	Length	Content
V4.1 Dynamic Mode Decomposition (Overview)	18:18	Snapshot matrices X, X' ; the best-fit linear operator A ; the algorithm (SVD, $\tilde{A}$, eigendecomposition, exact modes); truncation; prediction; DMD = PCA in space + Fourier in time.
V4.2 Dynamic Mode Decomposition (Examples)	7:43	Connection to Koopman; extensions (compressed, multiresolution, control, randomized, streaming, denoising, extended); applications; the toy video: PCA vs ICA vs DMD.
V4.3 Dynamic Mode Decomposition (Code)	8:15	The cylinder-wake code of the DMD book: singular values, rank $r = 21$, modes, eigenvalues on the unit circle.
V4.4 Compressed Sensing and Dynamic Mode Decomposition	30:35	Compressed sensing; compressive-sampling DMD (L1 reconstruction of modes) and compressed DMD (modes from the full X'); error analysis; invariance of DMD under unitary transformations.

Transcripts materials/transcripts/C04-V0*.txt. **Book** Sec. 7.2 (DMD); Sec. 1.4–1.5 (SVD, pseudo-inverse, PCA); Ch. 3 (compressed sensing). **Code** code/ch04_dmd.py, ch04_cdmd.py. The cylinder data of the videos are not used (they are on dmdbook.com); a toy data set reproduces the same effects.

Motivation

DMD is the first data-driven method of the course and the most widely used. From a movie of a high-dimensional system it extracts, purely from data, (a) a few spatial patterns, the *modes*, and (b) for each one a single complex frequency, i.e. a pure oscillation with exponential growth or decay; together they form a linear reduced-order model [► V4.1 @ 0:02]. It needs no equations of motion: “turn the crank” on any time-resolved data [► V4.1 @ 1:26]. For a physicist: it is a numerically robust way to find the normal modes of a system you can only film.

4.1 The DMD algorithm

History and scope. DMD was introduced in fluid dynamics by Peter Schmid and later connected to Koopman operator theory by Clancy Rowley, Igor Mezić and collaborators [► V4.1 @ 0:44]. Since then it has been applied to disease modelling, neuroscience, robotics,

finance, plasmas [► V4.1 @ 1:05], and it works equally on experimental data such as PIV images [► V4.1 @ 1:46].

Step 1: data. Take a movie of the flow, e.g. vortex shedding behind a cylinder, cut it into snapshots $\mathbf{x}_1, \dots, \mathbf{x}_m$ (each a vorticity field with $\sim 10^5$ values, reshaped into a tall column), and arrange them in two matrices shifted by one time step [► V4.1 @ 2:07] [► V4.1 @ 2:48][► V4.1 @ 3:29]:

$$X = [\mathbf{x}_1 \ \mathbf{x}_2 \ \cdots \ \mathbf{x}_{m-1}], \quad X' = [\mathbf{x}_2 \ \mathbf{x}_3 \ \cdots \ \mathbf{x}_m]. \quad (4.1)$$

For fluids these are very tall and skinny: $n \sim 10^5\text{--}10^9$ rows, $m \sim 10^2\text{--}10^3$ columns [► V4.1 @ 4:10].

Step 2: the hypothesis. Look for the best-fit linear operator with [► V4.1 @ 4:31][► V4.1 @ 4:51]

$$X' \approx AX, \quad A = \underset{A}{\operatorname{argmin}} \|X' - AX\|_F = X'X^\dagger, \quad (4.2)$$

where $X^\dagger$ is the Moore–Penrose pseudo-inverse. This does not claim that the dynamics is linear (Navier–Stokes is not): A simply has enough freedom to approximate the evolution of the data [► V4.4 @ 2:51]. But A is $n \times n$, a trillion entries for $n = 10^6$: it cannot even be stored [► V4.1 @ 5:12][► V4.1 @ 5:32].

Definition 4.1 (New concepts: DMD eigenvalues and modes). The **dynamic mode decomposition** of the data (4.1) is the set of leading eigenvalues λ_j (**DMD eigenvalues**) and eigenvectors ϕ_j (**DMD modes**) of $A = X'X^\dagger$, computed without forming A [► V4.1 @ 5:52][► V4.1 @ 14:07]. Each mode, reshaped, is a spatial pattern (an “eigen flow field”) that evolves in time as λ_j^k : a pure tone, a pure exponential growth or decay, or a growing or decaying oscillation [► V4.1 @ 6:13][► V4.1 @ 6:33].

HISTORY: Schmid, Rowley, Mezić

P. J. Schmid presented DMD in 2008 and published it as *Dynamic mode decomposition of numerical and experimental data*, J. Fluid Mech. 656 (2010) 5–28. C. W. Rowley, I. Mezić, S. Bagheri, P. Schlatter and D. S. Henningson, *Spectral analysis of nonlinear flows*, J. Fluid Mech. 641 (2009) 115–127, showed that DMD approximates the eigenvalues and modes of the Koopman operator, building on Mezić’s *Koopman modes* (2005). The “exact DMD” used today is from J. H. Tu, C. W. Rowley, D. M. Luchtenburg, S. L. Brunton and J. N. Kutz, J. Comput. Dyn. 1 (2014) 391–421. *References*: the papers cited; Kutz, Brunton, Brunton, Proctor, *Dynamic Mode Decomposition* (SIAM, 2016).

Step 3: the algorithm. If enough snapshots are collected, a few coherent patterns dominate, even with a million degrees of freedom [► V4.1 @ 7:56][► V4.1 @ 8:17]. The algorithm exploits this [► V4.1 @ 8:37]–[► V4.1 @ 12:23]:

1. **SVD**: $X = U\Sigma V^*$ (economy size), truncated to rank r : $U_r \in \mathbb{C}^{n \times r}$, Σ_r , V_r . The columns of U are the POD modes, ordered by the variance they capture [► V4.1 @ 8:37] [► V4.1 @ 8:57];

2. **projected operator**: with $A \approx X'V_r\Sigma_r^{-1}U_r^*$,

$$\tilde{A} = U_r^* A U_r = U_r^* X' V_r \Sigma_r^{-1} \in \mathbb{C}^{r \times r}, \quad (4.3)$$

the best-fit linear dynamics of the POD coefficients [► V4.1 @ 9:18][► V4.1 @ 9:59] [► V4.1 @ 10:41];

3. **eigendecomposition**: $\tilde{A}W = W\Lambda$; the eigenvalues of $\tilde{A}$ are the nonzero eigenvalues of A [► V4.1 @ 11:02][► V4.1 @ 11:22];
4. **exact DMD modes** (Tu et al. 2014, slightly different from Schmid’s $U_r W$):

$$\Phi = X'V_r\Sigma_r^{-1}W \quad (4.4)$$

[► V4.1 @ 11:42][► V4.1 @ 12:03].

That is all: four or five lines of code, fast and scalable [► V4.1 @ 12:44].

Proposition 4.2 (Why the algorithm works). Let $A = X'V_r\Sigma_r^{-1}U_r^*$ (the best fit restricted to the span of U_r), $\tilde{A} = U_r^*AU_r$, and $\tilde{A}\mathbf{w} = \lambda\mathbf{w}$ with $\lambda \neq 0$. Then $\phi = X'V_r\Sigma_r^{-1}\mathbf{w}$ satisfies $A\phi = \lambda\phi$, and $\phi \neq 0$.

Proof. Note $\tilde{A} = U_r^*X'V_r\Sigma_r^{-1}$, so $U_r^*\phi = \tilde{A}\mathbf{w} = \lambda\mathbf{w}$. Then $A\phi = X'V_r\Sigma_r^{-1}(U_r^*\phi) = \lambda X'V_r\Sigma_r^{-1}\mathbf{w} = \lambda\phi$. If $\phi = 0$ then $\lambda\mathbf{w} = U_r^*\phi = 0$, impossible. Conversely every eigenvector of A with $\lambda \neq 0$ lies in the range of $X'V_r$, so this gives all nonzero eigenvalues. $\square$

INTUITION: Lifting back with the data

The small eigenvector $\mathbf{w}$ lives in POD-coefficient space; (4.4) lifts it back to the full space by taking a linear combination of the *shifted* snapshots, columns of X' [► V4.4 @ 6:38]. Schmid's projected modes $U_r\mathbf{w}$ are the projection of ϕ onto the POD subspace; the two coincide when the columns of X' lie in the span of X .

Truncation. With $r = m - 1$ (exact SVD) one gets $m - 1$ eigenvalues and modes. If the first, say, five POD modes capture 99% of the energy, keep $r = 5$: a 5×5 $\tilde{A}$, five eigenvalues, five modes [► V4.1 @ 13:06][► V4.1 @ 13:27]. The rank r is the main knob of DMD [► V4.3 @ 3:28].

Prediction. With b_j the amplitude of each mode in the initial condition (e.g. $\mathbf{b} = \Phi^\dagger \mathbf{x}_1$) [► V4.1 @ 14:28],

$$\mathbf{x}_{k+1} \approx \sum_{j=1}^r b_j \lambda_j^k \phi_j, \quad \text{or} \quad \mathbf{x}(t) \approx \sum_j b_j e^{\omega_j t} \phi_j, \quad \omega_j = \frac{\ln \lambda_j}{\Delta t}. \quad (4.5)$$

This works well for periodic or quasi-periodic systems, even strongly nonlinear ones; strong transients, intermittency or non-stationarity need fancier regressions and a nonlinear framework [► V4.1 @ 14:49][► V4.1 @ 15:09].

Remark 4.3 (DMD is regression). The key observation (J. Tu): DMD is a best-fit linear regression from X to X' , so every regression technique applies: noise-robust regression, sparsity-promoting regression, and so on [► V4.1 @ 15:30][► V4.1 @ 15:50].

INTUITION: PCA and Fourier “had a baby”

The SVD step is PCA: it finds the fewest patterns that explain the data. Then a linear regression on the principal components ($\tilde{A}$) is diagonalized to find linear combinations of them that each have a single frequency in time. DMD is PCA in space plus the Fourier transform in time [► V4.1 @ 16:11][► V4.1 @ 16:52][► V4.1 @ 17:13]. Unlike a Fourier transform, the frequencies are not fixed by the record length and can grow or decay.

KEY FORMULAS

- $X' \approx AX$, $A = X'X^\dagger$; $X = U_r\Sigma_r V_r^*$; $\tilde{A} = U_r^*X'V_r\Sigma_r^{-1}$; $\tilde{A}\mathbf{w} = W\Lambda$; $\Phi = X'V_r\Sigma_r^{-1}W$.
- $\mathbf{x}_k \approx \Phi\Lambda^{k-1}\mathbf{b}$, $\omega_j = \ln \lambda_j / \Delta t$: $\text{Re } \omega_j$ growth rate, $\text{Im } \omega_j$ angular frequency.

Exercise 4.1 (DMD of a linear system). Generate data from $\mathbf{x}_{k+1} = A\mathbf{x}_k$ with a random $A \in \mathbb{R}^{50 \times 50}$ of rank 4 (scaled so that its eigenvalues have modulus ≤ 1). Show numerically that DMD with $r = 4$ recovers the nonzero eigenvalues of A exactly. What happens with $r = 6$? With noise?

Exercise 4.2 (Pseudo-inverse). Show that $A = X'X^\dagger$ minimizes $\|X' - AX\|_F$, and that among the minimizers it has the smallest Frobenius norm. Write $X^\dagger$ via the SVD.

4.2 Extensions, applications, and a toy example

Connection to Koopman. Although DMD fits a linear model, it is connected to strongly nonlinear dynamics: its eigenvalues and modes approximate those of the Koopman operator (Chapter 6) [► V4.2 @ 0:23][► V4.2 @ 0:44].

Extensions. Because DMD is linear regression, it is easily extended [► V4.2 @ 1:05]:

- **compressed / compressed-sensing DMD**, for spatially under-resolved data, and compressed sensing in time for data that are not time resolved (section 4.4) [► V4.2 @ 1:26];
- **multiresolution DMD** (N. Kutz): DMD on a hierarchy of time windows, separating slow, medium and fast modes; it captures transient phenomena such as El Niño [► V4.2 @ 1:46];
- **DMD with control** (J. Proctor): identify the model while the system is actuated, $\mathbf{x}_{k+1} \approx \mathbf{A}\mathbf{x}_k + \mathbf{B}\mathbf{u}_k$ [► V4.2 @ 2:07];
- **randomized DMD** (N. B. Erichson and others) for very large data [► V4.2 @ 2:28];
- **streaming, denoising and extended DMD** (augmenting the measurements with nonlinear functions of them) [► V4.2 @ 2:48][► V4.2 @ 3:09].

Literally dozens of papers add innovations to the original algorithm [► V4.2 @ 3:29].

APPLICATION: Where DMD is used

Most DMD papers are in fluid dynamics (a jet in cross flow by Rowley, Mezić et al., boundary layers, combustion, shock–boundary-layer interaction), but also neuroscience, disease modelling, weather and climate, robotics, finance, plasmas and video processing [► V4.2 @ 3:50][► V4.2 @ 4:10]. The common thread: high-dimensional data from a probably nonlinear, multiscale process with dominant coherent structures [► V4.2 @ 4:31].

CASE STUDY: A toy video: PCA, ICA and DMD

B. W. Brunton’s toy movie: 80×80 pixels (6400-dimensional data), with noise, in which two patterns oscillate at different frequencies, a fast square and a slow Gaussian; without noise it is a rank-2 system [► V4.2 @ 4:51][► V4.2 @ 5:12][► V4.2 @ 5:33].

- **PCA** (the SVD) gets the rank right, 2, but its modes, ordered by energy, *mix* the two frequencies [► V4.2 @ 5:53][► V4.2 @ 6:14];
- **ICA** extracts two shapes corresponding to fast and slow, but they are noisy and there is no model of their time evolution [► V4.2 @ 6:35];
- **DMD** separates square and Gaussian cleanly, denoises them, and identifies the two frequencies: a linear dynamical model is under the hood [► V4.2 @ 6:55][► V4.2 @ 7:15].

PYTHON: DMD function and the toy video (./code/ch04_dmd.py)

```
import numpy as np
import matplotlib.pyplot as plt

def dmd(X, Xp, r):
    """Exact DMD (Tu et al. 2014): eigenvalues and modes of the best-fit A with  $Xp \sim A$ 
```

```

X. """
U, S, Vh = np.linalg.svd(X, full_matrices=False) # economy SVD
U, S, V = U[:, :r], S[:r], Vh[:r].conj().T      # rank-r truncation
Atilde = U.conj().T @ Xp @ V / S                # r x r: U* A U
lam, W = np.linalg.eig(Atilde)                  # exact DMD modes
Phi = Xp @ V / S @ W
return lam, Phi, S

# toy "video" (after B. W. Brunton): an 80x80 square blinking fast and a Gaussian
# blinking slowly
n, m, dt = 80, 200, 0.05
xx, yy = np.meshgrid(np.linspace(-1, 1, n), np.linspace(-1, 1, n))
square = ((abs(xx - 0.15) < 0.3) & (abs(yy - 0.15) < 0.3)).astype(float)
gauss = np.exp(-((xx + 0.1)**2 + (yy + 0.1)**2) / 0.15) # overlapping shapes
t = np.arange(m) * dt
w1, w2 = 2 * np.pi * 2.0, 2 * np.pi * 0.4           # fast and slow
# frequencies
D = (np.outer(square.ravel(), np.exp(1j * w1 * t))
     + np.outer(gauss.ravel(), np.exp(1j * w2 * t))) # 6400 x 200 snapshots
D = D + 0.3 * np.random.default_rng(1).standard_normal(D.shape) # noise

X, Xp = D[:, :-1], D[:, 1:] # the two shifted matrices
lam, Phi, S = dmd(X, Xp, r=2)
omega = np.log(lam) / dt    # continuous-time
# eigenvalues
print('DMD frequencies (Hz):', np.sort(omega.imag / (2 * np.pi)).round(3)) # ~[0.4,
2.0]
print('growth rates      :', omega.real.round(3)) # ~0

U, s, _ = np.linalg.svd(X, full_matrices=False) # POD/PCA modes, for
# comparison
fig, ax = plt.subplots(1, 5, figsize=(12, 2.6))
ax[0].semilogy(s[:10], 'o-') # rank 2 + noise floor
ax[0].set_title('singular values')
for k in range(2):
    ax[1 + k].imshow(U[:, k].real.reshape(n, n), cmap='RdBu'); ax[1 + k].set_title(f'
POD mode {k+1}')
    ax[3 + k].imshow(abs(Phi[:, k]).reshape(n, n), cmap='viridis')
    ax[3 + k].set_title(f'DMD mode, {omega[k].imag / (2*np.pi):.2f} Hz')
for a in ax[1:]:
    a.set_xticks([]); a.set_yticks([])
plt.tight_layout(); plt.savefig('lecture-notes/figures/ch04-dmd-toy.png', dpi=200)

```

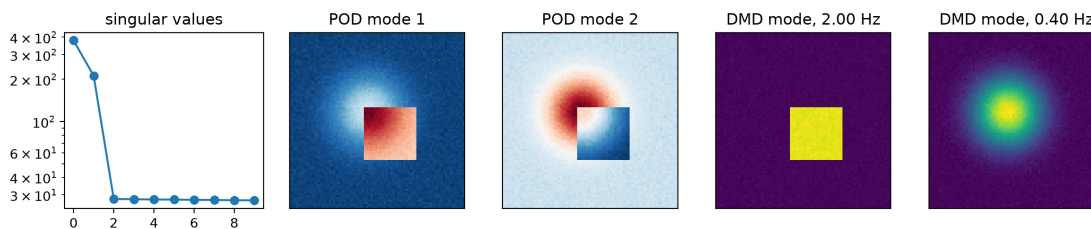


Figure 4.1: Output of `ch04_dmd.py`. Singular values: rank 2 plus a noise floor. POD modes 1–2 mix the square and the Gaussian (which overlap and are not orthogonal); the two DMD modes (modulus) separate them, with the right frequencies, 2.00 and 0.40 Hz, and growth rates ≈ 0 (the script prints -0.003 and -0.008 s^{-1} , a small bias due to the noise).

Remark 4.4 (Noise bias). With noise, DMD eigenvalues are biased towards the inside of the unit circle (spurious damping), because least squares attributes all the noise to X' and none to X . Remedies are noise-robust regressions: forward-backward DMD, total-least-squares DMD, optimized DMD.

Exercise 4.3 (Mean subtraction). Run DMD on the toy data plus a constant background. Show that the mean appears as a mode with $\lambda = 1$. What happens to DMD if the mean is subtracted first? (Hint: Chen, Tu, Rowley 2012 show that mean-subtracted DMD reduces to a temporal DFT.)

4.3 DMD in practice: the cylinder wake

The code of the DMD book (dmdbook.com) for the flow past a cylinder [► V4.3 @ 0:22] [► V4.3 @ 1:04]:

1. load the vorticity fields, one per column, and split them into X (columns $1..m-1$) and X' (columns $2..m$) [► V4.3 @ 1:24][► V4.3 @ 1:45];
2. economy SVD of X (a full SVD would be far too expensive) [► V4.3 @ 2:05];
3. plot the singular values on a log scale, always a good idea [► V4.3 @ 2:05]: the first one is the mean flow (not subtracted), then they come in *pairs*, the sine/cosine pairs of the periodic shedding and its harmonics [► V4.3 @ 2:47][► V4.3 @ 3:08];
4. truncate at $r = 21$: the mean plus ten harmonic pairs [► V4.3 @ 3:28][► V4.3 @ 3:48];
5. compute $\tilde{A}$ (21×21), its eigendecomposition and the modes Φ , “three lines” [► V4.3 @ 4:09][► V4.3 @ 4:50][► V4.3 @ 6:33];
6. plot the modes: pairs oscillating at one frequency, finer and finer for higher harmonics [► V4.3 @ 5:32][► V4.3 @ 5:52];
7. plot the eigenvalues with the unit circle (discrete time): complex-conjugate pairs on the circle, which explains the sine/cosine pairs of modes oscillating out of phase [► V4.3 @ 6:53][► V4.3 @ 7:14].

Here the DMD modes look much like the POD modes because the cylinder wake is a “very normal” system [► V4.3 @ 5:52][► V4.3 @ 6:12].

◊ BEYOND THE CORE: Why the modes come in pairs, and why $\text{DMD} \approx \text{POD}$ here

A real travelling or rotating pattern $\text{Re}(\phi e^{i\omega t}) = \phi_R \cos \omega t - \phi_I \sin \omega t$ needs two real spatial fields, hence two singular values of similar size and a conjugate pair $\lambda, \bar{\lambda}$. On the periodic attractor the time coefficients of different harmonics are orthogonal over a period, so the SVD, which produces orthogonal time series, cannot mix different frequencies, and the POD modes coincide (pairwise) with the DMD modes. When the dynamics is non-normal or transient this fails, and DMD and POD differ.

Exercise 4.4 (One real oscillation needs two modes). Take data $x(t) = \cos(\omega t) \mathbf{f}$ for a fixed spatial vector $\mathbf{f}$. What is the rank of X ? Can DMD find the frequency? Now take $\mathbf{f} \cos \omega t + \mathbf{g} \sin \omega t$ with $\mathbf{f} \perp \mathbf{g}$: show that $r = 2$ gives $\lambda = e^{\pm i\omega \Delta t}$. Relate this to the singular-value pairs of the cylinder. (This is the reason `ch04_dmd.py` uses complex data; Chapter 8 gives another cure: time delays.)

4.4 Compressed sensing and DMD

DMD needs a lot of data: acquiring and processing X, X' can be expensive [► V4.4 @ 9:42] [► V4.4 @ 10:03]. In PIV, for example, spatial and temporal resolution are limited by hardware: sometimes the data cannot even be transferred from the camera to memory fast enough [► V4.4 @ 10:23][► V4.4 @ 10:44]. Other uses of sparsity in DMD: selecting few important modes (Jovanović et al.), sparse sampling in time (Tu et al.) [► V4.4 @ 11:04][► V4.4 @ 11:25]. Here: sparse sampling in *space* [► V4.4 @ 11:46].

Compressed sensing in one paragraph. Most high-dimensional signals (images, flow fields, audio) are *sparse* in a suitable basis: in the Fourier domain most coefficients of an image are small and can be dropped, which is how compression works [► V4.4 @ 12:06] [► V4.4 @ 12:27][► V4.4 @ 12:48]. Compressed sensing (about ten years old at the time of the video) turns this around: measure much less and infer the sparse coefficients consistent with the measurements [► V4.4 @ 13:09][► V4.4 @ 13:30].

Definition 4.5 (New concepts: *sparsity, compressed sensing*). A vector $\mathbf{x} \in \mathbb{R}^n$ is K -*sparse* in a basis Ψ if $\mathbf{x} = \Psi \mathbf{s}$ with at most K nonzero entries in $\mathbf{s}$. Given $p \ll n$ measurements $\mathbf{y} = C\mathbf{x} = C\Psi\mathbf{s}$, **compressed sensing** recovers $\mathbf{s}$ by the convex problem

$$\min_{\mathbf{s}} \|\mathbf{s}\|_1 \quad \text{subject to} \quad \mathbf{y} = C\Psi\mathbf{s} \quad (4.6)$$

instead of the combinatorial search for the sparsest $\mathbf{s}$ (ℓ_0). It succeeds with high probability if $p \gtrsim K \log(n/K)$ and the measurements are *incoherent* with Ψ (e.g. random), so that $C\Psi$ satisfies the restricted isometry property: it acts almost like an isometry on sparse vectors [► V4.4 @ 13:51][► V4.4 @ 14:32][► V4.4 @ 14:52][► V4.4 @ 15:13].

The ℓ_1 problem is a linear program: its cost grows polynomially, so Moore's law lets it keep up with bigger data, which a brute-force search cannot [► V4.4 @ 15:13][► V4.4 @ 15:33]. Example: measure 1% of the velocities of a flow at random points, infer its sparse Fourier coefficients, and fill in the other 99% [► V4.4 @ 16:15][► V4.4 @ 16:36].

Two algorithms. With a measurement matrix $C \in \mathbb{R}^{p \times n}$ (e.g. random pixels), the compressed data are $Y = CX, Y' = CX'$ [► V4.4 @ 17:39][► V4.4 @ 17:59].

1. **Compressive-sampling (compressed-sensing) DMD:** only Y, Y' exist (sparse sensors, or trading spatial for temporal resolution). Do DMD on Y, Y' , getting Λ_Y, Φ_Y , then reconstruct each full mode by (4.6) with $\phi_Y = C\Psi\mathbf{s}$ [► V4.4 @ 18:20]. On the cylinder, ~ 1000 measurements of a $\sim 10^5$ -dimensional state (1%) reconstruct the leading modes well and the higher ones with some noise; the eigenvalues are almost exact, $\Lambda_X \approx \Lambda_Y$ to near machine precision [► V4.4 @ 18:41][► V4.4 @ 19:01].
2. **Compressed DMD:** the full data exist but the SVD is too expensive. Compress, do DMD on Y, Y' , and get the full modes with no ℓ_1 at all, as

$$\Phi_X \approx X'V_Y\Sigma_Y^{-1}W_Y, \quad (4.7)$$

linear combinations of the columns of the full X' [► V4.4 @ 19:43][► V4.4 @ 20:24]. On the cylinder 21 random pixels already give the first mode, 40 give all of them [► V4.4 @ 20:45]; cluster-sized computations fit on a laptop [► V4.4 @ 21:25].

The decision: only subsampled data $\Rightarrow$ compressive-sampling DMD with ℓ_1 ; full data but slow $\Rightarrow$ compressed DMD [► V4.4 @ 21:47][► V4.4 @ 22:28]. With enough buoys in the ocean

one could infer the DMD of a high-resolution sensor network [► V4.4 @ 23:09]. In the error analysis on a double-gyre flow, the mode error of compressed-sensing DMD decreases slowly (low at ~ 1000 measurements, 1%), that of compressed DMD much faster (very good at ~ 100) [► V4.4 @ 23:29][► V4.4 @ 24:10][► V4.4 @ 24:31].

PYTHON: compressed DMD on the toy video (./code/ch04_cdmd.py)

```
import numpy as np
from ch04_dmd import dmd, D, n          # same toy data (6400 pixels x 200 snapshots)

X, Xp = D[:, :-1], D[:, 1:]
lam_x, Phi_x, _ = dmd(X, Xp, r=2)       # DMD on the full data (reference)

rng = np.random.default_rng(2)
for p in [20, 100, 400]:                # number of random pixels measured
    C = np.zeros((p, n * n)); C[np.arange(p), rng.choice(n * n, p, replace=False)] = 1
    Y, Yp = C @ X, C @ Xp               # compressed data: p x (m-1)
    U, S, Vh = np.linalg.svd(Y, full_matrices=False)
    U, S, V = U[:, :2], S[:2], Vh[:2].conj().T
    lam_y, W = np.linalg.eig(U.conj().T @ Yp @ V / S)
    Phi = Xp @ V / S @ W                 # compressed DMD: modes from the full X'
    # compare with the reference (sort by frequency, normalise, remove the phase)
    ix, iy = np.argsort(np.angle(lam_x)), np.argsort(np.angle(lam_y))
    err = []
    for a, b in zip(Phi_x[:, ix].T, Phi[:, iy].T):
        a, b = a / np.linalg.norm(a), b / np.linalg.norm(b)
        err.append(np.linalg.norm(a - b * np.vdot(b, a) / abs(np.vdot(b, a))))
    print(f'p={p:3d} | eig error|={np.abs(lam_x[ix] - lam_y[iy]).max():.1e} mode errors={np.round(err, 3)}')
```

The script prints, for $p = 20, 100, 400$ random pixels out of 6400: eigenvalue errors 0.9, 1.4×10^{-2} , 5×10^{-3} and mode errors (0.41, 0.16), (0.02, 0.01), (0.008, 0.005). With 20 pixels too few of them fall on the square and the Gaussian, which are *localized*: in the pixel basis these signals are not incoherent with point measurements. 100 pixels (1.6%) are enough.

Why it works. Three facts [► V4.4 @ 24:51]:

1. DMD is invariant under *right* unitary transformations: shuffling the columns of X and of X' in the same way gives the same eigenvalues and modes, because the SVD is invariant [► V4.4 @ 25:12][► V4.4 @ 25:33];
2. DMD is almost invariant under *left* unitary transformations: if $Y = CX$, $Y' = CX'$ with C unitary (permuting rows, a spatial FFT, a POD basis), the eigenvalues are the same and the modes are $\Phi_Y = C\Phi_X$ [► V4.4 @ 25:53][► V4.4 @ 26:13][► V4.4 @ 26:34][► V4.4 @ 26:54];
3. a compression $C \in \mathbb{R}^{p \times n}$ is not unitary, but if the data are sparse, $X = \Psi S$, compressed sensing gives conditions under which $C\Psi$ acts like an isometry on sparse vectors, which is all that is needed [► V4.4 @ 27:35][► V4.4 @ 27:56][► V4.4 @ 28:17][► V4.4 @ 28:59].

The code of the paper (a sparse dynamical system on a torus driving five spatial Fourier coefficients) is available to try different compression ratios and samplings [► V4.4 @ 29:20][► V4.4 @ 29:40].

HISTORY: Compressed sensing

Compressed sensing was founded in 2004–2006 by E. Candès, J. Romberg and T. Tao (*Robust uncertainty principles*, IEEE Trans. Inf. Theory 52, 2006) and D. Donoho (*Compressed sensing*, same journal and year). An early inspiration: Candès reconstructed a Shepp–Logan phantom image almost exactly from very few Fourier samples using ℓ_1 (total-variation) minimization. The figure used in the video is from R. Baraniuk, *Compressive sensing*, IEEE Signal Process. Mag. (2007).
References: the papers cited; Brunton & Kutz, Ch. 3.

Proof of fact 2. If $Y = CX$ with $C^*C = I$, then $Y = (CU)\Sigma V^*$ is an SVD of Y , so $U_Y = CU$, $\Sigma_Y = \Sigma$, $V_Y = V$. Then $\tilde{A}_Y = U^*C^*CX'V\Sigma^{-1} = \tilde{A}_X$, so same Λ and W , and $\Phi_Y = CX'V\Sigma^{-1}W = C\Phi_X$. $\square$

Exercise 4.5 (Left-unitary invariance). Verify fact 2 numerically on the toy data with C the 2D FFT (normalized) and with a random orthogonal C . Then try a random Gaussian $C \in \mathbb{R}^{p \times 6400}$ and plot the eigenvalue error versus p .

Chapter 5

Sparse Identification of Nonlinear Dynamics

SOURCES FOR THIS CHAPTER

Video	Length	Content
V5.1 Sparse Identification of Nonlinear Dynamics (SINDy)	26:44	The SINDy regression on Lorenz; noisy data and TV derivatives; the cylinder wake and the mean-field model (history from Ruelle–Takens to Noack); kicking off the attractor; parametrized, time-dependent and forced models; SINDy in delay coordinates; SINDy with control; open questions.
V5.2 PDE FIND	10:36	(Speaker: Samuel Rudy; uploaded by N. Kutz.) Discovering PDEs $u_t = N(u, u_x, \dots)$ by sparse regression; derivatives; sequential threshold ridge regression; Navier–Stokes from 1% of the data; the soliton ambiguity (advection vs KdV); a gallery of PDEs.

Transcripts materials/transcripts/C05-V0*.txt. **Book** Sec. 7.3 (SINDy); Sec. 3.5 (sparse regression); Sec. 4.4–4.7 (regression, model selection). **Papers** S. L. Brunton, J. L. Proctor, J. N. Kutz, PNAS 113 (2016) 3932–3937; S. H. Rudy, S. L. Brunton, J. L. Proctor, J. N. Kutz, Sci. Adv. 3 (2017) e1602614. **Code** code/ch05_sindy.py, ch05_pdefind.py.

Motivation

DMD ([chapter 4](#)) fits a *linear* model to data. SINDy goes after the second primary challenge of [chapter 1](#), unknown dynamics, head on: it discovers the nonlinear right-hand side $\mathbf{f}$ itself, as a short formula, from measurements [[V5.1 @ 0:04](#)]. If we had $\mathbf{f}$ we could integrate trajectories, compute coherent structures and FTLEs, quantify uncertainty [[V5.1 @ 0:45](#)] [[V5.1 @ 1:06](#)]; in neuroscience or climate science we do not have it, so we want to discover it from data [[V5.1 @ 1:27](#)]. The key assumption is *parsimony*: in the right coordinates, $\mathbf{f}$ has only a few terms.

5.1 The SINDy algorithm

Sparsity in function space. In the Lorenz equations ([3.1](#)) the $\dot{x}$ equation contains only x and y ; $\dot{y}$ contains x , xz and y ; $\dot{z}$ contains z and xy [[V5.1 @ 1:49](#)][[V5.1 @ 2:09](#)]. Out of all possible right-hand sides this one is very special: it is *sparse* in the space of candidate functions [[V5.1 @ 2:30](#)][[V5.1 @ 3:11](#)].

Definition 5.1 (New concepts: **library**, **SINDy**). From m samples of the state and its derivative, build

$$X = \begin{bmatrix} \mathbf{x}^\top(t_1) \\ \vdots \\ \mathbf{x}^\top(t_m) \end{bmatrix} \in \mathbb{R}^{m \times n}, \quad \dot{X} \in \mathbb{R}^{m \times n},$$

and a **library** of candidate functions evaluated on the data, $\Theta(X) = [\mathbf{1} \ X \ X^{P_2} \ X^{P_3} \ \dots]$, where X^{P_2} holds all quadratic monomials ($x^2, xy, xz, y^2, \dots$), and so on [► V5.1 @ 3:32]. **SINDy** (Sparse Identification of Nonlinear DYnamics) solves

$$\dot{X} \approx \Theta(X) \Xi, \quad \Xi = [\xi_1 \dots \xi_n] \text{ sparse}, \quad (5.1)$$

one sparse regression per state variable: $\dot{x}_k \approx \Theta(X) \xi_k$ [► V5.1 @ 3:52][► V5.1 @ 4:12]. The identified model is $\dot{x}_k = \theta(\mathbf{x}^\top) \xi_k$, with θ the row of library functions.

Why sparse regression. Plain least squares would use “a little bit of all” the columns [► V5.1 @ 5:34]; an ℓ_1 -penalized (or thresholded) regression finds a good fit with as few terms as possible [► V5.1 @ 5:56]. Options: the LASSO of Tibshirani, or the *sequentially thresholded least squares* of the paper [► V5.1 @ 4:53].

Definition 5.2 (New concepts: **LASSO**, **sequentially thresholded least squares**). (a) **LASSO** (Least Absolute Shrinkage and Selection Operator, Tibshirani 1996): $\min_{\xi} \|\dot{X} - \Theta \xi\|_2^2 + \alpha \|\xi\|_1$.

(b) **STLSQ**: solve least squares; set to zero every coefficient with $|\xi_i| < \lambda$; solve least squares again on the surviving columns; repeat until the support stops changing. The threshold λ is the sparsity knob.

PYTHON: SINDy on the Lorenz system (../code/ch05_sindy.py)

```
import numpy as np
from itertools import combinations_with_replacement
from scipy.integrate import solve_ivp

def lorenz(t, x, s=10, r=28, b=8/3):
    return [s * (x[1] - x[0]), x[0] * (r - x[2]) - x[1], x[0] * x[1] - b * x[2]]

dt = 0.001
t = np.arange(0, 20, dt)
X = solve_ivp(lorenz, (0, 20), [-8, 8, 27], t_eval=t, rtol=1e-12, atol=1e-12).y.T
dX = np.gradient(X, dt, axis=0) # derivatives from the data (2nd
                                # -order FD)

def library(X, order=5, names='xyz'):
    """Theta(X): all monomials of the state up to the given order, and their names."""
    cols, labels = [np.ones(len(X))], ['1']
    for k in range(1, order + 1):
        for idx in combinations_with_replacement(range(X.shape[1]), k):
            cols.append(np.prod(X[:, idx], axis=1))
            labels.append('.'.join(names[i] for i in idx))
    return np.column_stack(cols), labels

def stlsq(Theta, dX, lam=0.025, iters=10):
    """Sequentially thresholded least squares (Brunton, Proctor, Kutz 2016)."""
    Xi = np.linalg.lstsq(Theta, dX, rcond=None)[0]
```

```

for _ in range(100):
    small = abs(Xi) < lam
    ...
    Xi[small] = 0
    for j in range(dX.shape[1]):
        survivors
        big = ~small[:, j]
        Xi[big, j] = np.linalg.lstsq(Theta[:, big], dX[:, j], rcond=None)[0]
    return Xi

Theta, labels = library(X)
Xi = stlsq(Theta, dX)
for j, v in enumerate('xyz'):
    terms = [f'{Xi[i, j]:+.3f} {labels[i]}' for i in np.flatnonzero(Xi[:, j])]
    print(f'{v}\n' =, ' '.join(terms))
print('least squares would keep', np.count_nonzero(abs(np.linalg.lstsq(Theta, dX,
    rcond=None)[0]) > 1e-6), 'of', Xi.size, 'terms')

```

The script, with derivatives computed from the data by finite differences and a library of all 56 monomials up to order 5, prints

$$\dot{x} = -10.000x + 10.000y, \quad \dot{y} = 27.998x - 1.000y - 1.000xz, \quad \dot{z} = -2.667z + 1.000xy,$$

the Lorenz system with its parameters; least squares would have kept 133 of the 168 coefficients. As in the video: same structure, virtually the same σ, ρ, β , same dynamics on the attractor [► V5.1 @ 6:16].

INTUITION: Occam's razor as an optimization

The threshold implements a Pareto trade-off between accuracy and complexity: as λ grows, terms drop out; the right model sits at the “knee”, where removing one more term makes the error jump. Choosing λ is model selection (cross-validation, information criteria such as AIC; book Sec. 4.6–4.7).

Noise and derivatives. One rarely measures $\dot{x}$; it must be computed from x [► V5.1 @ 6:58]. With very noisy measurements of x, y, z , *total-variation regularized* derivatives (TVDiff, R. Chartrand) still allow correct structure identification and an accurate reconstruction of the attractor [► V5.1 @ 7:18][► V5.1 @ 7:39]; both steps are numerically robust [► V5.1 @ 7:59].

◊ BEYOND THE CORE: Total-variation regularized differentiation

Find $u \approx \dot{f}$ as $\min_u \alpha \int |u'| dt + \frac{1}{2} \int \left| \int_0^t u - f \right|^2 dt$: the integral of the derivative must reproduce the signal, while the total variation of the derivative is penalized. Unlike finite differences, which amplify noise by $1/\Delta t$, this gives piecewise-smooth derivatives. Alternatives: Savitzky–Golay filters, local polynomial fits (as in PDE-FIND), or the weak (integral) form of SINDy, which avoids derivatives altogether.

Exercise 5.1 (Noise and threshold). Add Gaussian noise of standard deviation η to X in `ch05_sindy.py` before differentiating. For $\eta = 0.01, 0.1, 1$, find the range of λ for which the correct structure is found. Replace the finite differences by a Savitzky–Golay filter (`scipy.signal.savgol_filter` with `deriv=1`) and compare.

5.2 The cylinder wake: rediscovering the mean-field model

CASE STUDY: Thirty years in a regression

A benchmark of fluid dynamics: flow past a cylinder, with vortex shedding [► V5.1 @ 8:20]. A low-order model for it took decades [► V5.1 @ 8:40]:

1. Ruelle and Takens (1971), after Landau and Hopf, proposed a route to turbulence through successive Hopf bifurcations [► V5.1 @ 9:00][► V5.1 @ 9:21];
2. about 15 years later, Zebib, Jackson and others verified numerically that the cylinder wake undergoes a Hopf bifurcation at Reynolds number ≈ 47 , after which laminar periodic shedding sets in [► V5.1 @ 9:42][► V5.1 @ 10:02];
3. a paradox: the Hopf normal form (2.3) has *cubic* terms, but Navier–Stokes has only *quadratic* nonlinearity (convection) [► V5.1 @ 10:23][► V5.1 @ 10:43];
4. 15 years later again, Noack et al. (J. Fluid Mech. 2003) resolved it with a separation of time scales: the *mean-field model*

$$\dot{x} = \mu x - \omega y + Axz, \quad \dot{y} = \omega x + \mu y + Ayz, \quad \dot{z} = -\lambda(z - x^2 - y^2), \quad (5.2)$$

where x, y are the amplitudes of the two shedding modes and z the “shift mode” between the unstable steady flow and the mean flow. z is fast: it collapses onto the slow parabolic manifold $z = x^2 + y^2$, on which the dynamics winds out to periodic shedding [► V5.1 @ 11:04][► V5.1 @ 11:24][► V5.1 @ 11:46]. Only quadratic terms, yet a Hopf bifurcation [► V5.1 @ 12:06].

SINDy, fed the time series of the first two POD modes and the shift mode from a simulation, identifies a model with the same structure: only quadratic terms, the slow parabolic manifold, the spiral onto shedding [► V5.1 @ 12:27][► V5.1 @ 12:48][► V5.1 @ 13:10].

Proposition 5.3 (Cubic from quadratic). *On the slow manifold $z = x^2 + y^2$, model (5.2) reduces to the Hopf normal form $\dot{r} = \mu r + Ar^3$ (supercritical for $A < 0$), $\dot{\theta} = \omega$.*

Proof. With $r^2 = x^2 + y^2$: $r\dot{r} = x\dot{x} + y\dot{y} = \mu r^2 + Ar^2z$, so $\dot{r} = \mu r + Arz$; substituting $z = r^2$ gives $\dot{r} = \mu r + Ar^3$. Also $r^2\dot{\theta} = x\dot{y} - y\dot{x} = \omega r^2$. The cubic term is generated by eliminating the fast variable: adiabatic elimination. $\square$

Kick it off the attractor. If the data start on the slow manifold, SINDy wrongly identifies *cubic* terms (the reduced model of the proposition) [► V5.1 @ 13:51]. To get the quadratic model one must start off the attractor, e.g. from the mean flow, and watch the system fall back [► V5.1 @ 14:12]. General strategy: to identify the dynamics near an attractor, kick the system off it and see how it returns [► V5.1 @ 14:32].

Exercise 5.2 (Mean-field model). Simulate (5.2) with $\mu = 0.1$, $\omega = 1$, $A = -0.1$, $\lambda = 10$ from an initial condition on the slow manifold, and from one off it (z far from $x^2 + y^2$). Run SINDy with polynomials up to order 3 on both data sets: which one gives the quadratic model, which one the cubic normal form?

5.3 Extensions: parameters, forcing, delays, control

Parametrized dynamics “for free”. Treat a parameter μ as an extra state with $\dot{\mu} = 0$ and include it in the library $(\mu, \mu x, \mu x^2, \dots)$ [► V5.1 @ 14:53][► V5.1 @ 15:35]. For the logistic map, training on 10 values of μ yields a parametrized model that reproduces the whole bifurcation diagram [► V5.1 @ 15:14][► V5.1 @ 15:56]; likewise for the Hopf normal form [► V5.1 @ 15:56]. In

the same way one can add time ($\sin t, t^2, \dots$) for non-autonomous systems, or external forcing such as CO_2 in a climate model [► V5.1 @ 16:16][► V5.1 @ 16:36].

Delay coordinates. What if only x of the Lorenz system is measured [► V5.1 @ 16:58] [► V5.1 @ 17:18]? Build the *Hankel matrix* of shifted copies of the time series [► V5.1 @ 17:39] [► V5.1 @ 18:00],

$$H = \begin{bmatrix} x_1 & x_2 & \cdots & x_{m-p+1} \\ x_2 & x_3 & \cdots & x_{m-p+2} \\ \vdots & & & \vdots \\ x_p & x_{p+1} & \cdots & x_m \end{bmatrix} = U\Sigma V^*. \quad (5.3)$$

The SVD gives “eigen time-delay coordinates”: the first columns of V reconstruct an attractor topologically equivalent to Lorenz’s, as Takens’ embedding theorem guarantees [► V5.1 @ 18:21][► V5.1 @ 18:42]. Running SINDy on the first three columns of V gives a sparse model that reproduces the attractor [► V5.1 @ 19:02][► V5.1 @ 19:25]. The same SVD of a Hankel matrix underlies singular spectrum analysis (SSA) in climate science and the eigensystem realization algorithm (ERA) in system identification; Broomhead and King (1986) already plotted Lorenz in these coordinates. What is new is a *nonlinear model* on them [► V5.1 @ 19:45] [► V5.1 @ 20:06]. Chapter 8 (HAVOK) develops this.

Theorem 5.4 (Takens, 1981, informal). *Let the dynamics evolve on a compact attractor of box dimension d and let h be a generic scalar measurement. Then for $p > 2d$ the delay map $\mathbf{x} \mapsto (h(\mathbf{x}), h(\mathbf{F}_\tau \mathbf{x}), \dots, h(\mathbf{F}_{(p-1)\tau} \mathbf{x}))$ is an embedding: a one-to-one, smooth image of the attractor.*

SINDy with control. For $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + \mathbf{g}(\mathbf{u})$, where $\mathbf{u}$ may be an exogenous forcing or a feedback law and may enter nonlinearly ($u^3, \sin u$), include $\mathbf{u}$ in the library [► V5.1 @ 20:27] [► V5.1 @ 20:47][► V5.1 @ 21:08][► V5.1 @ 21:29]. In the example, Lorenz is trained for $t \in [0, 20]$ under a feedback $u = 26 - x$ plus noise; at $t = 20$ the control law is switched. Plain SINDy fails, because it never knew there was forcing; SINDy with control tracks the new data, because it disentangled the effect of the control from the dynamics [► V5.1 @ 22:10] [► V5.1 @ 22:31][► V5.1 @ 22:52].

The recipe and its open questions [► V5.1 @ 23:12][► V5.1 @ 23:33]:

1. measure (all the state if possible, otherwise build delay coordinates);
 2. choose a basis (library) in which the dynamics is sparse;
 3. sparse regression for the fewest terms that explain the data.
- *Did I measure the right variables?* Delay embeddings give tools to check whether enough variables are measured and whether they carry new information [► V5.1 @ 24:35] [► V5.1 @ 24:55].
 - *Is the library right?* Harder. Physics guides it (Navier–Stokes means quadratic terms); in general, try polynomials, trigonometric functions, rational functions [► V5.1 @ 24:55] [► V5.1 @ 25:17][► V5.1 @ 25:37].
 - The regression is the easy part [► V5.1 @ 25:58][► V5.1 @ 26:18].

Exercise 5.3 (Parametrized logistic map). Generate logistic-map data for $\mu \in \{2.5, 2.75, \dots, 3.95\}$ (10 values), build a library of monomials in (x, μ) up to order

4, and fit x_{k+1} (a map: no derivative needed). Use the model to draw the bifurcation diagram on $[2.5, 4]$ and compare with [fig. 3.2](#).

Exercise 5.4 (Delay embedding). Using only $x(t)$ from `ch03_lorenz.py`, build H with $p = 100$ and plot v_1, v_2, v_3 (columns of V). How many singular values are significant? Compare the picture with the Lorenz attractor.

5.4 PDE-FIND: discovering partial differential equations

The same idea for spatio-temporal data, presented by Samuel Rudy [\[V5.2 @ 0:03\]](#). Assume the data solve an unknown PDE with one time derivative,

$$u_t = N(u, u_x, u_{xx}, \dots, Q), \quad (5.4)$$

where Q is extra information (a potential, a velocity field), and that N is sparse in a guessed basis, e.g. polynomials in u and its derivatives [\[V5.2 @ 0:24\]](#)[\[V5.2 @ 0:45\]](#)[\[V5.2 @ 1:06\]](#).

Steps [\[V5.2 @ 1:48\]](#)–[\[V5.2 @ 3:35\]](#):

1. compute u_t and the spatial derivatives from the discretized data: finite differences if the data are clean; with noise, a smoothing method (local polynomial interpolation works, but choosing the number of points and the degree is tricky) [\[V5.2 @ 2:08\]](#);
2. stack every space-time point as one row: $U_t \approx \Theta(U, Q) \xi$, with columns such as $1, u, u_x, u^2, uu_x, \dots$ (nothing forbids $\cos u$ if physics suggests it) [\[V5.2 @ 2:30\]](#)[\[V5.2 @ 2:50\]](#)[\[V5.2 @ 3:10\]](#);
3. solve for a sparse ξ .

Least squares always gives dense solutions (a 100-term PDE whose terms are mostly noise), and the columns of Θ are correlated, so the matrix is ill-conditioned [\[V5.2 @ 3:35\]](#)[\[V5.2 @ 3:56\]](#)[\[V5.2 @ 4:17\]](#). Among LASSO, elastic net, greedy methods and thresholding, Rudy chose *sequential threshold ridge regression* (STRidge): ridge estimate, cut coefficients below a threshold, repeat on the survivors [\[V5.2 @ 4:17\]](#)[\[V5.2 @ 4:37\]](#). For Navier–Stokes the sparse fit keeps the 4 correct terms, least squares keeps all 60 [\[V5.2 @ 4:58\]](#)[\[V5.2 @ 5:19\]](#); the coefficients are within 1% [\[V5.2 @ 6:24\]](#).

HISTORY: Ruelle and Takens

David Ruelle and Floris Takens, *On the nature of turbulence*, Commun. Math. Phys. 20 (1971) 167–192, coined the term *strange attractor* and argued against Landau’s (1944) picture of turbulence as an infinite cascade of Hopf bifurcations: after only a few, a strange attractor generically appears. The paper was rejected by another journal first and appeared in a journal of which Ruelle was an editor. Takens’ embedding theorem appeared in *Detecting strange attractors in turbulence* (Lecture Notes in Math. 898, 1981), the same year as Packard, Crutchfield, Farmer and Shaw’s delay reconstruction (PRL 1980). *References*: D. Ruelle, *Chance and Chaos* (Princeton, 1991); the papers cited.

PYTHON: PDE-FIND on Burgers’ equation (`./code/ch05_pdefind.py`)

```
import numpy as np
from scipy.integrate import solve_ivp

# data: Burgers' equation u_t = -u u_x + nu u_xx, solved spectrally on a periodic
# domain
nu, L, n = 0.1, 16.0, 256
x = np.linspace(-L / 2, L / 2, n, endpoint=False)
k = 2 * np.pi * np.fft.fftfreq(n, d=L / n)
def burgers(t, u):
    uh = np.fft.fft(u)
    return np.real(-u * np.fft.ifft(1j * k * uh) - nu * np.fft.ifft(k**2 * uh))
t = np.linspace(0, 10, 201); dt = t[1] - t[0]
U = solve_ivp(burgers, (0, 10), np.exp(-(x + 2)**2), t_eval=t, rtol=1e-10, atol=1e-10).
y # n x m
```

```

# derivatives from the data with finite differences (clean data)
dx = x[1] - x[0]
Ut = np.gradient(U, dt, axis=1)
Ux = np.gradient(U, dx, axis=0)
Uxx = np.gradient(Ux, dx, axis=0)
Uxxx = np.gradient(Uxx, dx, axis=0)

# library: {1, u, u^2} x {1, u_x, u_xx, u_xxx}, every space-time point is one row
pw = {'': np.ones_like(U), 'u': U, 'u^2': U**2}
dv = {'': np.ones_like(U), 'u_x': Ux, 'u_xx': Uxx, 'u_xxx': Uxxx}
names = [(a + ' ' + b).strip() or '1' for a in pw for b in dv]
Theta = np.column_stack([(pw[a] * dv[b]).ravel() for a in pw for b in dv])
ut = Ut.ravel()

def stridge(Theta, y, lam=1e-5, tol=0.05, iters=10):
    """Sequential threshold ridge regression (Rudy et al. 2017), columns normalised."""
    norms = np.linalg.norm(Theta, axis=0); T = Theta / norms
    big = np.ones(T.shape[1], bool)
    for _ in range(iters):
        A = T[:, big]
        w = np.zeros(T.shape[1])
        w[big] = np.linalg.solve(A.T @ A + lam * np.eye(big.sum()), A.T @ y)
        big = abs(w / norms) > tol # threshold on the real
    coefficients
    w = np.zeros(T.shape[1]); w[big] = np.linalg.lstsq(T[:, big], y, rcond=None)[0]
    return w / norms

xi = stridge(Theta, ut)
print('u_t =', ' '.join(f'{c:+.4f} {s}' for c, s in zip(xi, names) if c != 0))
rows = np.random.default_rng(0).choice(len(ut), len(ut) // 100, replace=False) # 1%
of the data
xi1 = stridge(Theta[rows], ut[rows])
print('with 1% of the rows:', ' '.join(f'{c:+.4f} {s}' for c, s in zip(xi1, names) if
c != 0))

```

The script generates Burgers' equation $u_t = -uu_x + 0.1u_{xx}$ spectrally, computes all derivatives with finite differences, and from a library of 12 terms finds $u_t = 0.1016 u_{xx} - 1.0008 uu_x$; with a random 1% of the rows, $u_t = 0.1016 u_{xx} - 1.0030 uu_x$.

Sub-sampling. The number of rows is the number of data points, about 13 million for a 2D Navier–Stokes data set in time; using about 1% of them (dropping rows) gives the same result [► V5.2 @ 6:45][► V5.2 @ 7:06].

Which PDE? The soliton ambiguity. A function may solve many PDEs [► V5.2 @ 7:30]. A single KdV soliton, $u = \frac{\epsilon}{2} \operatorname{sech}^2\left(\frac{\sqrt{\epsilon}}{2}(x - ct)\right)$, also solves the advection equation $u_t = -cu_x$, and PDE-FIND returns the simpler one, advection [► V5.2 @ 7:50]. Two solitons of different speeds in one solution, or two separate solutions with solitons of different amplitudes, force a nonlinear PDE, and PDE-FIND finds KdV [► V5.2 @ 8:10][► V5.2 @ 8:31][► V5.2 @ 8:52]. The lesson: the data must be rich enough to exclude the alternatives, as in “kick it off the attractor”.

Gallery. KdV, Burgers, nonlinear Schrödinger, linear Schrödinger in a harmonic potential, Kuramoto–Sivashinsky, reaction–diffusion, vorticity equation of Navier–Stokes: all recovered [► V5.2 @ 9:13][► V5.2 @ 9:33]. With noise, Kuramoto–Sivashinsky keeps the right

sparsity pattern but its coefficients are badly underestimated (52% error) [► V5.2 @ 9:33]. Noise complicates everything through the derivatives; next steps are real data and latent variables [► V5.2 @ 9:55][► V5.2 @ 10:15].

Exercise 5.5 (Soliton ambiguity). Generate one KdV soliton and run `ch05_pdefind.py` with a library containing u_{xxx} and uu_x . Explain why the regression cannot distinguish $u_t = -cu_x$ from $u_t = -6uu_x - u_{xxx}$, and verify that both are satisfied (choose the soliton normalization accordingly).

KEY FORMULAS

- SINDy: $\dot{X} \approx \Theta(X)\Xi$, Ξ sparse (STLSQ or LASSO), one regression per variable.
- Library extensions: parameters ($\dot{\mu} = 0$), time, forcing $\mathbf{u}$ (SINDYc), delay coordinates from the SVD of the Hankel matrix.
- Mean-field model: quadratic terms + fast variable $\Rightarrow$ cubic Hopf on $z = x^2 + y^2$.
- PDE-FIND: $U_t \approx \Theta(U, U_x, U_{xx}, \dots)\xi$, STRidge; derivatives are the hard part.

Part III

The Koopman operator

Chapter 6

Koopman Spectral Analysis

SOURCES FOR THIS CHAPTER

Video	Length	Content
V6.1 Koopman Spectral Analysis (Overview)	27:49	Open problems; Koopman 1931 and the ergodic theorem; the operator on observables; eigenfunctions and invariant subspaces; the slow-manifold example; $\dot{x} = x^2$ and $e^{-1/x}$; the eigenfunction PDE; DMD as an approximation.
V6.2 Koopman Spectral Analysis (Representations)	16:01	Extended DMD; overfitting and VAC; sparse identification of eigenfunctions; deep Koopman autoencoders; error versus complexity; cross-validation.
V6.3 Koopman Spectral Analysis (Control)	15:26	LQR in Koopman coordinates; control of eigenfunctions (energy); state-dependent Riccati; well hopping and space missions; double gyre; other Koopman-control methods.
V6.4 Koopman Spectral Analysis (Continuous Spectrum)	12:43	Chaos and HAVOK; the pendulum and its continuous spectrum; autoencoders with an auxiliary network; action-angle coordinates.
V6.5 Koopman Spectral Analysis (Multiscale systems)	5:08	Delay coordinates for multiscale systems (coupled van der Pol); multiresolution DMD; summary.

Transcripts materials/transcripts/C06-V0*.txt. **Book** Sec. 7.4 (Koopman operator theory), Sec. 7.5 (data-driven Koopman analysis). **Code** code/ch06_koopman.py.

Motivation

The first primary challenge of [chapter 1](#) was nonlinearity. Koopman theory attacks it with a change of point of view: instead of the state, follow the *measurements* of the state. On the space of all possible measurements the dynamics acts *linearly*, through an infinite-dimensional operator [[► V6.1 @ 5:29](#)]. The price is the dimension; the game of modern data-driven Koopman theory is to find a few special measurements, the eigenfunctions, in which a nonlinear system looks linear, and then use textbook linear tools for prediction, estimation and control. A physicist has met this before: the Heisenberg picture, where observables evolve and states stay put, and action-angle variables, where an integrable system becomes uniform rotation.

The open problems that frame the series [[► V6.1 @ 2:04](#)]: unknown equations (no equation for the brain, the climate or an epidemic) [[► V6.1 @ 2:25](#)]; nonlinearity, where even a quadratic term confounds understanding [[► V6.1 @ 3:05](#)]; control (Brunton is a control theorist) [[► V6.1 @ 3:26](#)]; chaos, transients and intermittency [[► V6.1 @ 3:46](#)]; multiscale physics

[► V6.1 @ 4:07]; and, not treated here, noise and uncertainty [► V6.1 @ 4:48].

6.1 The Koopman operator

Koopman considered the Hilbert space of all possible measurements of the system: not only x , but x^2 , $\cos x$, $\arctan x$, $\dots$. In that infinite-dimensional space the nonlinear dynamics is a linear operator that advances measurements in time [► V6.1 @ 6:32][► V6.1 @ 6:52][► V6.1 @ 7:13]. “Measurement” is a fancy word for data, which is why the 1931 idea fits the data-science era [► V6.1 @ 7:33][► V6.1 @ 7:54].

Definition 6.1 (New concepts: observable, Koopman operator). An **observable** is a function $g : M \rightarrow \mathbb{C}$ of the state (in a suitable function space, e.g. L^2). For the discrete-time system $\mathbf{x}_{k+1} = \mathbf{F}(\mathbf{x}_k)$ (or the flow map $\mathbf{F}_t$) the **Koopman operator** acts on observables by composition:

$$\mathcal{K}g = g \circ \mathbf{F}, \quad \text{i.e.} \quad (\mathcal{K}g)(\mathbf{x}_k) = g(\mathbf{x}_{k+1}); \quad (6.1)$$

in continuous time $\mathcal{K}^t g = g \circ \mathbf{F}_t$, with generator $\mathcal{L}g = \lim_{t \rightarrow 0} (\mathcal{K}^t g - g)/t = \nabla g \cdot \mathbf{f}$. $\mathcal{K}$ is linear, $\mathcal{K}(a g_1 + b g_2) = a \mathcal{K}g_1 + b \mathcal{K}g_2$, however nonlinear $\mathbf{F}$ is [► V6.1 @ 10:59][► V6.1 @ 11:19]. It is named after B. O. Koopman; its adjoint, acting on densities, is the Perron–Frobenius (transfer) operator.

Geometric view: trajectories in state space, fixed points, manifolds, attractors. Operator view: how measurements evolve [► V6.1 @ 10:18][► V6.1 @ 10:39]. The trade: a finite-dimensional nonlinear problem for an infinite-dimensional linear one [► V6.1 @ 7:13]. Finding finite-dimensional approximations (matrices) of $\mathcal{K}$ has been the subject of intense research for a decade [► V6.1 @ 11:40][► V6.1 @ 12:00].

Definition 6.2 (New concepts: Koopman eigenfunction, invariant subspace). A **Koopman eigenfunction** is an observable φ with $\mathcal{K}\varphi = \mu\varphi$, i.e. $\varphi(\mathbf{x}_{k+1}) = \mu\varphi(\mathbf{x}_k)$: measured in φ , the system evolves exactly linearly, as a pure exponential or oscillation [► V6.1 @ 12:20]. In continuous time $\frac{d}{dt}\varphi(\mathbf{x}) = \lambda\varphi(\mathbf{x})$, $\mu = e^{\lambda\Delta t}$. A subspace of observables spanned by $\{g_1, \dots, g_p\}$ is **Koopman invariant** if $\mathcal{K}g_j$ stays in the span for every j ; eigenfunctions span such subspaces, and restricted to one, $\mathcal{K}$ is a $p \times p$ matrix [► V6.1 @ 12:42][► V6.1 @ 13:02].

Proposition 6.3 (Eigenfunction PDE). A smooth eigenfunction of the flow of $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x})$ satisfies the linear first-order PDE

$$\nabla\varphi(\mathbf{x}) \cdot \mathbf{f}(\mathbf{x}) = \lambda\varphi(\mathbf{x}). \quad (6.2)$$

Products of eigenfunctions are eigenfunctions: $\varphi_1\varphi_2$ has eigenvalue $\lambda_1 + \lambda_2$.

Exercise 6.1 (Koopman operator of a map). For the linear map $x_{k+1} = ax_k$ find all polynomial eigenfunctions of $\mathcal{K}$ and their eigenvalues. For the doubling map $x_{k+1} = 2x_k \bmod 1$ on $[0, 1)$, show that $e^{2\pi i 2^n x}$ is mapped by $\mathcal{K}$ to $e^{2\pi i 2^{n+1} x}$: $\mathcal{K}$ is a shift on Fourier modes, with no eigenfunction other than constants in L^2 .

Exercise 6.2 (Laurent series). Insert $\varphi(x) = \sum_{k=-\infty}^{\infty} c_k x^k$ into (6.2) for $f(x) = x^2$, derive the recursion for the coefficients, and show that the solution with $\lambda = 1$ is $e^{-1/x}$. What

HISTORY: Koopman, von Neumann, Birkhoff

Bernard O. Koopman, *Hamiltonian systems and transformation in Hilbert space*, PNAS 17 (1931) 315–318, observed that a Hamiltonian flow induces a *unitary* operator on L^2 functions of phase space (Liouville’s theorem preserves the measure). Within months John von Neumann used it to prove the mean ergodic theorem, and George D. Birkhoff the pointwise one (PNAS, 1931–32), a priority race that left bruises; Koopman and von Neumann also wrote a joint paper (1932) on systems with continuous spectrum. The operator view then slept for decades, while Poincaré’s geometric view dominated, until Igor Mezić and co-workers revived it (1999–2005) [► V6.1 @ 5:50][► V6.1 @ 8:14][► V6.1 @ 8:56].
References: C. C. Moore, *Ergodic theorem, ergodic theory, and statistical mechanics*, PNAS 112 (2015) 1907–1911 (the perspective recommended in the video); I. Mezić, *Nonlinear Dyn.* 41 (2005).

Proof. Chain rule: $\frac{d}{dt}\varphi(\mathbf{x}(t)) = \nabla\varphi \cdot \dot{\mathbf{x}} = \nabla\varphi \cdot \mathbf{f}$, which must equal $\lambda\varphi$ along every trajectory, hence everywhere [► V6.1 @ 18:55][► V6.1 @ 19:16][► V6.1 @ 19:37]. For the product, $\nabla(\varphi_1\varphi_2) \cdot \mathbf{f} = \varphi_2\lambda_1\varphi_1 + \varphi_1\lambda_2\varphi_2$. $\square$

Example 6.4 (Linear systems). For $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x}$ with left eigenvectors $\mathbf{w}_j^* \mathbf{A} = \lambda_j \mathbf{w}_j^*$, the linear observables $\varphi_j(\mathbf{x}) = \mathbf{w}_j^* \mathbf{x}$ are eigenfunctions: $\dot{\varphi}_j = \mathbf{w}_j^* \mathbf{A}\mathbf{x} = \lambda_j \varphi_j$. These are the coordinates

are the eigenfunctions for other λ ?

$\mathbf{z} = T^{-1}\mathbf{x}$ of [proposition 1.6](#). Koopman eigenfunctions generalize eigenvector coordinates to nonlinear systems.

6.2 Two examples: closure and non-closure

CASE STUDY: A slow manifold made linear

The system of [example 2.5](#) (the book's Sec. 7.4 example) [[V6.1 @ 13:02](#)][[V6.1 @ 13:23](#)]:

$$\dot{x}_1 = \mu x_1, \quad \dot{x}_2 = \lambda(x_2 - x_1^2). \quad (6.3)$$

If λ is much more negative than μ , x_2 quickly relaxes to x_1^2 : trajectories stick to the slow manifold $x_2 = x_1^2$ and slide to the origin [[V6.1 @ 13:23](#)][[V6.1 @ 13:43](#)]. Add the nonlinear measurement $y_3 = x_1^2$ to $y_1 = x_1$, $y_2 = x_2$ [[V6.1 @ 13:43](#)][[V6.1 @ 14:04](#)]:

$$\dot{y}_1 = \mu y_1, \quad \dot{y}_2 = \lambda y_2 - \lambda y_3, \quad \dot{y}_3 = 2x_1 \dot{x}_1 = 2\mu y_3, \quad \frac{d}{dt}\mathbf{y} = \begin{bmatrix} \mu & 0 & 0 \\ 0 & \lambda & -\lambda \\ 0 & 0 & 2\mu \end{bmatrix} \mathbf{y}.$$

An honest 3×3 linear system: it predicts the nonlinear dynamics as far ahead as wanted with a matrix exponential, and projected on (y_1, y_2) reproduces the slow-manifold behaviour [[V6.1 @ 14:25](#)][[V6.1 @ 14:45](#)][[V6.1 @ 15:06](#)][[V6.1 @ 15:27](#)]. Eigenfunctions: $\varphi_\mu = x_1$, $\varphi_{2\mu} = x_1^2$, and $\varphi_\lambda = x_2 - bx_1^2$ with $b = \lambda/(\lambda - 2\mu)$; the level set $\varphi_\lambda = 0$ is the unstable/slow manifold found in [example 2.5](#).

When it does not close: $\dot{x} = x^2$. Even simpler, and nasty: finite-time blow-up (“you can’t say this in an airport”) [[V6.1 @ 15:49](#)][[V6.1 @ 16:09](#)]. With $y_1 = x$, $y_2 = x^2$: $\dot{y}_1 = y_2$, but $\dot{y}_2 = 2x\dot{x} = 2x^3$, outside the span; adding x^3 produces x^4 , and so on forever [[V6.1 @ 16:30](#)][[V6.1 @ 16:50](#)]. Truncations converge slowly to the true solution [[V6.1 @ 17:12](#)]. Lesson: an arbitrary Taylor or Fourier basis is usually a bad basis; the right representation is in terms of eigenfunctions [[V6.1 @ 17:32](#)]. And there is one (Clancy Rowley wrote it down while waiting for a bus) [[V6.1 @ 17:53](#)]:

$$\varphi(x) = e^{-1/x}, \quad \dot{\varphi} = \frac{\dot{x}}{x^2} e^{-1/x} = \varphi,$$

an eigenfunction with $\lambda = 1$, not representable by a finite Taylor series (it is the textbook non-analytic smooth function) [[V6.1 @ 18:14](#)]. Solving (6.2) with a Laurent series ansatz, powers x^k and x^{-k} , produces exactly this function [[V6.1 @ 19:59](#)][[V6.1 @ 20:20](#)][[V6.1 @ 20:41](#)].

INTUITION: Eigenfunctions are clocks

$\varphi = e^{-1/x}$ is e^t up to a constant: it measures elapsed time. Indeed $-1/x$ grows by exactly 1 per unit time ($\frac{d}{dt}(-1/x) = 1$), the “uniform motion” coordinate of [chapter 1](#). Every eigenfunction with $\text{Re } \lambda \neq 0$ is $e^{\lambda \tau(x)}$ for some “time function” τ with $\dot{\tau} = 1$; with $\lambda = i\omega$ it is a phase. Finding eigenfunctions means finding coordinates in which time flows uniformly.

PYTHON: Koopman examples (`../code/ch06_koopman.py`)

```
import numpy as np
from itertools import combinations_with_replacement
```

```

from scipy.integrate import solve_ivp
from scipy.linalg import expm
from scipy.special import ellipk
import matplotlib.pyplot as plt

mu, lam = -0.05, -1.0                                # slow (mu) and fast (lam) eigenvalues

def f(t, x):
    return [mu * x[0], lam * (x[1] - x[0]**2)]        # x1' = mu x1, x2' = lam (x2 - x1^2)

# (a) the lifted system y = (x1, x2, x1^2) is exactly linear
K = np.array([[mu, 0, 0], [0, lam, -lam], [0, 0, 2 * mu]])
x0 = np.array([1.5, -1.0]); t = np.linspace(0, 20, 201)
x = solve_ivp(f, (0, 20), x0, t_eval=t, rtol=1e-12, atol=1e-12).y
y = np.array([expm(K * tk) @ [x0[0], x0[1], x0[0]**2] for tk in t]).T
print('(a) max |x - y[:2]| =', np.abs(x - y[:2]).max())

# (b) extended DMD: regress monomials of x at step k+1 on those at step k
def monomials(X, order=3):
    return np.array([np.prod(X[list(c)], axis=0) for k in range(order + 1)
                     for c in combinations_with_replacement(range(2), k)])

rng = np.random.default_rng(0)
dt = 0.1
X0 = rng.uniform(-2, 2, (2, 500))                    # many short bursts
X1 = np.array([solve_ivp(f, (0, dt), p, rtol=1e-12, atol=1e-12).y[:, -1] for p in X0.T
               ]).T
Psi0, Psi1 = monomials(X0), monomials(X1)             # 10 x 500
Kd = Psi1 @ np.linalg.pinv(Psi0)                     # EDMD matrix
ev = np.sort(np.log(np.linalg.eigvals(Kd)).real / dt)
print('(b) EDMD continuous-time eigenvalues:', ev.round(4)) # sums of mu, lam

# (c) continuous spectrum: the pendulum frequency depends on the energy
E = np.linspace(-0.999, 0.999, 300)                  # E = w^2/2 - cos(theta), g/L = 1
k2 = (1 + E) / 2                                     # modulus^2 of the elliptic integral
omega = np.pi / (2 * ellipk(k2))                     # frequency of the libration
plt.figure(figsize=(4, 2.6)); plt.plot(E, omega, color='#C2185B')
plt.xlabel('energy $E$'); plt.ylabel(r'frequency $\omega(E)$')
plt.tight_layout(); plt.savefig('lecture-notes/figures/ch06-pendulum-omega.png', dpi=200)
print('(c) omega from', omega[0].round(3), 'to', omega[-1].round(3))

```

6.3 Data-driven approximations

DMD. The workhorse: the best-fit linear operator A between snapshot matrices is a finite-dimensional approximation of $\mathcal{K}$ restricted to *linear* observables of the state; the connection was made by Rowley, Mezić et al. (2009) on a jet in cross flow [► V6.1 @ 21:23][► V6.1 @ 21:43][► V6.1 @ 22:04][► V6.1 @ 25:32]. It works remarkably well for periodic or quasi-periodic data, less well for transients and intermittency [► V6.1 @ 23:27][► V6.1 @ 23:48]. Further connections in fluids: Bagheri’s Koopman analysis of the cylinder wake, and the link with resolvent analysis (Sharma, Mezić, McKeon) [► V6.1 @ 25:52][► V6.1 @ 26:13][► V6.1 @ 26:33].

Extended DMD. Williams, Kevrekidis and Rowley (2015): a linear model cannot have multiple fixed points or genuinely nonlinear transients [► V6.2 @ 2:26], so augment the state with nonlinear observables (monomials, $\cos x$, radial basis functions, Bessel functions) and fit the best linear operator on the augmented state [► V6.2 @ 2:47][► V6.2 @ 3:07]. On the Duffing oscillator it finds eigenfunctions where DMD fails [► V6.2 @ 3:28][► V6.2 @ 3:48].

Definition 6.5 (New concepts: extended DMD). With a dictionary $\psi(\mathbf{x}) = (\psi_1(\mathbf{x}), \dots, \psi_p(\mathbf{x}))^\top$ and snapshot pairs $(\mathbf{x}_k, \mathbf{x}'_k)$, form $\Psi = [\psi(\mathbf{x}_1) \cdots]$, $\Psi' = [\psi(\mathbf{x}'_1) \cdots]$ and $K = \Psi' \Psi^\dagger$, the least-squares approximation of $\mathcal{K}$ on the span of the dictionary. Eigenvectors $\mathbf{v}$ of $K^\top$ give approximate eigenfunctions $\varphi(\mathbf{x}) = \mathbf{v}^\top \psi(\mathbf{x})$.

In the script, EDMD with the 10 monomials of degree ≤ 3 on (6.3) ($\mu = -0.05$, $\lambda = -1$) returns the exact eigenvalues $0, \mu, 2\mu, 3\mu, \lambda, \lambda + \mu$ ($0, -0.05, -0.1, -0.15, -1, -1.05$) and four *spurious* ones (-1.48 , a complex pair, $-1.89, -3.05$): eigenfunctions such as φ_λ^2 need x_1^4 , which is not in the dictionary, and the regression returns its best but wrong guess.

Overfitting. A huge dictionary means a huge matrix with many free parameters: massive overfitting unless one cross-validates [► V6.2 @ 4:09][► V6.2 @ 4:29]. The variational approach to conformation dynamics (VAC) of Frank Noé and co-workers (2013) builds cross-validation in, through a “VAC score” [► V6.2 @ 4:51][► V6.2 @ 15:09].

Sparse eigenfunctions. Eurika Kaiser’s alternative: skip the big matrix, whose eigenvectors are often spurious, and look for eigenfunctions directly, sparse in a library Θ , $\varphi(\mathbf{x}) = \Theta(\mathbf{x})\mathbf{c}$ [► V6.2 @ 5:11][► V6.2 @ 5:32][► V6.2 @ 5:53][► V6.2 @ 6:13]. Evaluating (6.2) on data gives $(\Gamma(\mathbf{x}, \dot{\mathbf{x}}) - \lambda\Theta(\mathbf{x}))\mathbf{c} = 0$, with $\Gamma = \nabla\Theta \cdot \dot{\mathbf{x}}$: a sparse vector in a null space [► V6.2 @ 6:54][► V6.2 @ 7:15]. One must know λ roughly, but lightly damped eigenfunctions ($\text{Re } \lambda \approx 0$) persist in the data and are the easiest to find, and they are also the ones that matter for prediction and control [► V6.2 @ 7:35][► V6.2 @ 7:56]. Conserved quantities are eigenfunctions with $\lambda = 0$: from Duffing data the method discovers the Hamiltonian [► V6.2 @ 8:16][► V6.2 @ 9:18]. In discrete time the same idea gives back VAC and EDMD, or sparse versions of them [► V6.2 @ 8:36].

Deep Koopman. Neural networks represent complicated functions well when data are plentiful [► V6.2 @ 11:02]. Many groups (Lusch, Kutz and Brunton among them) use an autoencoder: an encoder φ to a few latent variables, a decoder φ^{-1} back to the state, and a *linear* time-stepper K in the latent space [► V6.2 @ 10:00][► V6.2 @ 10:20][► V6.2 @ 10:42]. The field moves fast [► V6.2 @ 11:23].

INTUITION: Error versus complexity

Plot model error against complexity [► V6.2 @ 11:43][► V6.2 @ 12:04]. Exact Koopman theory is the “unicorn” on the far right: perfect linearization, infinite complexity. DMD is on the left: simple, good for periodic data, poor for transients and chaos [► V6.2 @ 12:25][► V6.2 @ 12:45]. The hope is a curved Pareto front with a sweet spot of low complexity and good error [► V6.2 @ 13:05][► V6.2 @ 13:25]. But on *training* data any EDMD or network can reach zero error; only *cross-validated* error on held-out trajectories reveals the front. “You will get garbage models if you don’t cross validate” [► V6.2 @ 13:46][► V6.2 @ 14:06][► V6.2 @ 14:27]. Sparse methods are parsimonious by construction [► V6.2 @ 14:48].

Exercise 6.3 (Eigenfunctions of the slow-manifold system). Verify that $x_2 - \frac{\lambda}{\lambda-2\mu}x_1^2$ is an eigenfunction of (6.3). Find the eigenvectors of K and relate them to the eigenfunctions. Then run the EDMD part of `ch06_koopman.py` with monomials up to degree 4 and 5: which spurious eigenvalues disappear?

6.4 Koopman and control

The promise: in Koopman coordinates, textbook linear control (LQR, Kalman filters, “one-line commands in Matlab”) works for strongly nonlinear systems; finding the coordinates is an up-front cost, everything downstream is easy [► V6.3 @ 0:42][► V6.3 @ 1:03]. For (6.3) with a control u on x_2 , the lifted system is $\dot{\mathbf{y}} = K\mathbf{y} + B\mathbf{u}$; LQR on it returns three gains, and the gain on $y_3 = x_1^2$ is a *nonlinear* feedback in the original variables. It is about three times cheaper (in the LQR cost) than LQR on the linearization at the origin, because it exploits the slow manifold instead of fighting it [► V6.3 @ 2:06][► V6.3 @ 2:26][► V6.3 @ 3:07][► V6.3 @ 3:48][► V6.3 @ 4:09][► V6.3 @ 4:29]. Chapter 7 works this example out. The pipeline: data $\rightarrow$ model (DMD, SINDy) $\rightarrow$ Koopman coordinates $\rightarrow$ linear optimal control [► V6.3 @ 4:49][► V6.3 @ 5:51].

Control in eigenfunction coordinates. With $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + B\mathbf{u}$ and eigenfunctions φ of the *unforced* system [► V6.3 @ 6:12][► V6.3 @ 6:32]:

$$\dot{\varphi} = \nabla\varphi \cdot (\mathbf{f} + B\mathbf{u}) = \lambda\varphi + \nabla\varphi(\mathbf{x}) \cdot B\mathbf{u}. \quad (6.4)$$

The dynamics stays linear in φ ; only the actuation becomes state dependent: the actuator is more or less effective depending on where the state is [► V6.3 @ 6:53][► V6.3 @ 7:13][► V6.3 @ 7:34]. One then steers the eigenfunctions themselves [► V6.3 @ 8:16]. For a conservative system the energy H is an eigenfunction ($\lambda = 0$): to swing up a pendulum, drive H to the energy of the upright position instead of driving the state to $(\pi, 0)$ [► V6.3 @ 8:37][► V6.3 @ 8:58]. The control is solved with a state-dependent Riccati equation (SDRE) [► V6.3 @ 9:18][► V6.3 @ 9:39].

APPLICATION: Well hopping and the Jupiter icy moons

Eurika Kaiser found, purely from data, the energy eigenfunctions of a quartic well, the Duffing oscillator and the pendulum, and designed controllers that move the energy up or down [► V6.3 @ 10:00][► V6.3 @ 10:21]; with multiple wells, two controllers in sequence push the system over the barrier and settle it in the next well [► V6.3 @ 10:42][► V6.3 @ 11:02]. This is the problem of energy-efficient space missions: each planet and moon is a potential well inside the Sun’s; touring the moons of Jupiter means hopping between wells (the *Jupiter Icy Moons Orbiter* studies, on which Brunton’s undergraduate advisor Jerry Marsden worked) [► V6.3 @ 11:22][► V6.3 @ 11:43][► V6.3 @ 12:04]. A higher-dimensional example: a double-gyre ocean model, where the method stabilizes coordinated swarms of drifters [► V6.3 @ 12:25][► V6.3 @ 12:46].

Other Koopman-control work: Koopman Kalman filters (A. Surana), model predictive control on EDMD models (M. Korda, I. Mezić), also for power-grid stabilization, and switching between Koopman models built for fixed control values (S. Peitz and co-workers) [► V6.3 @ 13:49][► V6.3 @ 14:11][► V6.3 @ 14:32].

Exercise 6.4 (Energy as an eigenfunction). Show that for any Hamiltonian system H is a Koopman eigenfunction with $\lambda = 0$, and so is any function of H . Write (6.4) for the pendulum with a torque u and derive the energy-pumping law $u = -k \dot{H} (H - H^*)$

from $\frac{d}{dt} \frac{1}{2}(H - H^*)^2 \leq 0$.

6.5 Continuous spectrum and multiscale systems

Chaos. The Fourier transform of a long Lorenz signal has a continuous power spectrum: no finite set of eigenvalues represents it [► V6.4 @ 0:43][► V6.4 @ 1:04]. The HAVOK method (Chapter 8) found that delay coordinates make even chaotic dynamics almost linear, with an intermittent forcing that signals the lobe switches [► V6.4 @ 1:24][► V6.4 @ 2:27][► V6.4 @ 2:48][► V6.4 @ 3:09][► V6.4 @ 3:31].

The pendulum. It frustrated Brunton that Koopman theory was applied to turbulence and the brain while nobody could write down the Koopman eigenfunctions of a pendulum [► V6.4 @ 4:12][► V6.4 @ 4:32]. The reason is a *continuous spectrum*: a small kick gives pure oscillations at the natural frequency, but the period grows with the energy and tends to infinity at the separatrix, so the frequency deforms continuously from ω_0 to 0, and there are no isolated eigenvalues [► V6.4 @ 5:13][► V6.4 @ 5:34][► V6.4 @ 5:54].

Proposition 6.6 (Frequency of the pendulum). For $\ddot{\theta} = -\sin \theta$ ($g/L = 1$) at energy $E = \frac{1}{2}\dot{\theta}^2 - \cos \theta \in (-1, 1)$, the libration frequency is

$$\omega(E) = \frac{\pi}{2K(k)}, \quad k^2 = \frac{1+E}{2},$$

with K the complete elliptic integral of the first kind; $\omega \rightarrow 1$ as $E \rightarrow -1$ and $\omega \rightarrow 0$ as $E \rightarrow 1$.

Proof. The period is $T = 4 \int_0^{\theta_m} d\theta / \sqrt{2(E + \cos \theta)}$ with $\cos \theta_m = -E$. Using $E + \cos \theta = 2(k^2 - \sin^2 \frac{\theta}{2})$ and the substitution $\sin \frac{\theta}{2} = k \sin s$ gives $T = 4K(k)$, so $\omega = 2\pi/T = \pi/(2K)$. $K(0) = \pi/2$, and $K(k) \sim \ln(4/\sqrt{1-k^2}) \rightarrow \infty$ as $k \rightarrow 1$. $\square$

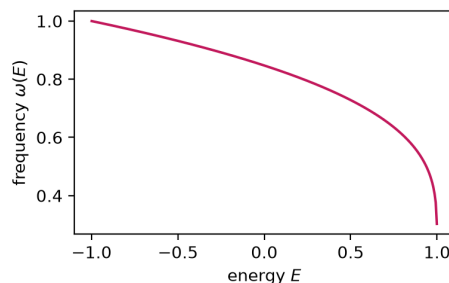


Figure 6.1: Output of `ch06_koopman.py`: the pendulum frequency $\omega(E)$ of [proposition 6.6](#), from 1 at the bottom to 0 at the separatrix. Each energy level carries its own pair of eigenvalues $\pm i\omega(E)$: a continuum.

Parametrized autoencoders. A Koopman autoencoder whose middle matrix has fixed eigenvalues $\pm i\omega$ cannot represent this; it would need all harmonics $\pm i\omega, \pm 2i\omega, \dots$ and a big latent layer [► V6.4 @ 6:35][► V6.4 @ 6:55][► V6.4 @ 7:16][► V6.4 @ 7:37][► V6.4 @ 7:57]. Bethany Lusch added an auxiliary network that learns ω as a function of the state (in effect of the energy), keeping a 2×2 Jordan-block structure with eigenvalues $\pm i\omega$ [► V6.4 @ 8:17][► V6.4 @ 8:38][► V6.4 @ 8:59]. The learned eigenfunctions map the pendulum to perfect sines and cosines whose frequency depends on the amplitude [► V6.4 @ 9:20][► V6.4 @ 9:41]; in magnitude and phase they are the action-angle coordinates of classical mechanics, as

Mezić pointed out [► V6.4 @ 10:01][► V6.4 @ 10:21]. It scales to the cylinder wake [► V6.4 @ 10:42] [► V6.4 @ 11:03].

INTUITION: Action-angle variables are Koopman eigenfunctions

For an integrable system with actions I and angles ϑ , $\dot{I} = 0$ and $\dot{\vartheta} = \omega(I)$. So any function of I is an eigenfunction with $\lambda = 0$, and $h(I)e^{i\vartheta}$ satisfies $\frac{d}{dt} = i\omega(I)h(I)e^{i\vartheta}$: an eigenfunction only on each torus, with an eigenvalue that depends on the torus. That is the continuous spectrum, and it is why the auxiliary network must output $\omega(I)$.

Continuous spectra are ubiquitous: most systems of interest have eigenvalues that deform continuously with energy or control inputs [► V6.4 @ 11:24][► V6.4 @ 11:44]. The two leading tools are delay coordinates and deep (parametrized) autoencoders [► V6.4 @ 12:04] [► V6.4 @ 12:25].

Multiscale systems. Kathleen Champion: DMD on delay coordinates gives nearly perfect closed linear models of quasi-periodic van der Pol, Lorenz and Rössler oscillators, improving with the rank and the delay [► V6.5 @ 1:04][► V6.5 @ 1:25][► V6.5 @ 1:46]; with a slow and a fast van der Pol oscillator coupled, it separates the time scales from the single measurement $x_1 + x_2$ [► V6.5 @ 2:06][► V6.5 @ 2:27]. Kutz's multiresolution DMD pulls out intermittent phenomena such as El Niño from sea-surface temperature [► V6.5 @ 2:27] [► V6.5 @ 2:48]. Summary: regression methods (DMD, EDMD, sparse eigenfunctions, networks) find coordinates in which a nonlinear system is linear; the coordinates are a one-time cost that pays off downstream in control; open questions are scaling, where it works, and whether the up-front cost is worth it [► V6.5 @ 3:08][► V6.5 @ 3:29][► V6.5 @ 4:10].

Exercise 6.5 (Spectrum of a signal). Compute the power spectrum of $x(t)$ for Lorenz (from `ch03_lorenz.py`) and for the pendulum at two energies. Compare the broadband Lorenz spectrum with the lines of the pendulum. What happens to the pendulum spectrum if the record is an ensemble over many energies?

KEY FORMULAS

- $\mathcal{K}g = g \circ \mathbf{F}$ (linear); generator $\mathcal{L}g = \nabla g \cdot \mathbf{f}$.
- Eigenfunctions $\mathcal{K}\varphi = \mu\varphi$, $\nabla\varphi \cdot \mathbf{f} = \lambda\varphi$; products add eigenvalues.
- Slow manifold: (x_1, x_2, x_1^2) closes, $K = \begin{pmatrix} \mu & 0 & 0 \\ 0 & \lambda & -\lambda \\ 0 & 0 & 2\mu \end{pmatrix}$; $\dot{x} = x^2$: $\varphi = e^{-1/x}$, $\lambda = 1$.
- EDMD $K = \Psi'\Psi^\dagger$; cross-validate. Control: $\dot{\varphi} = \lambda\varphi + \nabla\varphi \cdot B\mathbf{u}$.
- Pendulum: $\omega(E) = \pi/2K(k)$, continuous spectrum; action-angle variables.

Chapter 7

Koopman Observable Subspaces and Control

SOURCES FOR THIS CHAPTER

Video	Length	Content
V7.1 Koopman Observable Subspaces & Finite Linear Representations of Nonlinear Dynamics for Control	31:31	Koopman operator and invariant subspaces; the slow-manifold example; eigenfunctions from left eigenvectors; Koopman operator optimal control; when finite closure including the state is impossible (multiple fixed points); logistic map and $\dot{x} = x^2$; truncations.
V7.2 Koopman Observable Subspaces & Nonlinearization	4:38	The 3D picture: initial-condition paraboloid, slow-manifold sheet, slow Koopman subspace.
V7.3 Koopman Operator Optimal Control	10:18	The code: LQR on the linearization versus LQR on the Koopman system; the nonlinear gain; one third of the cost.

Transcripts materials/transcripts/C07-V0*.txt. **Book** Sec. 7.4; Sec. 8.4 (LQR). **Paper** S. L. Brunton, B. W. Brunton, J. L. Proctor, J. N. Kutz, *Koopman invariant subspaces and finite linear representations of nonlinear dynamical systems for control*, PLoS ONE 11 (2016) e0150171. **Code** code/ch07_koopman_lqr.py. **Order** In the playlist these videos are numbers 17, 19 and 20 (the HAVOK video 18 sits between them).

Motivation

[Chapter 6](#) sketched the program; this chapter, built on one paper, asks the precise question: when does a nonlinear system admit a *finite-dimensional, exact* linear representation that *also contains the state itself*? When it does, linear optimal control applies verbatim and produces a nonlinear optimal controller [\[► V7.1 @ 0:08\]](#)[\[► V7.1 @ 0:30\]](#). The answer is “rarely”, and the reason is topological.

7.1 Koopman invariant subspaces that contain the state

Recap [\[► V7.1 @ 0:51\]](#)–[\[► V7.1 @ 6:45\]](#): for $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x})$ with flow map $\mathbf{F}_t$, the Koopman operator $\mathcal{K}_t g = g \circ \mathbf{F}_t$ is linear and infinite dimensional; in continuous time $\dot{g} = \mathcal{K}g$ with the infinitesimal generator. Interest revived with Mezić (2005) and with the 2009–2010 links to fluid data [\[► V7.1 @ 1:53\]](#)[\[► V7.1 @ 2:14\]](#).

The picture. Measure m observables $\mathbf{y} = \mathbf{g}(\mathbf{x})$ (e.g. $x_1, x_1^2, x_2 x_3$). We want a matrix K with $\mathbf{y}_{k+1} = K\mathbf{y}_k$ whenever $\mathbf{x}_{k+1} = \mathbf{F}(\mathbf{x}_k)$ [\[► V7.1 @ 7:06\]](#)[\[► V7.1 @ 7:47\]](#). This is possible exactly

when the span of $\mathbf{g}$ is a *Koopman-invariant subspace*; eigenfunctions span such subspaces, but they are hard to find [► V7.1 @ 8:08][► V7.1 @ 8:49][► V7.1 @ 9:09]. For control we also want the state $\mathbf{x}$ itself among the observables, since the cost penalizes $\mathbf{x}$ [► V7.1 @ 9:30][► V7.1 @ 9:50].

Proposition 7.1 (What closure with the state requires). Let $\mathbf{y} = (\mathbf{x}, g_{n+1}(\mathbf{x}), \dots, g_m(\mathbf{x}))$ satisfy $\dot{\mathbf{y}} = K\mathbf{y}$. Then

- (i) every component of $\mathbf{f}$ lies in the span of $\mathbf{y}$: $\mathbf{f}(\mathbf{x}) = K_{1:n,:} \mathbf{y}(\mathbf{x})$;
- (ii) the derivatives $\nabla g_j \cdot \mathbf{f}$ of the added observables lie in the span too [► V7.1 @ 21:11][► V7.1 @ 21:31][► V7.1 @ 22:12];
- (iii) the left eigenvectors $\mathbf{c}$ of K , $\mathbf{c}^\top K = \lambda \mathbf{c}^\top$, give Koopman eigenfunctions $\varphi = \mathbf{c}^\top \mathbf{y}(\mathbf{x})$ [► V7.1 @ 15:00].

Proof. (i) and (ii) are the first n and the remaining rows of $\dot{\mathbf{y}} = K\mathbf{y}$. (iii) $\dot{\varphi} = \mathbf{c}^\top \dot{\mathbf{y}} = \mathbf{c}^\top K\mathbf{y} = \lambda \mathbf{c}^\top \mathbf{y} = \lambda \varphi$. $\square$

CASE STUDY: The slow-manifold system, again

For (6.3), $\mathbf{y} = (x_1, x_2, x_1^2)$ closes with $K = \begin{pmatrix} \mu & 0 & 0 \\ 0 & \lambda & -\lambda \\ 0 & 0 & 2\mu \end{pmatrix}$ [► V7.1 @ 10:31][► V7.1 @ 11:54][► V7.1 @ 12:35][► V7.1 @ 13:17]. Item (i) is why x_1^2 is needed (it appears in f_2), item (ii) is the “lucky” fact that $\frac{d}{dt}x_1^2 = 2\mu x_1^2$ closes [► V7.1 @ 22:33]. Trajectories of the 3D linear system, projected on (y_1, y_2) , reproduce the nonlinear dynamics exactly [► V7.1 @ 13:38].

The 3D picture [► V7.2 @ 1:26]–[► V7.2 @ 4:33]. In (y_1, y_2, y_3) :

- the *blue* paraboloid $y_3 = y_1^2$ is where physical initial conditions must start (the third coordinate must equal the square of the first) [► V7.2 @ 2:07][► V7.2 @ 2:27];
- the *red* parabolic sheet $y_2 = y_1^2$ is the slow manifold, extended in y_3 [► V7.2 @ 2:07];
- the *green* plane is the slow subspace of K , spanned by the eigenvectors of μ and 2μ ; white trajectories of the linear system fall quickly onto it and slide to the origin [► V7.2 @ 2:48][► V7.2 @ 3:09].

The green plane does not quite pass through the intersection of red and blue; as λ becomes faster it tilts up until it does [► V7.2 @ 3:30][► V7.2 @ 3:51]. Indeed the eigenvector of 2μ is $(0, \frac{\lambda}{\lambda-2\mu}, 1)$, so the green plane is $y_2 = \frac{\lambda}{\lambda-2\mu}y_3$, which tends to $y_2 = y_3$ for $|\lambda| \gg |\mu|$.

Exercise 7.1 (Which systems close?). Decide whether a finite Koopman-invariant subspace containing the state exists, and find it if so: (a) $\dot{x}_1 = \mu x_1$, $\dot{x}_2 = \lambda(x_2 - x_1^3)$; (b) $\dot{x}_1 = \mu x_1$, $\dot{x}_2 = \lambda x_2 + x_1 x_2$; (c) $\dot{x} = x - x^3$. For (b), consider the observable $x_2 e^{-x_1/\mu}$.

Exercise 7.2 (Slow subspace). Compute the eigenvectors of K for the slow-manifold system and show that the slow (green) subspace is $y_2 = \frac{\lambda}{\lambda-2\mu}y_3$. Plot its intersection with $y_3 = y_1^2$ for $\lambda = -1, -10, -100$ ($\mu = -0.1$) and compare with $y_2 = y_1^2$.

7.2 Koopman operator optimal control

Add a control on x_2 : $\dot{x}_2 = \lambda(x_2 - x_1^2) + u$. The lifted system is $\dot{\mathbf{y}} = K\mathbf{y} + B u$ with $B = (0, 1, 0)^\top$: same K , input only on y_2 [► V7.1 @ 16:02][► V7.1 @ 16:22].

Definition 7.2 (New concepts: LQR). For $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$ the **linear quadratic regulator** minimizes $J = \int_0^\infty (\mathbf{x}^\top \mathbf{Q}\mathbf{x} + \mathbf{u}^\top \mathbf{R}\mathbf{u}) dt$; the optimum is the proportional feedback $\mathbf{u} = -\mathbf{C}\mathbf{x}$, $\mathbf{C} = \mathbf{R}^{-1}\mathbf{B}^\top \mathbf{P}$, where $\mathbf{P}$ solves the algebraic Riccati equation $\mathbf{A}^\top \mathbf{P} + \mathbf{P}\mathbf{A} - \mathbf{P}\mathbf{B}\mathbf{R}^{-1}\mathbf{B}^\top \mathbf{P} + \mathbf{Q} = 0$. $\mathbf{Q}$ weights deviations of the state, $\mathbf{R}$ control expenditure [► V7.1 @ 16:42][► V7.1 @ 17:03][► V7.3 @ 3:12][► V7.3 @ 3:32]. (Named for the linear dynamics and the quadratic cost; Kalman, 1960.)

Two controllers [► V7.3 @ 2:10]–[► V7.3 @ 8:01]:

1. **naive**: linearize at the origin, $\mathbf{A} = \begin{pmatrix} \mu & 0 \\ 0 & \lambda \end{pmatrix}$, $\mathbf{Q} = \mathbf{I}$, $\mathbf{R} = 1$, one call to `lqr`, $\mathbf{u} = -\mathbf{C}\mathbf{x}$ [► V7.3 @ 2:31][► V7.3 @ 3:53][► V7.3 @ 4:13];
2. **Koopman**: LQR on $(\mathbf{K}, \mathbf{B})$ with $\mathbf{Q} = \text{diag}(1, 1, 0)$ (the *same* cost: no penalty on the fictitious y_3) [► V7.3 @ 5:36][► V7.3 @ 5:56]. The gain has three entries, so

$$u = -c_1 x_1 - c_2 x_2 - c_3 x_1^2, \quad (7.1)$$

a *nonlinear* feedback obtained from a linear method [► V7.1 @ 17:44][► V7.1 @ 18:05][► V7.3 @ 6:57][► V7.3 @ 7:40].

In the video the Koopman controller reaches the origin with less deviation and about one third of the total cost [► V7.1 @ 18:25][► V7.1 @ 18:46][► V7.3 @ 8:42][► V7.3 @ 9:03][► V7.3 @ 9:23], and seems to work even from very large initial conditions, as if globally optimal [► V7.3 @ 9:44].

PYTHON: LQR on the linearization vs on the Koopman system (./code/ch07_koopman_lqr.py)

```
import numpy as np
from scipy.integrate import solve_ivp
from scipy.linalg import solve_continuous_are, expm

mu, lam = -0.1, 1.0                                # x2 direction unstable: control is needed
B = np.array([[0.0], [1.0]])                        # u acts on x2

def f(t, x, gain):                                  # x1' = mu x1, x2' = lam (x2 - x1^2) + u
    u = -gain(x)
    return [mu * x[0], lam * (x[1] - x[0]**2) + u, x[0]**2 + x[1]**2 + u**2]    # 3rd:
    # running cost

def lqr(A, B, Q, R):
    P = solve_continuous_are(A, B, Q, R)
    return np.linalg.solve(R, B.T @ P)              # u = -C x

R = np.array([[1.0]])
# (1) LQR on the linearization at the origin: A = df/dx(0)
C1 = lqr(np.array([[mu, 0], [0, lam]]), B, np.eye(2), R)
# (2) LQR on the Koopman linear system y = (x1, x2, x1^2), same cost (no weight on y3)
K = np.array([[mu, 0, 0], [0, lam, -lam], [0, 0, 2 * mu]])
C2 = lqr(K, np.vstack([B, [[0.0]]]), np.diag([1.0, 1.0, 0.0]), R)
print('linearized LQR gains:', C1.round(3), '\nKoopman LQR gains   :', C2.round(3))

x0 = [-5.0, 5.0, 0.0]                               # large initial condition, as in the
# paper
for name, gain in [('linearized', lambda x: (C1 @ x[:2]).item()),
                   ('Koopman',    lambda x: (C2 @ [x[0], x[1], x[0]**2]).item())]:
    s = solve_ivp(f, (0, 50), x0, args=(gain,), rtol=1e-10, atol=1e-10)
    print(f'{name} total cost J = {s.y[2, -1]:.1f}')
```

```
# truncated Koopman models of  $x' = x^2$  in the basis  $x, x^2, \dots, x^N$ :  $y_{k'} = k y_{k+1}$ 
x0, t = 0.5, np.linspace(0, 1.8, 10) # blow-up at  $t = 1/x_0 = 2$ 
for N in (1, 2, 4, 8, 16):
    Kn = np.diag(np.arange(1, N), 1).astype(float)
    print(f'N={N:2d}:  $x(1.8) =$ ', (expm(Kn * 1.8) @ x0 ** np.arange(1, N + 1))[0].round(3))
print('exact :  $x(1.8) =$ ', round(x0 / (1 - x0 * 1.8), 3))
```

With the parameters of the paper, $\mu = -0.1$, $\lambda = 1$, $x_0 = (-5, 5)$, the script prints the gains (0, 2.414) and (0, 2.414, -1.496) and total costs $J = 4485$ (linearized) and $J = 990$ (Koopman), a factor ≈ 4.5 ; the exact ratio depends on the initial condition.

INTUITION: Why the extra term helps

The linearized controller sees only $\dot{x}_2 = \lambda x_2 + u$ and fights the term $-\lambda x_1^2$ as an unknown disturbance. The Koopman controller “knows” that x_2 is pulled towards x_1^2 : with $c_3 < 0$ the feedback $-c_3 x_1^2$ cancels part of $-\lambda x_1^2$, so it cooperates with the slow manifold instead of fighting it (section 6.4).

Exercise 7.3 (Cost versus initial condition). Modify `ch07_koopman_lqr.py` to compute the ratio of the two costs on a grid of initial conditions in $[-5, 5]^2$ and plot it. Where does the Koopman controller help most? Does the linearized controller ever fail to stabilize?

7.3 When finite closure is impossible

Proposition 7.3 (Topological obstruction). *If $\dot{x} = f(x)$ has more than one isolated fixed point, or a periodic orbit, or a more complicated attractor (e.g. Lorenz), it admits no finite-dimensional linear system $\dot{y} = Ky$, $y = g(x)$, with x among the components of y* [► V7.1 @ 22:55] [► V7.1 @ 23:15].

This is disappointing but expected: we traded nonlinearity for infinite dimension, and non-trivial dynamics really uses it [► V7.1 @ 24:18]. There may still be special eigenfunctions spanning finite subspaces, but they will not include the state [► V7.1 @ 26:43].

The logistic map. Try $y_k = (x_k, x_k^2)$ for $x_{k+1} = rx_k(1 - x_k)$: already $x_{k+1}^2 = r^2(x_k - x_k^2)^2$ contains x_k^4 ; then x_k^8 , and so on: a cascade of complexity [► V7.1 @ 25:01][► V7.1 @ 25:42] [► V7.1 @ 26:02][► V7.1 @ 26:22]. In the monomial basis the rows of K are shifted rows of Pascal’s triangle times powers of r [► V7.1 @ 27:04]: indeed $x_{k+1}^j = r^j x_k^j (1 - x_k)^j = r^j \sum_i \binom{j}{i} (-1)^i x_k^{j+i}$. Truncations to 5×5 have little predictive value; no good finite approximation was found [► V7.1 @ 27:45][► V7.1 @ 28:05].

$\dot{x} = x^2$, **truncated.** A single fixed point, but finite-time blow-up [► V7.1 @ 28:27]. In the basis $y_k = x^k$, $\dot{y}_k = kx^{k-1}\dot{x} = ky_{k+1}$: K has $1, 2, 3, \dots$ on the superdiagonal [► V7.1 @ 28:47] [► V7.1 @ 29:08]. Truncating to $N \times N$ gives better and better approximations as N grows [► V7.1 @ 29:29][► V7.1 @ 29:49][► V7.1 @ 30:10]. The script, for $x_0 = 0.5$ (blow-up at $t = 2$), gives $x(1.8) \approx 0.5, 0.95, 1.72, 2.85, 4.07$ for $N = 1, 2, 4, 8, 16$, against the exact 5: the truncation is the Taylor polynomial of $x_0/(1 - x_0 t)$ in t , which converges for $t < 2$, ever more slowly near the singularity.

Sketch (the argument of the paper, not a full proof). Since x is a component of y , the map g is injective and conjugates the nonlinear flow to the linear flow restricted to the image $g(M)$, which is invariant. A fixed point $\bar{x}$ maps to a y with $Ky = 0$, i.e. to $\ker K$, a linear subspace; the fixed points of a linear flow form a connected set, and the linear flow near $\ker K$ cannot reproduce two separated basins (a rigorous version must handle the restriction to the curved image $g(M)$); a linear flow has no isolated periodic orbits (limit cycles) and no strange attractors, since solutions are sums of $e^{\lambda t}$ terms. The video’s phrasing: the flows cannot be topologically conjugate, one cannot be continuously deformed into the other [► V7.1 @ 23:36][► V7.1 @ 23:57]. □

Exercise 7.4 (Logistic-map Koopman matrix). Write the first four rows of K in the basis $1, x, x^2, \dots, x^8$ for the logistic map. Truncate to the first N monomials, iterate from $x_0 = 0.3$ for $r = 2.5$ (a stable fixed point) and $r = 3.9$ (chaos), and compare with the true orbit.

KEY FORMULAS

- Closure with the state: $\mathbf{f} \in \text{span } \mathbf{y}$ and $\nabla g_j \cdot \mathbf{f} \in \text{span } \mathbf{y}$; eigenfunctions from left eigenvectors, $\varphi = \mathbf{c}^\top \mathbf{y}$.
- Koopman LQR: LQR on (K, B) , $Q = \text{diag}(1, 1, 0) \Rightarrow u = -c_1 x_1 - c_2 x_2 - c_3 x_1^2$.
- Obstruction: several fixed points, limit cycles, chaos $\Rightarrow$ no finite closure containing $\mathbf{x}$.

Chapter 8

Hankel Alternative View of Koopman (HAVOK)

SOURCES FOR THIS CHAPTER

Video	Length	Content
V8.1 Hankel Alternative View of Koopman (HAVOK) Analysis [FULL]	47:07	Chaos; Koopman invariant subspaces; Hankel matrix and eigen time-delay coordinates; the Koopman reading of the Hankel matrix; the forced linear model; intermittent forcing and lobe switching; Perron–Frobenius almost-invariant sets; magnetic field and measles; the structure of the model, integer coefficients.
V8.2 Hankel Alternative View of Koopman (HAVOK) Analysis [SHORT]	22:41	The same method in short; choice of r by the optimal hard threshold; only what is new is cited.
V8.3 Magnetic field reversal and Measles outbreaks: HAVOK models of chaos	9:39	The two real-world examples in more detail.
V8.4 Linear model for chaotic Lorenz system [HAVOK]	5:27	The sparse antisymmetric structure of A ; the integer-coefficient model.

Transcripts materials/transcripts/C08-V0*.txt. **Book** Sec. 7.5 (“Time-delay coordinates”, HAVOK). **Paper** S. L. Brunton, B. W. Brunton, J. L. Proctor, E. Kaiser, J. N. Kutz, *Chaos as an intermittently forced linear system*, Nat. Commun. 8 (2017) 19. **Code** code/ch08_havok.py (original Matlab code: faculty.washington.edu/sbrunton/HAVOK.zip). The SHORT video repeats the FULL one: it is cited only where it adds something.

Motivation

The last chapter brings together everything: delay coordinates ([section 5.3](#)), Koopman invariant subspaces ([chapter 7](#)), DMD/SINDy regression ([chapters 4](#) and [5](#)) and chaos ([chapter 2](#)). The result: a chaotic system, observed through a single time series, is well described as a *linear* system driven by an *intermittent forcing*, and the forcing announces rare nonlinear events, such as lobe switching in Lorenz, before they happen [[► V8.1 @ 0:02](#)] [[► V8.1 @ 0:22](#)]. [Proposition 7.3](#) said that no finite linear model containing the state can reproduce several fixed points; HAVOK sidesteps it by letting a forcing carry the essential nonlinearity.

8.1 Chaos and the Koopman view

Chaotic systems are everywhere: climate, turbulence, the stock market, epidemics, planetary motion [► V8.1 @ 0:44]. Linear systems have the full toolbox of prediction, estimation and control; nonlinear ones have multiple fixed points (the double well), periodic orbits, and with more nonlinearity chaos [► V8.1 @ 2:25][► V8.1 @ 2:46][► V8.1 @ 3:07]. Two double pendulums released side by side from nearly the same state stay synchronized for a while, then do completely different acrobatics [► V8.1 @ 3:48][► V8.1 @ 4:09]. A cube of initial conditions in Lorenz is stretched and folded “like taffy” over the attractor [► V8.1 @ 4:50][► V8.1 @ 5:11][► V8.1 @ 5:32].

INTUITION: Chaos is not randomness

The smeared cube is not random: it forms structured, coherent patterns. Chaos balances order and structure against uncertainty amplification in forward time [► V8.1 @ 5:52][► V8.1 @ 6:12][► V8.2 @ 1:05]. That structure is what HAVOK exploits.

Koopman recap: propagate measurements $g(x)$ linearly, at the price of infinite dimension; a Koopman-invariant subspace gives a matrix, but such subspaces are hard to find [► V8.1 @ 6:53][► V8.1 @ 7:55][► V8.1 @ 9:17][► V8.1 @ 9:58]. HAVOK is an alternative way to get one for chaotic systems; the acronym “goes nicely with chaos” [► V8.1 @ 10:19][► V8.1 @ 10:41].

8.2 Delay coordinates and the Koopman reading of the Hankel matrix

Embedding. Measure only $x(t)$ of Lorenz: it sits on one lobe, switches to the other, unpredictably [► V8.1 @ 11:23][► V8.1 @ 11:43]. Build the Hankel matrix (5.3) from time-shifted copies of the series [► V8.1 @ 12:04][► V8.1 @ 12:24] and take its SVD, $H = U\Sigma V^*$. The columns of U and V are ordered by singular value: the first ones are the time series most self-similar to the measurement [► V8.1 @ 12:45][► V8.1 @ 13:06][► V8.1 @ 13:27]. The idea is old: the eigensystem realization algorithm (1985), singular spectrum analysis in climate, and the recent nonlinear Laplacian spectral analysis of Giannakis and Majda [► V8.1 @ 13:48][► V8.1 @ 14:09]. Plotting (v_1, v_2, v_3) gives an attractor diffeomorphic to Lorenz’s, by Takens’ theorem (theorem 5.4); x is a good measurement, z would be a bad one (it cannot distinguish the two lobes, by the symmetry $(x, y, z) \mapsto (-x, -y, z)$) [► V8.1 @ 14:29][► V8.1 @ 14:50][► V8.1 @ 15:11].

The Koopman reading. Since $x(t_{k+1}) = \mathcal{K}x(t_k)$, the Hankel matrix is a Krylov sequence of the Koopman operator [► V8.1 @ 15:53][► V8.1 @ 16:13][► V8.1 @ 16:33]:

$$H = \begin{bmatrix} x(t_1) & x(t_2) & \cdots \\ x(t_2) & x(t_3) & \cdots \\ \vdots & & \end{bmatrix} = \begin{bmatrix} x(t_1) & \mathcal{K}x(t_1) & \mathcal{K}^2x(t_1) & \cdots \\ \mathcal{K}x(t_1) & \mathcal{K}^2x(t_1) & \cdots & \\ \vdots & & & \end{bmatrix}. \quad (8.1)$$

Thousands of columns, but numerically of low rank: for Lorenz the first $r = 15$ columns of U express every column of H to almost machine precision [► V8.1 @ 16:54][► V8.1 @ 17:15][► V8.1 @ 17:37]. Each column is $\mathcal{K}$ applied to the previous one, so the span of the leading columns of U is (approximately) *Koopman invariant*: a data-driven invariant measurement subspace of eigen time-delay coordinates [► V8.1 @ 17:58][► V8.2 @ 12:07][► V8.2 @ 12:27]. In the SHORT video $r = 15$ is chosen with the optimal hard threshold of Gavish and Donoho [► V8.2 @ 18:21].

Remark 8.1 (Why chaos helps). This works best for chaotic systems: the trajectory never returns exactly and densely explores the attractor, never stuck at a fixed point or a periodic orbit, so one global model for the attractor makes sense, and more data make the linear dependence better [► V8.1 @ 18:19][► V8.1 @ 18:39][► V8.1 @ 19:00]. It holds for data on the attractor, not for every initial condition [► V8.2 @ 12:27].

◊ **BEYOND THE CORE: Singular values of the Lorenz Hankel matrix**

In `ch08_havok.py` ($dt = 0.001$, $q = 100$ delays, $t \in [0, 200]$) the singular values relative to the first are 1, 0.15, 0.02, 2.4×10^{-3} , 2.8×10^{-4} , 3×10^{-5} , $\dots$: a geometric decay by roughly a factor 8 each. For a short window ($q \, dt = 0.1$) the delay vector is essentially a Taylor expansion of $x(t)$ around the window, which is why the modes u_k look like Legendre polynomials (see [section 8.5](#)) and why the singular values decay geometrically.

Exercise 8.1 (Choice of q and r). Run `ch08_havok.py` with $q = 50, 100, 200$ and $r = 11, 15, 21$. How do the singular values, the fit residuals and the correlation of the model change? Use the Gavish–Donoho optimal hard threshold ($\approx 2.858 \sigma_{\text{med}}$ for a square-ish matrix with unknown noise; for a $q \times m$ matrix use their formula $\omega(\beta)$) to pick r objectively.

Exercise 8.2 (A bad measurement). Repeat the analysis with $z(t)$ instead of $x(t)$. Explain, using the symmetry $(x, y, z) \mapsto (-x, -y, z)$, why the embedded attractor cannot be diffeomorphic to Lorenz and what happens to the forcing.

8.3 Chaos as an intermittently forced linear system

Regress the time derivatives of the eigen time-delay coordinates on the coordinates themselves, as in DMD or SINDy but on $\mathbf{v} = (v_1, \dots, v_r)$ [► V8.1 @ 19:41][► V8.1 @ 20:01][► V8.1 @ 20:22]. The first $r - 1$ equations fit very well; the last one, for v_r , fits terribly, even with nonlinear terms [► V8.1 @ 20:43][► V8.1 @ 21:05][► V8.1 @ 21:25][► V8.1 @ 21:46]. So treat v_r as an external input [► V8.1 @ 22:06]:

$$\frac{d}{dt} \begin{bmatrix} v_1 \\ \vdots \\ v_{r-1} \end{bmatrix} = A \begin{bmatrix} v_1 \\ \vdots \\ v_{r-1} \end{bmatrix} + B v_r(t). \quad (8.2)$$

Definition 8.2 (New concepts: HAVOK). The **Hankel alternative view of Koopman** (HAVOK) model of a scalar time series is (8.2): a linear system in the first $r - 1$ eigen time-delay coordinates, forced by the r -th, obtained by least-squares regression. A **Hankel** matrix (after Hermann Hankel, 1839–1873) is constant along its anti-diagonals.

What the model does. Fed with the measured forcing v_r (in red), the linear model reproduces the dynamics (blue over the white data) and the attractor; and the forcing bursts exactly when the trajectory switches lobes [► V8.1 @ 22:48][► V8.1 @ 23:08][► V8.1 @ 23:29][► V8.1 @ 23:50]. Its statistics are strongly non-Gaussian: long tails, rare extreme events driving the switches, as in the work of Sapsis, Majda and co-workers on intermittency in climate, ocean waves and turbulence [► V8.1 @ 24:10][► V8.1 @ 24:30][► V8.1 @ 24:51].

PYTHON: HAVOK on the Lorenz system (../code/ch08_havok.py)

```

import numpy as np
from scipy.integrate import solve_ivp
from scipy.signal import lsim
import matplotlib.pyplot as plt

def lorenz(t, x, s=10, r=28, b=8/3):
    return [s * (x[1] - x[0]), x[0] * (r - x[2]) - x[1], x[0] * x[1] - b * x[2]]

dt = 0.001
t = np.arange(0, 200, dt)
x = solve_ivp(lorenz, (0, 200), [-8, 8, 27], t_eval=t, method='DOP853', rtol=1e-12,
              atol=1e-12).y[0]

# Hankel matrix of the single measurement x(t): q delayed copies as rows
q, r = 100, 15
H = np.array([x[i:len(x) - q + i + 1] for i in range(q)])          # q x (m - q + 1)
U, S, Vh = np.linalg.svd(H, full_matrices=False)                  # eigen time-delay
V = Vh[:r].T                                                       # eigen time-delay
                           coordinates v1..vr
print('first singular values / s1:', (S[:r + 2] / S[0]).round(5))

# regression dv/dt = Xi v; the first r-1 rows are the linear model, v_r is the forcing
dV = (V[2:] - V[:-2]) / (2 * dt)                                  # central
                           differences
Vc = V[1:-1]
Xi = np.linalg.lstsq(Vc, dV, rcond=None)[0].T                     # r x r
A, B = Xi[:r - 1, :r - 1], Xi[:r - 1, r - 1:]
res = lambda k: np.linalg.norm(dV[:, k] - Vc @ Xi[k]) / np.linalg.norm(dV[:, k])
print('relative fit residual: rows 1..r-1 max =', max(res(k) for k in range(r - 1)).
      round(4),
      ' row r =', res(r - 1).round(4))
print('A super-diagonal:', np.diag(A, 1)[:8].round(2))             # ~ multiples of 5
                           (antisymmetric)

# simulate the forced linear model with the measured forcing v_r
n = 50000                                                           # first 50 time
                           units
tt = np.arange(n) * dt
_, Y, _ = lsim((A, B, np.eye(r - 1), 0 * B), V[:n, r - 1], tt, X0=V[0, :r - 1])
print('v1 model vs data, correlation:', np.corrcoef(Y[:, 0], V[:n, 0])[0, 1].round(4))

# forcing active (v_r^2 above a threshold) vs lobe switching (sign changes of x)
active = V[:, r - 1]**2 > 2e-5
fig, ax = plt.subplots(3, 1, figsize=(8, 5), sharex=True)
ax[0].plot(tt, V[:n, 0], 'k', lw=0.8, label='data'); ax[0].plot(tt, Y[:, 0], '--',
                        color='#1565C0', lw=0.8, label='model')
ax[0].set_ylabel('$v_1$'); ax[0].legend(loc='upper right', fontsize=7)
ax[1].plot(tt, V[:n, r - 1], color='#D32F2F', lw=0.6); ax[1].set_ylabel('$v_r$ (
                        forcing)')
ax[2].plot(tt, x[:n], 'k', lw=0.6)
ax[2].plot(tt[active[:n]], x[:n][active[:n]], '.', color='#D32F2F', ms=1)
ax[2].set_ylabel('$x$'); ax[2].set_xlabel('$t$')
plt.tight_layout(); plt.savefig('lecture-notes/figures/ch08-havok.png', dpi=200)

plt.figure(figsize=(3.2, 3)); m = abs(Xi).max()
plt.imshow(Xi[:r - 1], cmap='RdBu_r', vmin=-m, vmax=m); plt.title('[A | B]', fontsize
=9)
plt.xticks([]); plt.yticks([]); plt.tight_layout()
plt.savefig('lecture-notes/figures/ch08-havok-A.png', dpi=200)

```

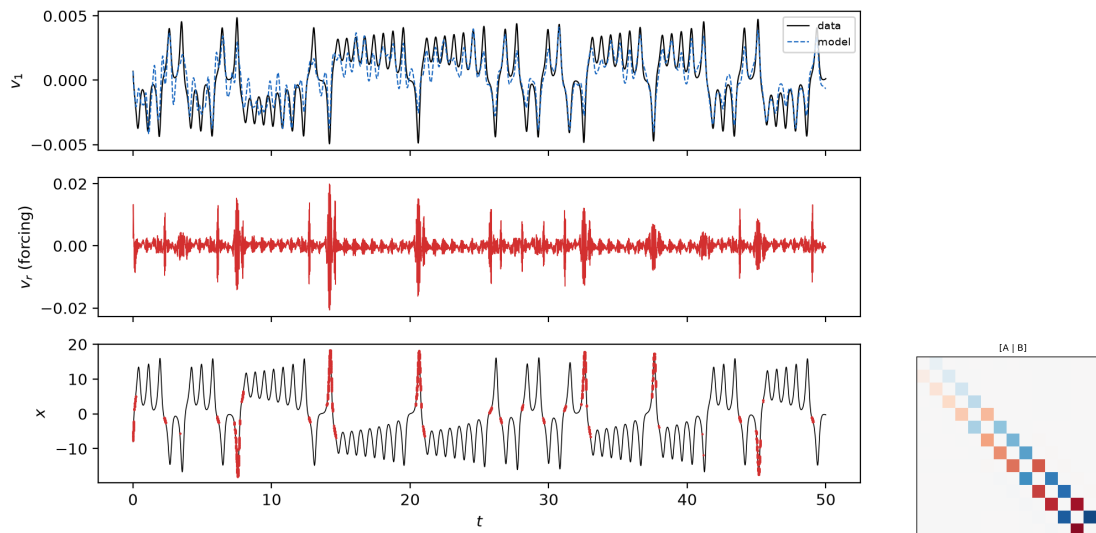


Figure 8.1: Output of `ch08_havok.py`. Top: v_1 from the data (black) and from the linear model (8.2) driven by the measured forcing (blue dashed); correlation 0.92 over 50 time units. Middle: the forcing v_r , quiet with intermittent bursts. Bottom: $x(t)$, red where v_r^2 exceeds a threshold: the bursts sit at the lobe switches. Right: the matrix $[A | B]$, sparse, antisymmetric, nonzero only next to the diagonal.

The script prints a relative residual of 0.97 for the v_r row, i.e. it is not fitted at all, against at most 0.36 for the other rows (and much less for the leading ones).

Prediction. v_r cannot be predicted, but it can be *measured* in real time: project the last window of data onto the columns of U [► V8.1 @ 27:16][► V8.1 @ 27:37]. When v_r^2 exceeds a threshold, colour the trajectory red: the red flag rises before the switch, about one revolution ahead [► V8.1 @ 27:57][► V8.1 @ 28:18][► V8.1 @ 28:38][► V8.1 @ 28:59]. It is “not perfect”, not every switch is caught [► V8.2 @ 19:23]. On the embedded attractor, white (small forcing) regions are one big linear Koopman model surrounding the three fixed points; red regions, coherent and not random, mark where essential nonlinearity acts [► V8.1 @ 29:19][► V8.1 @ 29:40][► V8.1 @ 30:01][► V8.1 @ 30:21]. A typical sequence: four inner revolutions on the right lobe (white), the flag goes up at point B, one more revolution, switch to the left lobe; there the forcing drops, three revolutions, flag at D, switch back [► V8.1 @ 33:08][► V8.1 @ 33:29][► V8.1 @ 33:49].

Remark 8.3 (Perron–Frobenius almost-invariant sets). The Perron–Frobenius operator, which advances densities, is dual to Koopman. Its almost-invariant sets for Lorenz are two interleaved regions (the inner part of one lobe joined to the outer band of the other), and their outer boundaries line up with the red regions of HAVOK: two very different analyses agree [► V8.1 @ 30:42][► V8.1 @ 31:02][► V8.1 @ 31:43]. That the linear part fails somewhere is unavoidable: a finite linear model has one fixed point, Lorenz has three [► V8.1 @ 32:27][► V8.1 @ 32:47].

Exercise 8.3 (Precursor statistics). Using the script, define switches as sign changes of x that persist, and red-flag events as upward crossings of the threshold by v_r^2 . Compute the fraction of switches preceded by a flag within one revolution and the false-alarm rate, as functions of the threshold. Remember that $v_r(t_i)$ uses the window $[t_i, t_i + q \, dt]$: shift it to make the forecast causal.

8.4 Real systems: magnetic field reversal and measles

HAVOK has been applied to Rössler, the Mackey–Glass delay equation, the double pendulum, a model of Earth’s magnetic field, electrocardiogram (ECG) and sleep electroencephalogram (EEG) recordings, and measles in New York City; all the code is online [► V8.1 @ 25:11][► V8.1 @ 25:32][► V8.1 @ 25:53][► V8.1 @ 26:14].

CASE STUDY: Reversals of Earth’s magnetic field

A simulation of averaged magnetohydrodynamic equations, forced by turbulent fluctuations in the Earth’s interior, reproduces geomagnetic reversals [► V8.1 @ 34:30][► V8.1 @ 34:50][► V8.3 @ 3:07]. The embedded attractor has two clusters, North pole up and North pole down, with long stays and sudden jumps; reversals are recorded in the Earth’s crust [► V8.1 @ 35:11][► V8.1 @ 35:31]. Colouring red where v_r^2 exceeds the threshold, the red segments are the reversals [► V8.1 @ 36:12]; there are also near-reversals that come back [► V8.3 @ 4:10]. Zooming in: just before a flip, v_1 is still within its normal fluctuations, and no one could tell by eye, but the forcing jumps “clear as day” [► V8.1 @ 36:53][► V8.1 @ 37:13][► V8.3 @ 5:52]. Not 100%, but a real heads-up [► V8.3 @ 6:13].

CASE STUDY: Measles in New York City

Biweekly measles cases in New York, 1928–1964, a classic of delay embedding since Sugihara and May (Nature, 1990) [► V8.1 @ 37:35][► V8.3 @ 6:33]. In eigen time-delay coordinates the quiet state is the origin and each outbreak is a burst-and-return loop; the bigger the loop, the worse the outbreak [► V8.1 @ 37:56][► V8.1 @ 38:16]. Large outbreaks come with large forcing; the largest outbreak has the largest forcing event [► V8.1 @ 38:36]. The method ignores small dips in the data that might have been false alarms and waits for the real precursor [► V8.1 @ 38:57][► V8.3 @ 7:56]; the first moments of the forcing are already larger for the worst outbreak, hinting at a prediction of severity [► V8.1 @ 39:18][► V8.3 @ 8:16][► V8.3 @ 8:36].

8.5 The structure of the HAVOK model

The Lorenz HAVOK matrix has a striking structure [► V8.1 @ 39:59][► V8.4 @ 0:44][► V8.4 @ 1:05]:

- *sparse*: nonzero only on the first super- and sub-diagonals [► V8.1 @ 40:20][► V8.1 @ 41:02];
- *antisymmetric*: entries come in equal and opposite pairs [► V8.4 @ 1:25];
- the signs on one off-diagonal follow a pattern: four of one sign, then one of the other, three, one, two, one, one [► V8.1 @ 41:22][► V8.4 @ 3:08];
- the coefficients are nearly integer multiples of 5: $\pm 4.9, \pm 9.9, \pm 15.1, \dots$ [► V8.1 @ 41:43][► V8.4 @ 2:27][► V8.4 @ 2:47].

It was discovered by applying SINDy to delay coordinates: even with nonlinear terms allowed, the most parsimonious model was this sparse linear one, which suggested the connection to Koopman [► V8.1 @ 40:20][► V8.1 @ 40:41][► V8.4 @ 1:45][► V8.4 @ 2:07]. The script reproduces it: the super-diagonal of A is $-5.18, -9.88, -13.63, -18.76, +23.25, -28.62, -33.47, -38.88$: multiples of 5 (approximately) with a sign flip after the fourth.

HISTORY: Sugihara and May

George Sugihara and Robert May, *Nonlinear forecasting as a way of distinguishing chaos from measurement error in time series*, Nature 344 (1990) 734–741, used delay embeddings and nearest-neighbour forecasts on the New York measles and chickenpox records: measles forecasts degraded with the horizon as for low-dimensional chaos, chickenpox looked like noise around a seasonal cycle. May is the same ecologist who popularized the logistic map (section 3.3).
References: the paper cited; Brunton et al., Nat. Commun. 8 (2017).

Sketch. Write $x(t+s) = \sum_k v_k(t)u_k(s)$ for s in the window; then $\dot{v}_k = \langle u_k, \partial_s x \rangle$, so A is the matrix of ∂_s in the Legendre basis, truncated. The recur-

Proposition 8.4 (Antisymmetric tridiagonal structure from Legendre polynomials). *If the delay window is short and the modes u_k are (discretized, normalized) Legendre polynomials P_{k-1} on the window, then the derivative operator in this basis is approximately tridiagonal and antisymmetric, with entries growing linearly with k .*

rence $(2n+1)P_n = P'_{n+1} - P'_{n-1}$ makes $\langle P_m, P'_n \rangle$ vanish unless $m < n$ with $m+n$ odd; a sliding window (data moving through it) symmetrizes this into the antisymmetric part, and the coefficients scale like $n/\text{window length}$, with nearest neighbours dominating. A precise derivation is in the literature on HAVOK and on the “Legendre memory” of delay embeddings. $\square$

Polynomial modes. The eigen time series that matter, the columns of U , are polynomials for almost all examples: constant, linear, parabola, cubic, $\dots$ [► V8.1 @ 42:25][► V8.1 @ 42:45]. Convolving a short sliding window of data with this basis gives the current $v_1, \dots, v_{r-1}$ and the forcing v_r (the 15th, in red) [► V8.1 @ 42:04][► V8.1 @ 43:05].

The integer model. Brunton built a “neighbouring” system by hand with exactly this pattern and coefficients 5, 10, 15, 20, -25 , 30, 35, 40, -45 , $\dots$, and drove it with the v_{15} measured from the real Lorenz system: it shadows the Lorenz dynamics, lobe switching included, not perfectly but far better than expected; nobody knows why [► V8.1 @ 43:46][► V8.1 @ 44:07][► V8.1 @ 44:27][► V8.1 @ 44:48][► V8.4 @ 3:50][► V8.4 @ 4:32].

The general lesson. From data to dynamics [► V8.1 @ 45:09][► V8.1 @ 45:30][► V8.2 @ 21:46]:

1. measure the right things (embedding theory and observability tell what is a good measurement);
2. find a representation in which the dynamics is simple: for chaotic systems, eigen time-delay coordinates, a Koopman invariant basis in which the dynamics is linear over most of phase space;
3. regression is the easy part.

Where the forcing is active the system is genuinely nonlinear, but those are exactly the interesting regions: lobe switches, reversals, outbreaks [► V8.1 @ 46:10][► V8.1 @ 46:31].

Exercise 8.4 (Legendre basis). Plot the first six columns of U against $s \in [0, q \, dt]$ and compare them with the Legendre polynomials $P_0, \dots, P_5$ mapped to the window. Compute $\langle u_m, \partial_s u_n \rangle$ numerically and compare with A .

Exercise 8.5 (Integer model). Build the antisymmetric tridiagonal matrix with super-diagonal $-5, -10, -15, -20, 25, -30, \dots$ (signs as in the script) and drive it with the measured v_r . Compare its v_1 with the data and with the fitted HAVOK model.

KEY FORMULAS

- Hankel matrix of $x(t) = \text{Krylov sequence } x, \mathcal{K}x, \mathcal{K}^2x, \dots$; $H = U\Sigma V^*$; low rank $\Rightarrow$ leading u_k span an approximately Koopman-invariant subspace.
- HAVOK: $\dot{\mathbf{v}}_{1:r-1} = A\mathbf{v}_{1:r-1} + Bv_r$; v_r intermittent, heavy-tailed, active before lobe switches.
- A : sparse, tridiagonal, antisymmetric, entries $\approx$ multiples of 5 for Lorenz; modes $\approx$ Legendre polynomials.